

OUTFINITE MATHEMATICS

A Research Programme for
Executable Science,
Resource-Aware Mathematics,
and Meta-Rational Foundations



Existence, limits, proof, scientific theories,
and machine-checkable knowledge
in an age of artificial intelligence

OUTFINITE MATHEMATICS

A Research Programme for Executable Science, Resource-Aware
Mathematics, and Meta-Rational Foundations

*Existence, limits, proof, scientific theories, and machine-checkable knowledge in an age
of artificial intelligence*

Sînică Alboaie

Conceptual research programme, 2026

Copyright © [2026] Axiologic Research

All rights reserved.

KDP publishing rights held by Outfinity SRL (Iași, Romania).

Available for free on Axiologic website (www.axiologic.net).

Author's Note

This work was created under the umbrella of Axiologic Research as part of two interconnected research programs, one dedicated to Artificial Intelligence and the other to Outfinitism, both led by Sînică Alboaie. To explore the details of how these two programs intersect and build upon each other, we invite readers to visit the Outfinity Venture Studio page at www.outfinity.ch.

While Artificial Intelligence language models were used to assist with text generation, the foundational philosophy, thematic structure, and analytical approach derive exclusively from the author's decades of dedicated study of social phenomena, social technologies, and genuine hard technologies resulting from various European and self funded research projects. Recently, within the Achilles research project, we have been deeply studying AI generated literature and its automated verification using neuro symbolic approaches; all the free books made available to the public are part of the suite of works we use to test and popularise our latest ideas and technologies.

Status and scope of the programme

Outfinitism is, at present, primarily a philosophical position. It begins from a simple suspicion: several distinctions that are treated as secondary in ordinary mathematical practice become foundational when mathematics is used to describe physical systems, computation, evolving organisms, institutions, collective phenomena, and scientific theories that must be interpreted by machines. The central distinctions concern kinds of existence, the difference between a rule and its possible realizations, the difference between theoretical and physical objects, the explicit treatment of resource and observation limits, and the status of a conclusion when an analysis has reached a frontier rather than a proof or a refutation.

The ambition of the present programme is to investigate whether these philosophical intuitions can be transformed into one or more genuine mathematical sciences. It does not claim that classical mathematics is false, nor does it propose that the mathematics of infinity should be abolished. The stronger and more defensible claim is that the usual foundations of mathematics were optimized for a different purpose: they provide extraordinarily powerful languages for ideal structures, while leaving many transitions from ideal structure to physical realization, computation, measurement, or institutional persistence outside the foundational vocabulary. Outfinite mathematics asks whether those transitions can themselves become mathematical objects.

This document should therefore be read as a research programme rather than as a completed foundation. Terms such as Outfinite Core Logic and Outfinite Mathematical Language name targets of investigation, not finished formalisms. The programme deliberately separates semantic commitments from surface syntax. A controlled natural-language notation may eventually become useful, especially for human and artificial-intelligence interaction, but no particular notation is assumed to carry the foundational burden. The foundational claims must ultimately be stated in a small formal semantics that can be independently implemented, analyzed, and challenged.

The programme is intentionally compatible with existing mathematics and formal methods. Set theory, type theory, category theory, algebra, theorem proving, abstract interpretation, symbolic execution, model checking, non-classical logics, numerical analysis, probability, and statistics are not treated as rivals that must be replaced. They are treated as bodies of knowledge whose assumptions and proof obligations can be reinterpreted inside a framework that explicitly distinguishes kinds of existence, domains of validity, modes of evidence, and computational frontiers. The novelty, if there is one, must come from this integration and from the new results it enables, not from renaming established techniques.

A credible outcome of the programme would have two forms. The theoretical outcome would be a mathematically precise account of typed existence, outfinite processes, frontiers, theory-local validity, bridges between theories, and typed forms of evidence. The engineering outcome would be a machine-checkable intermediate representation in which scientific and mathematical theories can be expressed, reviewed, translated, executed where appropriate, and linked to data and implementations without silently confusing ideal claims with empirical ones. Either outcome can fail. The programme is scientific only to the extent that

the proposed distinctions can be formalized, compared with alternatives, and subjected to experiments that could show that they add no explanatory or practical value.

Contents

Part I. Motivation and philosophical foundations	4
Chapter 1. Why an outfinite research programme may now be scientifically and economically consequential	5
Chapter 2. Infinity as a historical success and an ontological ambiguity	11
Chapter 3. Types of existence and the meta-rational organization of theories	17
Chapter 4. Outfinite processes and a mathematics of frontiers	23
Part II. Requirements for a formal foundation	28
Chapter 5. Requirements for an Outfinite Core Logic	29
Chapter 6. Theories, bridges, evidence, and the limits of formal truth	34
Chapter 7. The theorem as a typed finite justification	38
Chapter 8. Formal analysis methods as complementary evidence-producing engines	43
Part III. Reinterpreting mathematical structure	53
Chapter 9. Arithmetic, finite witnesses, and algorithmic forms of proof	54
Chapter 10. Algebraic structures, lattices, and the algebra of information	58
Chapter 11. Category theory as a structural grammar for an outfinite foundation	62
Chapter 12. Non-classical logics, algebraic logic, and controlled pluralism	67
Part IV. Reconstructing major mathematical domains	72
Chapter 13. Collections, set-like theories, size, and diagonal reasoning	73
Chapter 14. Analysis without implicit physical infinity	77
Chapter 15. Probability, statistics, and the separation of uncertainty types	81
Chapter 16. Geometry, topology, optimization, and dynamical systems under finite observation	85
Chapter 17. Processes, phenomena, and collective existence as mathematical primitives	89
Part V. Executable science and artificial intelligence	93
Chapter 18. Outfinite mathematics as a foundation for executable science	94
Chapter 19. Artificial intelligence as co-developer, user, and beneficiary	99
Chapter 20. A future mathematical language and implementation architecture	104
Part VI. Validation, scientific trajectory, and potential impact	108
Chapter 21. A falsifiable research programme	109
Chapter 22. Scientific and economic trajectories if the programme succeeds	113
Chapter 23. Integrated exemplars for testing the research programme	117
Chapter 24. A staged research trajectory	122
Conclusion. From a philosophy of limits to a candidate science of executable knowledge	126
Appendix A. Evidence profile of major formal and analytical techniques	129
Appendix B. Core conceptual vocabulary	131
Appendix C. Consolidated hypotheses and direct falsification routes	132
Bibliography	133

Part I. Motivation and philosophical foundations

Chapter 1. Why an outfinite research programme may now be scientifically and economically consequential

1.1 Mathematics succeeded by idealizing away limits

Modern mathematics is one of the most successful intellectual constructions ever created. A major source of its power is precisely its freedom from many constraints of the physical world. A mathematical line need not be manufactured. A real number need not be stored with all of its digits. A function can be defined on a domain that no computer could enumerate. A proof may establish a statement for arbitrarily large values without executing the corresponding physical computation. This separation between mathematical object and material realization is not an accidental weakness. It is one of the reasons mathematical structures can be reused across very different sciences.

For pure mathematics, there is often little advantage in forcing every theorem to record the amount of memory required by a computation, the precision of an instrument, or the persistence conditions of an organism. Such information would obscure general structure. A mature outfinite programme must therefore resist the temptation to treat abstraction itself as a mistake. The question is narrower and more interesting: what happens when the distinctions that mathematics legitimately abstracts away become the very subject of a theory?

In applied work, transitions between levels are everywhere. A differential equation may be exact within an ideal model while its numerical solver is approximate. A security proof may be valid under an adversarial model while the deployed implementation leaks information through an unmodeled channel. A statistical parameter may exist as part of a probabilistic model while the available observations only constrain it within an interval. A company may exist legally while no longer functioning as a coordinated organization. A simulation can contain an entity that has no observed physical counterpart. A machine-learning system may treat a symbolic object as if it were an empirical fact because the distinction is expressed only in surrounding prose.

Experts usually know how to navigate such distinctions. The difficulty is that much of the knowledge is tacit. A paper often states equations formally while leaving the semantics of measurement, approximation, applicability, model selection, numerical representation, and evidential status in natural language. A human reader reconstructs the missing context from disciplinary training. A machine must infer it. The transition from human-centered scientific communication to AI-mediated scientific work therefore changes the economics of explicitness.

When mathematical and scientific reasoning was produced and consumed mainly by humans, the cost of deep formalization was high and its benefit was concentrated in specialized domains. Artificial intelligence reverses part of that balance. Generating candidate explanations, equations, code, hypotheses, or even formal proofs is becoming inexpensive relative to checking that the generated object preserves meaning and remains valid in the intended world. This suggests a general economic principle for scientific AI: as generation becomes cheap, semantic validation becomes scarce.

Outfinite mathematics is motivated by the possibility that a foundational vocabulary designed around existence types, processes, validity regions, approximation, and resource frontiers could make validation substantially more mechanical. The scientific value would lie in discovering whether these concepts can be stated with enough precision to support theorems. The economic value would arise only if such precision reduces the cost of reviewing theories, simulations, scientific software, AI-generated models, and evidence chains.

The strongest industrial opportunity is therefore not a new notation for mathematicians. It is a possible infrastructure layer between informal scientific knowledge and executable computation. If a theory can be represented in a form that distinguishes what is assumed, what is defined, what has been observed, what is computed, what is approximated, what is proven, and where each claim stops being justified, then machines can perform forms of review that remain difficult when those distinctions are implicit. Such infrastructure could matter in safety-critical engineering, life-science modeling, scientific software, cybersecurity, regulated AI, high-assurance data analysis, and any domain where the cost of a plausible but inapplicable conclusion is substantial.

This potential should not be exaggerated. Existing proof assistants, formal specification languages, modeling environments, unit systems, probabilistic programming languages, and verification tools already express many of these distinctions. The research question is whether a common foundation centered on typed existence and frontiers produces a meaningful gain in interoperability, semantic fidelity, or automation. If it merely recreates existing capabilities behind new terminology, the programme has failed in one of its central claims.

1.2 Artificial intelligence changes both the problem and the feasible solution

AI can influence the programme in two opposite directions. It increases the need for a more explicit foundation, and it may make the construction of that foundation economically possible.

The need arises because generative models can move fluently between mathematical, scientific, and ordinary language while failing to preserve all of the conditions under which a statement is valid. A model can reproduce the algebra of a theorem while dropping a continuity assumption, convert a theoretical quantity into an apparent empirical fact, overlook that a solver only explored a bounded state space, or apply a result derived under one probabilistic model to data generated under another. These errors are not all failures of arithmetic or logic. Many are failures to preserve context and type of existence.

A framework that makes such information structural rather than rhetorical could act as a semantic discipline for AI. An AI system could still generate conjectures and translations, but its outputs would have to pass through explicit checks. A theorem derived inside an ideal theory would remain labeled as a consequence of that theory. A numerical result would remain attached to a representation and an error model. An empirical statement would remain attached to observations and measurement assumptions. A transfer from one layer to another would require an explicit bridge rather than an implicit change of meaning.

AI may also make the foundation itself feasible to build. Coding agents can already perform long sequences of software-engineering operations, interact with compilers and formal verifiers, and revise failed attempts. In a mature research workflow, specialized agents could

extract definitions from literature, propose type assignments, discover hidden assumptions, generate proof obligations, search for invariants, suggest abstract domains, construct counterexamples, translate claims to external proof assistants, and compare the resulting formal statement with the original source. Other agents could perform adversarial review, deliberately searching for semantic drift or illicit theory bridges.

The important architectural principle is that these agents should not be trusted merely because they are capable. Their role is generative and exploratory. Validation should be delegated, where possible, to small deterministic kernels, independently checkable proof objects, solver certificates, executable tests, reproducible computations, and explicit human review for those semantic choices that cannot yet be mechanized. This resembles the successful separation in modern proof assistants between sophisticated automation and a relatively small trusted core. Recent AI-assisted formalization systems reinforce the plausibility of this arrangement: agentic workflows can construct and repair formal developments while the final artifacts remain subject to machine checking (Meadows, Zhang, and Freitas 2026; Nie et al. 2026). The open problem is whether the same architecture can be extended from formal mathematics to scientific theories whose semantics includes measurement, approximation, and empirical reference.

If successful, this creates a reciprocal relation between outfinite mathematics and AI. AI would help build the formal libraries, translators, abstractions, and proof strategies required by the new mathematics. In return, the new mathematics would provide AI with a more disciplined representation of applicability. This is potentially more important than simply improving theorem proving. Many failures of practical AI reasoning occur not because a local derivation is syntactically invalid but because a valid result is transported into a context where its assumptions do not hold. A system that explicitly represents such boundaries could improve the rigor of AI-mediated engineering and science even when no new mathematical theorem is discovered.

1.3 Executable science as a possible destination

The most ambitious practical interpretation of the programme is executable science. The phrase should not be understood as a claim that scientific judgment can be reduced to program execution. Science includes concept formation, choice of relevant variables, experimental design, interpretation, social validation, and revision of ontologies. A formal language cannot simply replace these activities.

A more defensible objective is the near-complete formalization of the operational core of a scientific theory. Such a representation would contain the theoretical entities the model refers to, the processes it permits, the equations or transformation rules it uses, the assumptions under which they apply, the relation between theoretical quantities and observations, the uncertainty and precision of measurements, the numerical methods used for computation, and the claims that follow from the theory under specified conditions. It would also record what remains informal or empirically underdetermined.

Once a scientific theory is represented in this form, a computer can perform a qualitatively different kind of review. It can trace which assumptions support a conclusion, determine which conclusions are affected by changing a parameter or axiom, compare two versions of the theory, verify dimensional and representation consistency, reproduce computations, detect uses of results outside their stated domain, and search for experiments or data that

discriminate between competing models. AI can then operate over the formal structure rather than only over the prose description.

This vision is closely related to current work on autoformalization, formal mathematics, scientific knowledge representation, digital twins, mechanized reasoning, and reproducible computational science. The distinct outfinite hypothesis is that an executable theory needs a richer foundation than a proof language alone. It must be able to distinguish proof from empirical support, theoretical existence from physical instantiation, exact value from measured value, and logical openness from physically realizable continuation. Whether that hypothesis is correct is one of the central questions of the programme.

1.4 Why apparently obvious distinctions did not become foundational earlier

The fact that several outfinite intuitions sound obvious is not evidence that the programme is redundant. In the history of successful formalisms, an important distinction can remain conceptually familiar for a long time without becoming foundational. Engineers have always known that machines are finite, physicists have always known that measurements have finite precision, and logicians have long distinguished syntax from semantics. Yet a foundation is not a list of truths that practitioners already know. It is a choice about which distinctions the formal system forces every subsequent theory to preserve. The central claim of the outfinite programme is therefore not that mathematicians failed to notice limits. It is that mathematics had strong reasons to abstract them away, while contemporary scientific computation and AI create a new class of tasks in which those abstractions become operationally expensive.

Several historical incentives worked against a limit-aware foundation. Classical mathematics achieved enormous compression by treating ideal objects as stable and complete. Once a theorem had been proved over real numbers, groups, topological spaces, or probability spaces, the result could be transferred to many applications without rebuilding the formal machinery for each physical implementation. Foundational work consequently optimized for generality, consistency, expressiveness, and elegant reconstruction of existing mathematics. Questions about finite precision, resource exhaustion, model mismatch, sensor uncertainty, compiler correctness, or institutional persistence were assigned to applied disciplines. This division of labor was rational. A theorem of analysis should not become unreadable because every sentence carries a processor model and an error budget.

Computer science then developed many of the missing distinctions independently. Operational semantics gave precise meaning to execution. Type systems distinguished classes of admissible values. Hoare logic separated preconditions from postconditions. Model checking exposed bounded and state-based reasoning. Abstract interpretation formalized sound approximation. Numerical analysis made truncation and round-off error explicit. Formal verification distinguished source specifications from implementations and trusted components. Category theory supplied languages for structure-preserving translation. None of these developments, however, was required to reorganize the general ontology of mathematics. They were usually treated as specialized theories built on top of existing foundations.

The outfinite proposal becomes interesting only if this historical separation is now becoming a bottleneck. The strongest candidate bottleneck is machine-mediated science. Human experts can often repair an underspecified formula by context. They know that an experimentally

fitted parameter is not the same kind of object as an exact constant in a theoretical model. They understand that a proof about integers does not automatically apply to fixed-width arithmetic, and that a simulator's success over a finite horizon is not a proof about all future behavior. Artificial systems can learn these distinctions statistically, but unless they are represented formally there is no general mechanism that prevents a model from silently moving between them.

A second reason the programme may be timely is the falling cost of formal artifact generation. Historically, one of the main arguments against deeper formalization was labor. A scientific paper written for trained readers could express in twenty pages what a fully formal development might require thousands of lines to encode. AI-assisted formalization, code generation, theorem search, invariant synthesis, and translation validation do not eliminate this cost, but they can redistribute it. If machines can generate candidate formalizations while a small trusted kernel verifies the resulting artifacts, then the economically relevant question changes. It is no longer whether humans are willing to write every formal detail manually, but whether the formal layer catches enough costly errors to justify its maintenance.

This suggests a concrete adoption hypothesis. Outfinite mathematics is unlikely to spread because a community accepts a philosophical criticism of infinity. It can spread if it solves problems that existing stacks solve only through an accumulation of disconnected conventions. A researcher should not need one informal convention to mark theoretical entities, another to record data provenance, a third to document numerical tolerance, a fourth to describe verification bounds, and a fifth to tell an AI which conclusions may be transported across models. If a common semantic layer can connect these without reducing them to the same kind of object, then the programme has a practical reason to exist.

The economic case therefore follows from validation rather than from notation. In research-intensive industries, the expensive errors are often not elementary calculation mistakes. They are errors of applicability: a model is used outside its calibrated regime, a theorem is applied after one of its hypotheses has ceased to hold, an approximation is mistaken for an exact quantity, a statistical association is treated as a mechanistic claim, or a software guarantee is transported across a compilation or deployment boundary that was never verified. These failures are costly because they are locally plausible. A system that makes applicability conditions and theory bridges machine-checkable could reduce review cost, shorten iteration cycles, and make AI-generated scientific work easier to audit.

The programme should consequently be judged by whether it creates a useful new formal interface between pure mathematical structure and realized scientific systems. If existing type theories, specification languages, proof assistants, uncertainty frameworks, and provenance standards can provide the same interface with no meaningful loss, then a new foundation is unnecessary. If the outfinite vocabulary allows those components to interoperate with less semantic ambiguity, the fact that its constituent intuitions were already familiar becomes a strength rather than a weakness. The novelty would lie in the architecture of the distinctions and in the new automation that architecture supports.

1.5 Research obligations and falsifiability

The opening claim is testable. Outfinite mathematics should not be judged by whether its philosophical language is attractive but by whether it improves a task that matters. A first class of experiments would compare direct formalization of scientific claims in existing systems with an outfinite representation that explicitly models existence type, validity

boundary, representation, and evidence. Independent experts would assess semantic fidelity. If the additional structure does not reduce errors such as model-reality conflation, scope loss, hidden approximation, or unjustified transfer between theories, then the central motivation is weakened.

A second class of experiments would evaluate AI. The same scientific text could be given to coding agents asked to formalize it directly in an existing proof environment and to agents using an outfinite intermediate representation before compilation to the same environment. The measurable outcomes would include formal validity, semantic fidelity, amount of human correction, number of hidden assumptions recovered, and rate of successful independent verification. If the intermediate representation adds substantial complexity without improving these outcomes, the claim that outfinite mathematics is especially suitable for AI would not be supported.

The economic thesis is also falsifiable. Formalization costs, review time, defect discovery, reuse across projects, and maintenance under theory change can all be measured. An outfinite toolchain that cannot demonstrate a credible reduction in one or more of these costs should not be sold as infrastructure merely because its philosophical motivation is strong. The programme begins with a large potential precisely because it also accepts a clear possibility of failure.

Chapter 2. Infinity as a historical success and an ontological ambiguity

2.1 Why the history matters

The outfinite programme is motivated by a critique of how infinity is interpreted, not by the claim that mathematicians failed to notice its difficulties. The opposite is true. Infinity has been one of the most carefully contested ideas in the history of mathematics, and the modern acceptance of actual infinity emerged from a long sequence of conceptual choices. Reconstructing that history is useful because it reveals that several questions commonly compressed into the word infinity are logically distinct.

Ancient Greek mathematics already possessed methods that, from a modern perspective, operate over unbounded mathematical structure. Yet Greek mathematicians often formulated results without treating an infinite totality as a completed object. Euclid's argument about prime numbers is an instructive case (Euclid, Book IX, Proposition 20; Heath 1926). Its logical force does not require a completed collection containing every prime. Given any finite collection of primes, the construction produces a reason why that collection cannot exhaust the primes. The result is naturally read as extensibility beyond every presented finite collection. Modern mathematics can, of course, summarize this by saying that there are infinitely many primes, but the ancient formulation keeps the constructive structure visible.

Aristotle developed the canonical distinction between potential and actual infinity (Aristotle, *Physics*, Books III-IV). He was prepared to treat a process as indefinitely extendable in the relevant logical sense while rejecting the idea that the infinite exists as a completed magnitude in the same way as a finite object. This distinction shaped mathematical thought for centuries. It is also close to, but not identical with, the outfinite intuition.

The difference matters. Potential infinity is commonly understood as the claim that after any stage another stage can be taken. In pure mathematics this is coherent and extremely useful. From an outfinite perspective, however, it still combines two claims that should be separated. A rule may contain no internal logical stopping condition, while a physical or computational realization of that rule may fail to continue because memory, energy, time, precision, organization, or the environment has reached a frontier. The absence of a stop in the rule is not the same as an unconditional guarantee of future realization.

Galileo's discussion of natural numbers and perfect squares reveals another problem. Each natural number can be paired with exactly one square, yet squares appear progressively sparse among the naturals when finite prefixes are compared. Cantorian cardinality later gives a precise and powerful resolution by selecting one-to-one correspondence as the criterion relevant to cardinal size. The outfinite lesson is not that Cantor's criterion is wrong. It is that there are several legitimate notions of comparison: pairability, finite-prefix density, generative cost, representational cost, and other views need not agree. A paradox can arise when a single word such as size is used across incompatible views.

The nineteenth century transformed the status of infinity. Cantor's work made completed infinite sets and transfinite numbers first-class mathematical objects (Cantor 1915). This was not merely a new computational technique; it was an ontological expansion inside mathematics. A totality such as the natural numbers could be treated as a completed object

over which new constructions were performed. The results were extraordinarily fruitful and became central to modern analysis, topology, measure theory, logic, and set theory.

Resistance did not disappear. Gauss is often cited for rejecting the use of an actually infinite quantity as a completed object, while accepting infinity as a manner of speaking about unbounded processes. Later foundational debates made the issue more precise. Hilbert defended the indispensable role of ideal elements while trying to secure mathematics through finitary metamathematical reasoning (Hilbert 1926). Brouwer rejected unrestricted use of classical principles, especially when existence could not be tied to construction (Brouwer 1908). Gödel and Turing then demonstrated limits that no simple foundational optimism could remove: sufficiently expressive formal systems encounter incompleteness, and general computation encounters undecidability (Gödel 1931; Turing 1936).

The historical conclusion is not that one side won and the others were irrational. Different programmes optimized different desiderata. Cantorian mathematics gained expressive unity and theoretical reach. Constructive and finitistic traditions preserved stronger links between existence and construction. Formalism separated symbolic derivation from metaphysical commitment. Computer science later turned finite procedures, execution traces, types, and resource constraints into mathematical objects in their own right.

Outfinitism can therefore be interpreted less as a rebellion against modern mathematics than as an attempt to recombine lessons that became institutionally separated. It accepts that actual infinity can be coherent as a theoretical object while refusing to promote that coherence automatically to physical or computational existence. It accepts the insight of potential infinity that some rules are open-ended while refusing to identify logical openness with guaranteed realizability. It accepts formalism's distinction between symbolic manipulation and interpretation while insisting that the interpretation layer be made explicit when mathematics is applied.

2.2 Infinity as a kind of theoretical existence

The phrase "infinity does not exist in the real world" is rhetorically strong but mathematically insufficient. We do not possess an observational procedure that surveys the entire universe and proves that no physically infinite structure exists. More importantly, the interesting issue is not a metaphysical slogan but a typing problem. What exactly is asserted when a theory says that an infinite set exists?

Inside an axiomatic theory, the answer can be perfectly precise. Existence means that an object satisfying the axioms is admitted by the theory or can be constructed within its semantics. The symbol is finite, the rules governing it are finite inscriptions, and finite reasoners can derive consequences about the ideal object without enumerating all of its members. This is an important clue for outfinite semantics: a completed infinite object can be treated as a theoretical existence whose manipulation is mediated by finite symbolic rules.

Such an interpretation explains both the coherence and the limitation of actual infinity. Coherence is possible because the object need not be materially instantiated as a collection of all its elements. It is represented intensionally by axioms, definitions, constructors, or semantic clauses. The limitation is that the theory itself does not establish a physical counterpart merely by being internally coherent.

This distinction is familiar in scientific practice. An ideal gas, a perfectly rigid body, a point particle, a continuous field, and an infinite sequence can all be legitimate theoretical objects

even when no physical system instantiates the idealization exactly. Outfinitism proposes that mathematics should be able to represent this status explicitly rather than leaving it to informal interpretation.

The same discipline applies to potential infinity. The statement that a successor operation is defined without a logical maximum is a property of a rule. The statement that a particular physical machine can continue applying that rule forever is a claim about a physical process. A third statement, that for every finite value provided as input there exists a mathematically specified successor, is a parametric property of the theory. These three claims are often harmlessly identified in pure mathematics, but they should be distinct in a foundation intended to bridge mathematics and implementation.

The proposed term outfinite is reserved for the intermediate case that matters to this programme: a finitely specified rule or process has no internal logical terminal state of the relevant kind, yet every realized trace is finite and any further continuation must be supported by an actual extension event. The process is open without being completed, and it is not endowed with an unconditional metaphysical promise of continuation.

This definition is deliberately modest. It does not establish that every important use of infinity can be replaced by an outfinite construction. Some classical theorems may have no useful outfinite analogue, or their analogues may be substantially more cumbersome. If the outfinite programme succeeds, it will probably coexist with theories that admit actual infinity. The foundational innovation would then be the ability to state which kind of existence is being used and to control the bridges between them.

2.3 From a dispute over infinity to a science of limits

Focusing exclusively on infinity would make the programme too narrow. The deeper theme is the mathematical status of limits. Physical systems have finite resolution, finite energy, finite duration, and bounded accessibility. Computations have finite memory, time, precision, and communication. Observers have finite measurement capacity. Organizations have limits of coordination and persistence. Scientific models have domains in which their approximations remain credible and boundaries beyond which other models are required.

Traditional mathematics can describe all of these limits, but it usually describes them within a foundation whose own ideal objects are not required to expose similar boundaries. Outfinitism asks whether limits can become semantic obligations attached to the entities and transformations under study. A theorem could then state not only a relation but the region over which the relation has been justified. A process could terminate in a frontier state that records why continuation is unavailable. A numerical construction could expose residual error as part of its mathematical value. A theory bridge could fail because two models apply at incompatible scales.

The word frontier is therefore more important than the word finite. Outfinitism is not a doctrine that every interesting object must be small, enumerated, or computationally easy. It is a doctrine that the difference between what has been represented, what can be generated under a given rule, what has been realized, and what is only postulated by a theory should remain visible.

2.4 Actual, potential, and outfinite infinity as different existential commitments

The historical dispute over infinity can be reformulated in a way that is more useful for the present programme. Actual infinity, potential infinity, and the proposed outfinite notion need not be treated as three competing answers to a single metaphysical question. They can be treated as three different contracts governing what a theory is entitled to infer from a finite rule.

Actual infinity allows a completed totality to exist as an object of the theory. Once such an object is admitted, quantification over the totality and constructions on it are legitimate within that theory. This move is mathematically coherent in the ordinary relative sense: a formal system can define the object, state axioms governing it, and support a large body of deductions whose consistency can be studied relative to other accepted systems. The outfinite programme has no reason to deny this form of existence. Its objection is to the unmarked transfer from theoretical existence to stronger claims about physical instantiation, completed computation, or realized enumeration. A completed infinite set can be an excellent theoretical object even if no physical system realizes its members simultaneously.

Potential infinity makes a different commitment. Rather than postulating a completed totality, it treats a process as indefinitely extensible. In ordinary mathematical reasoning, this is often expressed by saying that for every stage there is a next stage. The phrase appears modest, but it contains a global commitment: the possibility of continuation is asserted without reference to resources, physical time, representation, or the continued availability of the process that performs the extension. In a purely logical domain this may be exactly what is intended. In a physical or computational domain it conflates the absence of a stopping rule with the guaranteed realizability of further steps.

The outfinite notion tries to separate these claims. An outfinite process is specified by a finite rule whose internal semantics does not contain a final state of the relevant kind, while each realized trace remains finite and each extension is an event that requires a witness. The witness may be a successful computation, a construction, an observation, or another theory-local certificate. Reaching a frontier does not prove that no extension is possible under any circumstances. It proves only that the present realization has exhausted the declared context or that the system can no longer justify a next step under its current assumptions.

This distinction is particularly important for computers. Consider natural-number generation. The rule that maps a natural number to its successor is finite and contains no largest value. That fact can be inspected symbolically. A particular implementation, however, uses a representation, storage, and execution environment. It may overflow, exhaust memory, or become physically impossible to continue. The logical successor rule has not thereby become false. Conversely, the existence of the successor rule does not imply that a given machine can represent every successor. The theory contains an open rule; the realization contains a finite trace and a frontier.

The same analysis applies outside arithmetic. A blockchain protocol may contain no designated final block while every actual ledger has finite length. A scientific research programme may always admit another experiment while every historical research process is finite. A refinement algorithm may admit smaller error tolerances in theory while a device eventually reaches sensor noise or arithmetic limits. A biological lineage may have no predefined terminal generation while every observed lineage has finite history. These are not

metaphorical uses if the framework identifies the common structure: finite rule, finite realization, witnessed extension, explicit frontier.

The concept must nevertheless be defended against a serious objection. One could argue that outfinite processes are simply ordinary potentially infinite processes plus engineering annotations. The programme has to show that the added structure changes mathematical reasoning. It should lead to metatheorems about frontier extension, theorem transport, residual obligations, or representation change that are difficult to state cleanly when the frontier is external metadata. For example, a safety property established within one resource profile may be preserved when resources increase if the semantics is monotone, while an approximation certificate may become invalid when the representation changes. Such statements turn the frontier into a mathematical parameter rather than a prose qualification.

The status of infinity also clarifies the intended meaning of consistency. A theoretical infinite object can be coherent because its symbolic operations are well defined and because the surrounding formal theory has an accepted model or relative consistency argument. This coherence should not be rhetorically upgraded into physical existence. The outfinite view therefore does not attempt to "disprove infinity." It attempts to type the sense in which an infinite object exists and to require explicit bridges before consequences are exported into another ontological domain.

The historical controversy can then be understood as a conflict partly produced by under-typed existence. Cantorian mathematics is extraordinarily successful when completed infinite totalities are treated as theoretical objects. Aristotelian and finitist intuitions remain compelling when attention shifts to realized processes, constructions, and physical limits. An outfinite foundation would try to make both positions locally legitimate while preventing silent movement between them. Its success would be measured not by resolving an ancient metaphysical argument once and for all, but by showing that typed interpretation yields cleaner scientific and computational reasoning.

2.5 Research obligations and falsifiability

The historical narrative does not by itself establish a new mathematics. Its role is to motivate distinctions that must survive formal scrutiny. The first falsifiable obligation is definitional: actual infinity, potential infinity, and outfinite extensibility must be represented in a common meta-theory without equivocation. If the proposed distinction between potential and outfinite processes collapses under formalization, or if every outfinite construction turns out to be merely a verbose reformulation of an existing constructive notion with no new consequences, the conceptual claim should be revised.

A second obligation concerns translation. Representative classical results involving infinity should be reconstructed in several ways. Theorems about primes, convergent sequences, real numbers, compactness, and asymptotic behavior provide progressively harder tests. The research question is not whether the classical theorem remains true, but whether an outfinite formulation can preserve the practically relevant content while making its ontological commitments more explicit. Failure to recover useful versions of major results would define the natural boundary of the programme.

A third obligation is comparative. Formal users should be asked to reason about examples in which theoretical, computational, and physical claims diverge. If existing type systems or specification languages express the distinction just as naturally and with less machinery, then

outfinite mathematics must either integrate those systems rather than duplicate them or abandon its claim to foundational novelty.

Chapter 3. Types of existence and the meta-rational organization of theories

3.1 Why existence should be typed

The ordinary mathematical existential quantifier deliberately abstracts from the manner in which an object exists. Within a formal theory, that abstraction is often exactly what is needed. Yet when mathematical theories are connected to computation and observation, the single word exists carries several incompatible burdens.

A number can exist as a value represented in memory. A geometrical point can exist as an ideal object of a theory. A particle can exist as a hypothesized entity in a physical model, as a detected event in an instrument, or as a simulated object in software. A company can exist as a legal entity even when its productive organization has disappeared. A storm can exist as a phenomenon whose material constituents change continuously. A swarm can persist while its members enter and leave. A scientific theory can refer to an exact quantity that no measurement can ever instantiate with infinite precision.

The programme proposes that these should not all be treated as instances of one unqualified existential status when a theory is intended for machine interpretation. The first foundational research task is therefore to construct a small family of existence modes with explicit introduction and transfer rules.

A minimal ontology should distinguish at least theoretical existence, symbolic or representational existence, computational realization, physical existence, processual existence, phenomenal existence, and collective existence. These categories are not assumed to be mutually exclusive. The same referent may be represented under several modes. A scientific model is symbolically represented, computationally instantiated in a simulator, and theoretically defined; the physical system it models has a different mode of existence. A collective may simultaneously have legal, physical, processual, and social manifestations.

The crucial requirement is that a statement proven under one mode cannot silently migrate into another. From the fact that a mathematical model contains a solution, one may not infer that a corresponding physical system exists. From the fact that a simulation realizes a process, one may not infer that nature realizes it. From the fact that an institution exists in a registry, one may not infer that it remains operational as a collective. Such inferences may be valid, but they require bridges that carry their own assumptions and evidence.

This is more than metadata. If existence modes are foundational, they influence the admissible notion of identity, the meaning of negation, the kind of witness required, and the form of proof. Theoretical existence may be justified by derivation from axioms. Computational realization requires an executable representation and a trace or constructive witness. Physical existence requires an observational or causal bridge. Collective existence requires a persistence criterion that is not reducible to identity of membership.

3.2 Identity as a family of relations

The law of identity remains indispensable. The outfinite programme does not attempt to make a value different from itself. The problem arises when a static equality relation is used as the universal model of persistence.

A value in an immutable representation can be compared by exact equality. A state of a process can also be compared by exact equality, but the process itself usually persists through unequal states. A river changes its water, shape, and flow. An organism replaces material components. A software service changes its internal memory. A university changes students, staff, buildings, and regulations while remaining recognizably the same institution under certain criteria.

Outfinite mathematics therefore treats persistence as theory-dependent. A process is the same process across two stages if a declared continuity relation connects those stages. A phenomenon reappears if a recognizer identifies the relevant structural pattern. A collective persists if an organizational invariant remains satisfied despite member changes. These relations can be formal and exact even though they are not ordinary value equality.

This approach has a useful consequence. It makes explicit that many objects treated as static nouns are abstractions over processes. The object is not unreal; it is stable under a chosen view. A physical object can be modeled as persistent because changes below a certain scale are irrelevant to the question being asked. At another scale, those changes become the central dynamics. The framework therefore needs views or abstraction contracts that state what differences are ignored and why.

Abstract interpretation offers a technical analogy. A concrete state contains more detail than an abstract state, but the abstract state can be sound for a specified property. Outfinitism extends this intuition beyond software analysis: an object can be understood as a stable abstraction over a changing process, provided the abstraction and its domain of validity are explicit.

3.3 Meta-rationality as controlled pluralism

The philosophical affinity between outfinitism and meta-rationality is strongest at this point. Meta-rationality, as used here, is not the claim that all theories are equally true or that contradiction is harmless. It is the disciplined ability to recognize that different models can organize the same reality from different scales, purposes, and constraints, and that rationality sometimes requires switching between models rather than forcing one model to absorb every question.

Classical mechanics, thermodynamics, statistical mechanics, continuum fluid models, and molecular simulations can describe overlapping aspects of physical systems. A legal theory of a company and an organizational theory of the same company answer different questions. A Boolean model of collective membership and a graded model of participation can both be useful if their semantics are kept distinct. The problem begins when a conclusion from one model is transferred to another without a justified translation.

Typed existence offers a possible mathematical realization of this meta-rational ideal. Instead of demanding one total ontology, the foundation allows multiple theories to declare their own objects, identity conditions, logics, and evidence standards. A meta-layer records when terms correspond, when one theory refines another, when a translation loses information, and when no valid bridge is known. The result is controlled pluralism rather than relativism.

This may be one of the programme's most important departures from conventional foundational ambitions. The goal is not necessarily a single final theory from which every legitimate mathematical and scientific statement is derived. The goal may instead be a disciplined ecology of theories, each explicit about its commitments and connected through

typed transformations. Category theory, modal logic, type theory, and formal semantics all offer tools for making such an ecology precise.

A meta-rational foundation is especially attractive for executable science because scientific practice is already plural. Different models operate at different regimes. Approximation schemes are deliberately chosen for tractability. Experimental evidence may support one model at one scale and another at another scale. A machine-readable science should not hide this plurality behind one undifferentiated predicate of truth. It should record which theory licenses which conclusion and under what bridge that conclusion can be used elsewhere.

3.4 Collective and phenomenal existence

Collectives and phenomena provide difficult tests because they resist reduction to static membership. A swarm is not simply the set of insects present at an instant. An ecosystem is not simply the list of organisms. A market is not simply its current participants. A research community can persist despite complete turnover of members over a sufficiently long period.

An outfinite theory of collective existence would define a collective through a combination of participants, relations, memory, coordination, boundary conditions, and persistence criteria. Membership may be crisp in some theories and graded in others. The theory must distinguish the existence of a legal or symbolic representation of the collective from the existence of the functional organization itself.

Phenomena are similarly process-dependent. Turbulence, inflation, an epidemic wave, a phase transition, or social polarization are not single objects. They are patterns recognized in histories. Their identity is therefore closer to structural recurrence than to material sameness. A phenomenon exists in a given mode when a specified recognizer, operating on admissible evidence, establishes that the pattern is present. The recognizer and its uncertainty become part of the theory rather than invisible human judgment.

These examples show why non-classical logics may later become relevant. Graded membership does not imply probabilistic uncertainty. Conflicting reports do not necessarily justify logical explosion. Temporal existence requires operators that distinguish what is true now from what remains true through a process. The existence ontology therefore naturally leads toward a modular logic rather than a single universal truth calculus.

3.5 From a taxonomy of existence to a formal discipline of admissible inference

A useful list of existence types is only the beginning. The difficult task is to make the distinction operational. If physical existence, theoretical existence, processual existence, phenomenal existence, and collective existence are merely labels attached to otherwise ordinary propositions, then the foundation gains vocabulary but not logic. The research programme must determine what changes in inference when the type of existence changes.

The first principle is that existence should be local to a mode of interpretation. A theoretical object exists because a theory introduces it through axioms, definitions, constructions, or accepted semantic rules. A physical object exists because an observational and causal practice supplies evidence that is interpreted as instantiation. A process exists through an ordered continuity of states or events. A phenomenon exists through a pattern that recurs or persists across multiple realizations. A collective exists when relations among changing participants

support a stable level of description that carries explanatory or predictive force. These descriptions are intentionally different because the witnesses and identity conditions are different.

The second principle is that inference across existence types must be mediated. A theorem about a perfect pendulum does not directly establish a fact about a particular laboratory apparatus. It participates in a bridge that also contains measurement models, parameter estimates, error bounds, and assumptions about neglected effects. Similarly, an observed periodic signal does not by itself identify the theoretical mechanism that generated it. The mapping from observation to theory is abductive or model-selective, not deductive identity. A credible outfinite logic must preserve this asymmetry.

This requirement points toward modal or indexed semantics. One possibility is to treat existence modes as contexts in which propositions are judged. Another is to treat them as modalities that qualify claims. A third is to use dependent types in which the witness carried by an object determines which eliminations are legal. Category-theoretic semantics might then describe bridges as structure-preserving maps between local theories. The programme should not choose among these architectures prematurely, but it must insist on a semantic property: it should be impossible to transport a conclusion from one mode of existence to another without passing through an explicitly named and checkable rule.

Identity becomes the second major test. Classical equality is indispensable for values, but it is too coarse as a universal account of persistence. An organism at two moments can be the same organism despite molecular replacement. A company can remain legally identical after complete turnover of employees. A river can persist while none of the water is the same. A phenomenon such as inflation or turbulence can recur in different material systems. The foundation therefore needs several relations that answer different questions: exact identity of represented values, equivalence under a chosen observation, continuity of a process, recurrence of a pattern, and persistence of a collective under membership change.

This does not mean abandoning the law of identity. It means refusing to use one unqualified relation for every ontology. A value can remain exactly identical because the theory freezes all relevant variation. A process is instead related to its later state through a continuity criterion. A collective is related to its later configuration through an invariant over organization, memory, role structure, or coordinated behavior. The choice of invariant is theory-dependent and therefore falsifiable. If the alleged collective loses predictive or explanatory coherence when members change, the claim of collective existence should fail.

Phenomenal existence deserves particular attention because it is central to science. Many scientific objects are not observed as stable material individuals. A wave, a phase transition, a recession, an epidemic wave, and a synchronization regime are patterns recognized in evolving systems. Their identity is closer to structural recurrence than to material sameness. A formal system intended for executable science must allow a theory to specify the recognizer, the observations on which it operates, the tolerance under which two occurrences count as the same phenomenon, and the evidence that the recognizer is reliable.

Collective existence extends this logic to systems whose macro-properties depend on relations among participants. A swarm, market, organization, distributed protocol, ecosystem, or scientific community may survive replacement of many members while losing its identity if coordination, memory, or boundary conditions collapse. This suggests a mathematical definition in terms of a stable abstraction from microstates to macrostates. The abstraction

earns ontological significance when it supports predictions, interventions, or invariants that cannot be expressed economically at the member level. Abstract interpretation is relevant here not only as a software technique but as a general model of how a coarse description can remain sound with respect to many concrete realizations.

The danger is obvious: if every useful abstraction is granted a new kind of existence, the ontology becomes arbitrary. The programme must therefore impose criteria. A proposed existence type or collective object should have explicit introduction conditions, persistence conditions, ways of being refuted, and a declared scope. It should improve reasoning rather than merely redescribe it. Different views may coexist, but their incompatibilities must be visible. This is where the relationship with meta-rationality becomes concrete. Meta-rationality is not the assertion that all perspectives are equally valid. It is the controlled use of multiple partial theories, each with known commitments and known limits, together with rules governing when translation between them is warranted.

If this part of the programme succeeds, typed existence becomes a practical defense against category errors in both human and AI reasoning. An AI system would not merely store that a parameter was "found"; it would preserve whether it was defined theoretically, estimated statistically, observed experimentally, inferred from another model, or computed under a finite approximation. The formalism would then constrain which subsequent operations can treat the value as exact, which can treat it as empirical evidence, and which require another bridge. If no robust inference discipline emerges from the taxonomy, the proposal should be reduced to philosophical terminology rather than advertised as a new mathematical foundation.

3.6 Research obligations and falsifiability

The theory of existence types is the most important place where the programme could fail. A taxonomy is not mathematics. The research obligation is to provide formal introduction rules, elimination rules, identity criteria, and bridge conditions for each proposed mode. If these rules can be encoded more cleanly as ordinary predicates inside an existing dependent type theory without loss of the intended distinctions, then the case for a new foundational layer must be reconsidered.

The bridge calculus should be tested on deliberately adversarial examples. A system should reject an inference from a simulated observation to a physical observation unless an empirical bridge is supplied. It should distinguish the legal persistence of an organization from its functional persistence. It should represent the exact solution of an ideal equation separately from a numerical approximation. If independent formalizers repeatedly disagree about how to assign existence types, the ontology may be too subjective to serve as a foundation and may need to be reduced or reorganized.

Meta-rationality also creates a falsifiable engineering claim. A multi-theory representation should improve model interoperability without erasing disagreements. Benchmarks can compare a monolithic formalization with a theory-local formalization connected by explicit bridges. If the latter merely creates administrative overhead and does not improve error localization, reuse, or semantic clarity, then the meta-rational architecture has not justified its cost.

Chapter 4. Outfinite processes and a mathematics of frontiers

4.1 From unbounded rule to realized process

The term outfinite should earn its place by naming a structure that is not adequately captured by the usual opposition between finite and infinite. The intended structure begins with a finite specification. A rule can describe how to produce a successor state, refine an approximation, extend a history, add an element to a growing structure, or continue an experiment. The rule contains no relevant internal terminal condition. Nevertheless, every realized execution of the rule has finite history, finite representation, and finite resource use at every observed stage.

This definition differs from a merely large finite process because there is no distinguished logical maximum built into the rule. It differs from actual infinity because no completed totality of all stages is required to exist as a realized object. It differs from the usual interpretation of potential infinity because continuation is not asserted as an unconditional future possibility. A new stage becomes part of the realized process only when an extension event occurs under some context and resource regime.

The formal core can be described without committing to a particular syntax. There is a space of states, a transition specification, a context that includes relevant resources and environmental assumptions, and an outcome type for attempted extension. An attempted extension may produce a successor state, but it may also produce a frontier result explaining why no successor has been realized under the current context. The frontier is not synonymous with mathematical impossibility. It records the boundary encountered by this realization.

Different frontiers should be distinguished. A computational frontier may be caused by memory, time, numerical representation, solver capacity, or communication. A physical frontier may be caused by energy, material, duration, or causal accessibility. An observational frontier may be caused by resolution, noise, or missing instrumentation. An epistemic frontier may arise because available evidence does not discriminate between models. An organizational frontier may appear when a collective can no longer coordinate enough participants to preserve its identity.

This plurality is essential. A proof that a process has no logical terminal state says nothing by itself about the absence of a physical frontier. Conversely, a particular machine reaching a memory limit does not prove that the mathematical process cannot be extended on another machine or under another representation. The theory must allow frontiers to move when context changes while preserving the provenance of the earlier result.

4.2 Frontier-sensitive theorems

Once frontiers become explicit, the form of a theorem changes. A conventional theorem often states a relation over a domain and leaves questions of implementation to later application. An outfinite theorem may additionally state what kind of domain is being quantified over, what resources or observations are relevant, and what happens when the analysis cannot extend its justification further.

This does not mean that every theorem must carry a physical budget. Purely theoretical theorems can remain theory-local and resource-independent at the level of meaning. The requirement is that a later bridge to computation or observation cannot erase the distinction. A theorem about exact real numbers may remain exact inside an ideal analysis, while a numerical application of the theorem must connect the ideal quantity to a finite representation and an error model.

Frontier-sensitive reasoning is particularly useful for bounded verification. Suppose a model checker explores every reachable state up to a depth of one thousand. The mathematically correct result is not simply "safe" or "unsafe." If no counterexample is found, the result is that no modeled counterexample exists within the explored bound, unless an independently established completeness threshold shows that the bound covers all reachable behavior. Outfinite semantics treats the unexplored region as part of the result rather than as a footnote.

The same applies to symbolic execution. If an engine explores a subset of paths and finds a concrete model that violates a claim, the counterexample can be fully formal even though the exploration was incomplete. If it finds no counterexample, incompleteness remains visible. Abstract interpretation can sometimes establish a global safety property without enumerating paths because the abstract semantics soundly over-approximates them. Its frontier is different: precision may be insufficient, or the abstraction may require widening that loses information. These differences later motivate a typed calculus of evidence.

4.3 Outfinite analysis and stabilization

Analysis provides a second family of examples. Traditional limits describe behavior as an index becomes arbitrarily large or as a variable approaches a point. The outfinite programme does not need to reject the classical definition. It asks whether practical mathematical knowledge can often be restated as a constructive stabilization contract.

A stabilization theorem has a finite challenge-response structure. Given a requested tolerance, the theory constructs a finite frontier beyond which every admissible finite stage satisfies the tolerance. The theorem does not require a stage called infinity. It provides a rule that turns a finite demand for precision into a finite obligation.

For the geometric series, for example, finite partial sums can be related to a residual that shrinks under each step. The classical statement that the infinite series has a sum of two is compact and elegant. The outfinite companion statement says that for any positive tolerance one can determine a finite stage at which the residual is below that tolerance, subject to the arithmetic representation used. The exact value two may remain a theoretical object; the realized computation produces certified finite information about it.

The same style can be extended to root finding, numerical integration, optimization, simulation, and statistical estimation. The critical concept is not approximation alone but the explicit relation between the ideal target, the finite evidence, and the residual uncertainty. A frontier is informative because it records what would be required to improve the result.

4.4 Frontiers as first-class scientific objects

The scientific importance of frontiers may exceed the foundational debate about infinity. Many failures in science and engineering are boundary failures. A model is extrapolated beyond the regime where it was validated. A numerical method becomes unstable. A security

argument assumes an adversary weaker than the deployed environment. A social indicator is used after the meaning of the measured category has changed. An AI system answers beyond the evidence in its context while presenting the result with the same linguistic confidence.

A mathematical language that treats frontiers as first-class objects could make such failures easier to detect. The frontier of a theorem would not be a warning attached by an author but a structured part of the claim. Theory composition would have to intersect or transform frontiers rather than silently discarding them. An AI reasoner could be required to show that the context of a requested conclusion lies inside the validated region of every theorem used.

This is a strong claim and should be approached cautiously. Scientific model validity is often difficult to characterize sharply. Boundaries may be fuzzy, empirically estimated, or dependent on variables not included in the model. An outfinite framework must therefore allow uncertain or provisional frontiers rather than pretending that every domain of validity can be made exact. The important requirement is that uncertainty about the boundary itself be represented.

4.5 Toward a calculus of frontier extension

Treating frontiers as first-class objects is useful only if the framework can reason about them. A frontier should not be a decorative field stating that a computation stopped after a thousand steps. It should participate in composition, comparison, and theorem transport. The research challenge is therefore to define a calculus in which one can say when a conclusion remains valid as a frontier is extended, when it must be recomputed, and when the very meaning of the analyzed system changes.

The simplest case is monotone extension. Suppose a verification result establishes that no modeled execution of length at most one hundred violates a safety property. Extending the bound to two hundred cannot invalidate the claim about the first hundred steps, although it may reveal a later violation. The earlier theorem survives as a theorem about a smaller horizon. Likewise, a concrete counterexample found within the smaller horizon remains a counterexample after the horizon expands, provided that the underlying semantics has not changed. These elementary monotonicity facts should be expressed as metatheorems of the frontier system rather than as informal expectations.

Other extensions are not monotone. Increasing numerical precision can change a branch decision, reveal instability, or invalidate an approximation whose error model assumed a particular representation. Increasing the strength of an adversary can destroy a cryptographic guarantee. Adding a physical interaction to a model can invalidate a safety theorem that was sound under the simplified dynamics. Changing the observer can split one phenomenological category into several. A frontier calculus must therefore distinguish extension of resources from revision of semantics.

This distinction suggests that a frontier has structure. It may include a horizon of time or steps, a resource budget, a representation, an observational resolution, a set of modeled interactions, and a theory version. Not every component needs to appear in every theorem. Pure arithmetic may have no physical time frontier, while a numerical simulation may have all of them. The core requirement is that the components relevant to validity be part of the claim rather than ambient assumptions.

A bridge between two frontiers should carry a witness explaining why a result can be transported. If a larger memory budget is the only change, a theorem about already explored

states may transfer automatically. If the numeric type changes from mathematical integers to fixed-width words, a range proof may be required. If a model of a physical process is extended with friction, a theorem derived under frictionless dynamics may have to be re-established or weakened. This makes extension an explicit mathematical event.

Frontier composition is equally important. Scientific workflows chain several analyses. A data-processing step may be validated only for a measurement range, a parameter estimation method only under distributional assumptions, a solver only for a stability region, and a final theorem only for a state-space approximation. The valid region of the composite result is constrained by all of them. A machine-checkable theory should be able to calculate or conservatively approximate the intersection of these frontiers and expose which component is limiting the conclusion.

The concept of residual is useful here. When an analysis stops, it should not merely return failure. It should return the part of the obligation that remains open. In symbolic execution, residual states are unexplored path conditions. In abstract interpretation, the residual may be an imprecise region that cannot be discharged. In numerical analysis, it may be an interval of remaining error. In experimental science, it may be a parameter regime lacking observations. Residuals make a frontier actionable because they describe what further computation, observation, or modeling would be needed to continue.

This leads to a research hypothesis with direct engineering implications. If frontiers and residuals are represented compositionally, AI agents can plan formal work rather than merely generate proofs. An agent can see that one theorem is blocked by a missing range lemma, another by an unverified translation, and a third by insufficient data. It can choose whether to call a solver, generate a test, refine an abstraction, request an experiment, or weaken the claim. The value of the outfinite approach would then appear not only in final certificates but in the organization of open mathematical work.

The concept can be falsified by comparison. Existing systems already track many bounds, proof obligations, and assumptions. A frontier calculus must demonstrate that treating them uniformly enables theorem composition or automated planning that would otherwise require bespoke glue. If no such calculus can be made precise, or if its semantics becomes so general that every domain needs a separate interpretation, the claim that frontiers belong in the foundation should be narrowed. The important result is not to force every limitation into one datatype, but to discover which classes of limitation share reusable mathematical structure.

4.6 Research obligations and falsifiability

The formal definition of an outfinite process must support nontrivial metatheorems. The programme should investigate conditions under which extending a resource context preserves previously established safety properties, conditions under which a counterexample remains valid under extension, and conditions under which changing representation invalidates a prior certificate. If no useful calculus of frontier extension can be derived, then the term outfinite may remain philosophical rather than mathematical.

A second test concerns classical limits and open-ended constructions. The programme should select a corpus of well-understood results and compare classical proofs with frontier-sensitive formulations. The relevant measures include theorem strength, proof complexity, computational usefulness, and clarity of assumptions. If outfinite formulations systematically

become verbose restatements with no gain in applicability or verification, the proposed foundation should be narrowed to domains where frontiers genuinely matter.

A third test is empirical. Real software, numerical models, and scientific workflows can be seeded with boundary errors: overflow beyond a threshold, unstable numerical regions, unsupported extrapolation, missing sensor precision, and verification cutoffs. An outfinite representation should detect or at least preserve these boundaries more reliably than a baseline that represents only the ideal property. The ability to expose boundary failure is one of the clearest ways to falsify or support the programme.

Part II. Requirements for a formal foundation

Chapter 5. Requirements for an Outfinite Core Logic

5.1 Why a research programme needs a small semantic core

A philosophical programme becomes a mathematical programme only when its central distinctions can be made precise enough to support derivation, counterexample, and independent implementation. The most important next step for outfinitism is therefore not the design of a friendly notation. It is the construction of a small semantic core in which existence modes, process identity, theory locality, frontiers, and evidence can be represented without ambiguity.

The provisional name Outfinite Core Logic, or OCL, is useful only as a label for this research target. At this stage it would be misleading to present OCL as an already established logic. The programme must discover whether one new logic is actually necessary. It may turn out that the right implementation is a disciplined extension of dependent type theory, a modal type system, a family of indexed logics, or a categorical framework that hosts several local logics. The foundational question is semantic rather than branding: what is the smallest formal machinery required to preserve the distinctions that motivate outfinitism?

The core should be small for two reasons. The first is epistemic. If the foundation depends on a large collection of sophisticated inference procedures, it becomes difficult to know which part of the system is responsible for soundness. The second is computational. A small kernel can validate artifacts generated by larger, more fallible systems, including theorem provers, abstract interpreters, SAT and SMT solvers, model checkers, and AI agents. This separation between discovery and checking is one of the most reusable lessons of modern formal methods.

The core must not, however, be small by deleting the very distinctions the programme was created to represent. A conventional proposition calculus plus untyped metadata would not be enough if existence modes and frontiers never constrain inference. The core needs rules that make illicit transfers impossible or at least explicit.

A candidate judgment should contain more than a proposition. It should identify the theory in which the proposition is interpreted, the context and assumptions on which it depends, the existence modes of the relevant entities, the kind of evidence produced, and any frontier limiting the claim. The exact data structure is a research question, but these dimensions must have semantic effect. A proof in an ideal theory and an observation in a physical experiment should not become interchangeable merely because both are stored as records with a field called result.

5.2 Theory locality and explicit bridges

The programme rejects the idea that every legitimate mathematical or scientific statement must live in one homogeneous theory. Instead, theories are treated as local semantic environments. A theory declares its primitives, its existence assumptions, the relations it considers fundamental, the logic it uses, and the transformations it accepts as valid.

This locality makes it possible to retain classical mathematics without granting every classical object the same status in every application. A set-theoretic theory may freely quantify over

completed infinite sets. A constructive theory may require witnesses for existence. A physical theory may use continuous fields as ideal objects while connecting them to measurements through an approximation model. A theory of a collective may define persistence through organizational invariants rather than member identity.

The difficult part is not coexistence but translation. A bridge between theories must specify what is preserved, what is approximated, what is forgotten, and which new obligations are introduced. A bridge from exact real arithmetic to floating-point arithmetic, for example, is not a renaming operation. It changes equality, introduces rounding behavior, may destroy associativity, and requires error bounds. A bridge from a legal model of an organization to a functional model must explain which legal facts count as evidence of operational coordination and which do not.

This bridge calculus is a natural place for category theory, logical relations, refinement types, and modal operators to contribute. Yet the programme should avoid choosing a heavy formalism before the requirements are clear. The key semantic property is that a conclusion transported across a bridge must carry enough provenance to reconstruct the assumptions under which the transport was valid.

Bridges also provide a disciplined interpretation of scientific reduction and approximation. A molecular model and a continuum model need not be declared globally equivalent. The bridge can state that, under certain scale assumptions, specified observables of the molecular model are approximated by the continuum model within a tolerance. This is more compatible with actual scientific practice than a single ontology in which one model must be declared the ultimate reality and the other merely derivative.

5.3 Identity, negation, and open status

Typed existence requires a corresponding discipline of identity. The core should distinguish exact identity of immutable values from persistence of processes, observational equivalence of implementations, recurrence of phenomena, and continuity of collectives. These relations can share algebraic properties where appropriate, but they should not be collapsed into one unqualified equality.

Negation also becomes more subtle. In classical logic, not exists x can be meaningful inside a fixed domain. In an open empirical context, failure to observe x is not the same as proving that x does not exist. In a bounded computational search, failure to find a counterexample is not equivalent to proving its absence. The core therefore needs to keep proof-theoretic negation separate from statuses such as not observed, not constructed, unreachable within a bound, or unsupported by available evidence.

This requirement motivates a richer set of result statuses. A query may be proven, refuted, unresolved, blocked by a frontier, or locally inconsistent. These are not five truth values in one logic. They are statuses of a reasoning process. A theorem can still be classically true or false within a local theory while the system's current knowledge about it is open. This distinction is essential for AI systems, which otherwise tend to turn lack of evidence into a linguistically definite answer.

A core logic that tracks such statuses should also localize contradiction. Scientific repositories routinely contain incompatible measurements, rival models, and versioned assumptions. A contradiction in one context should not trivialize unrelated contexts. Paraconsistent logics may help at the evidence layer, while classical or intuitionistic logic may remain appropriate

inside specific deductive modules. The foundation should therefore be modular enough to avoid a single global rule of explosion where it would be epistemically destructive.

5.4 Resource and frontier awareness without making everything physical

One possible failure mode of outfinitism would be to attach time, memory, and energy budgets to every mathematical proposition, thereby confusing the semantics of mathematics with the behavior of current hardware. The programme should avoid this. A theoretical result can be independent of any particular machine. Resource awareness becomes mandatory when a claim crosses into an executable realization or when the proof procedure itself reports a bounded result.

This suggests a layered semantics. At the theoretical layer, a function may be total in the model. At the computational layer, an implementation of the function may be partial because of representation or resource limitations. A bridge states the conditions under which implementation results correspond to theoretical values. The fact that the computational layer can reach a frontier does not retroactively make the theoretical function partial; it reveals that the realization is partial.

The same distinction applies to proof search. A theorem may be true in a theory even when the current prover times out. The timeout is a frontier of the search process, not a refutation of the theorem. Conversely, a verified proof certificate can establish the theorem even if the search procedure that found it was heuristic. The core should therefore distinguish the semantics of a claim from the operational status of the procedure that attempts to justify it.

This separation is important for AI. An AI agent can be arbitrarily complex, nondeterministic, and difficult to verify. If it produces a certificate accepted by a small checker, the validity of the certificate need not depend on trusting the agent. If it fails, its failure should remain a frontier of search rather than an epistemic conclusion about the underlying mathematics.

5.5 Candidate semantic architectures and why surface syntax must come later

The earliest versions of an outfinite language are naturally tempting to write in a controlled form of mathematical English. Such notation is useful for thinking because it exposes hidden assumptions and can be read by researchers who are not specialists in proof assistants. It would be a mistake, however, to infer the foundation from the convenience of the notation. A controlled natural language can be one interface to the system, but the research programme first needs a semantics that is small enough to analyze independently of wording.

There are several plausible semantic architectures, and choosing among them is itself a research problem. A dependent type-theoretic architecture could make witnesses and existence modes part of types, allowing the type checker to reject unsupported transfers. A modal type theory could represent theoretical, physical, observed, computed, or possible existence by modalities with explicit introduction and elimination rules. An indexed logic could treat each theory, resource profile, or observational regime as an index on judgments. A categorical semantics could organize these local systems and their translations. A hybrid design may be required, but hybridization should follow from formal obligations rather than from a desire to include every attractive technique.

Whatever architecture is chosen, several properties are non-negotiable. The core must distinguish assertion from evidence. It must preserve the theory under which a proposition is meaningful. It must make bridges explicit when evidence or entities move between theories. It must allow open or frontier-limited status without collapsing it into falsity. It must be possible to check a certificate with a smaller trusted component than the engines that produced it. Finally, the core must support modular extension so that arithmetic, category theory, probability, temporal logic, or empirical evidence can be added without turning the kernel into an encyclopedia of mathematics.

The semantics should also make clear what remains classical. Outfinitism does not need to reject classical propositional reasoning in every context. A finite decidable proposition can still obey ordinary excluded middle. Exact algebraic equality can remain exact. A theory may even import classical set theory and completed infinite objects as theory-local entities. The novelty lies in controlling the conditions under which those entities and rules are exported into other contexts. This modularity is important because a foundation that unnecessarily forbids established mathematics would create adoption costs without corresponding scientific benefit.

A useful design target is a small judgmental interface. Conceptually, the kernel should be able to check statements of the following kind: under a named theory and a finite context of assumptions, a claim has a particular evidence status and is valid within a declared frontier. The exact notation is not important. What matters is that the proof object or certificate preserves enough information for later composition. If a theorem depends on an idealization, a bounded search, a numerical approximation, or an empirical bridge, this dependence should survive simplification and reuse.

This requirement creates a productive tension with convenience. Human mathematicians prefer concise notation and routinely suppress inferable context. Machines benefit from explicit structure. The future language should therefore separate a human-facing surface from a machine-facing core. The surface may use conventional symbols, controlled natural language, or domain-specific notation. Compilation would elaborate omitted information, resolve names, attach theory and existence types, and produce a canonical intermediate representation. The elaborator can be complex and AI-assisted; the checker should remain small.

This separation also creates a practical path for adoption. Existing mathematical notation and proof languages do not need to be replaced. Translators can map fragments of Lean, Isabelle, SMT-LIB, probabilistic programs, scientific Python, or domain-specific simulation models into the core representation, while export tools can render verified results back into familiar forms. The foundation is then judged by interoperability and semantic preservation rather than by whether researchers enjoy a new syntax.

The syntax question should return only after the semantic experiments have stabilized. At that stage, a controlled natural-language layer may prove valuable because AI systems can generate and revise it more reliably than dense notation, and domain experts can review it without learning a low-level proof language. But the language should be presented as an engineered interface whose correctness is established by elaboration and checking, not as the source of rigor itself. If the semantic core cannot be specified without relying on the ambiguities of the surface notation, the project has not yet reached a credible foundation.

5.6 Research obligations and falsifiability

The core logic should be treated as a design problem with explicit competitors. At least one prototype should be implemented inside an existing dependent type theory, one using a modal or indexed type discipline, and one using a more direct operational semantics. The programme should compare how naturally each expresses existence modes, bridges, frontier-sensitive results, and theory locality. If an existing formalism expresses all requirements without awkward encodings, a new foundational calculus may be unnecessary and the programme should adopt that result rather than defend novelty for its own sake.

The semantic rules should also be tested for conservativity. When an outfinite theory embeds a classical fragment, ordinary theorems should not change meaning merely because additional existence information is available. Conversely, when two existence modes are distinguished, the system should block at least some inferences that an untyped embedding would permit. These properties can be mechanized in a proof assistant.

A final falsification criterion is independent implementation. Two small checkers built from the same public semantics should agree on a corpus of certificates. If the semantics is too informal for independent teams to reproduce, then OCL has not yet become a formal foundation.

Chapter 6. Theories, bridges, evidence, and the limits of formal truth

6.1 Deductive truth is only one scientific relation

A major risk in the formalization of science is to mistake formal proof for scientific truth. A theorem prover can establish that a conclusion follows from axioms and assumptions. It cannot, by that act alone, establish that the assumptions describe the physical world. The strength of formal verification is exactly its precision about implication; its weakness appears only when users forget the conditional nature of that implication.

Outfinite mathematics should make this conditional structure visible. A scientific statement may have several components: a theoretical claim, an empirical observation, a model-to-observation mapping, an uncertainty model, and a deductive consequence. These components require different forms of justification. The theoretical claim may be proven. The observation may be instrumentally certified. The mapping may be calibrated. The uncertainty model may be statistically supported. The final scientific conclusion depends on the integrity of the entire chain.

This is why the programme should not define proof as a single undifferentiated token. Instead, it should develop a typed evidence calculus in which different evidence objects have different admissible uses. A deductive certificate justifies a logical implication within a theory. A counterexample justifies refutation of a universal claim under the semantics used. A measurement record supports an empirical proposition under assumptions about instrumentation. A statistical interval supports a probabilistic or inferential statement, not a logical certainty. A simulation trace establishes what a particular implemented model did, not what nature must do.

The same distinction applies within formal methods. A proof that software is memory-safe in a machine model is deductively strong, but the result can still fail to describe a physical execution if hardware faults or unmodeled foreign code violate the model. Soundness should therefore always be read as soundness relative to a semantics and a trusted base, not as metaphysical infallibility.

6.2 Provenance as a mathematical dependency

Scientific reproducibility requires more than storing a citation. A claim depends on theory versions, parameter values, datasets, calibration choices, algorithms, numerical representations, and proof procedures. When any of these changes, some conclusions may cease to be justified.

Outfinite mathematics should treat provenance as a dependency graph that can be queried and transformed. This does not mean that every file path or software commit belongs in the logic. It means that the formal claim should refer to the artifacts whose semantics matters to its validity. A numerical theorem may depend on a particular interval arithmetic implementation and a verified rounding model. An empirical claim may depend on a dataset with a defined acquisition protocol. A bridge between two theories may depend on a calibration study.

Once provenance is structural, theory review becomes more powerful. A change to an axiom can trigger identification of all theorems whose certificates depend on it. A revised measurement can identify which fitted parameters and conclusions should be recomputed. A discovered implementation error can invalidate only the claims that depended on that implementation rather than forcing an indiscriminate retraction.

AI agents are particularly well suited to maintaining such graphs because the administrative burden is high and much of the work is repetitive. They can trace references, compare theory versions, regenerate proofs, and flag broken dependencies. Yet the graph becomes useful only if the meaning of its edges is formally constrained. A textual similarity between two definitions is not a proof that they can be substituted.

6.3 Bridges as explicit scientific hypotheses

A bridge between theory and reality is often itself a scientific hypothesis. Consider a continuum model of fluid behavior. The equations may be derived and manipulated exactly inside the model. The claim that a particular experiment is adequately described by those equations depends on scale, regime, boundary conditions, material properties, and measurement assumptions. The bridge is not a logical tautology; it is part of the empirical content of the science.

This suggests an important extension of falsifiability. Theories are not the only things that can be falsified. Bridges can fail. A model may remain mathematically coherent while its bridge to a class of physical systems becomes empirically unsupported. An implementation may cease to refine a specification after an optimization. A statistical model may remain well defined while its sampling assumptions no longer match the data-generating process.

Representing bridges as named, testable objects could make scientific revision more precise. Instead of declaring an entire theory "wrong," one can identify which bridge failed and which internal results remain valid. This is closely aligned with meta-rationality: theories can remain useful in their proper contexts even when their universal interpretation is rejected.

The bridge concept also helps explain idealization. An infinite-dimensional space, a continuous field, or an exact real number can be fully legitimate inside a theory. What requires justification is the map from finite observations and computations to that ideal object. Outfinitism does not need to deny the ideal object. It needs to prevent an unnoticed change of existence mode.

6.4 The empirical layer as a missing foundation

The current programme is stronger on deductive structure than on empirical semantics, and this should be acknowledged as a major open problem. A foundation for executable science must eventually represent measurement, uncertainty, experimental procedure, sampling, calibration, causal intervention, and model comparison in a way that interacts coherently with deductive proof.

No single existing logic solves this problem. Probability theory captures uncertainty under a model. Statistics studies inference from data. causal models formalize some intervention structures. metrology provides disciplined accounts of measurement and uncertainty. scientific workflow systems capture provenance. Formal epistemology distinguishes knowledge and belief. An outfinite empirical layer should integrate selected parts of these traditions without pretending that empirical support becomes deductive certainty.

A promising direction is to treat empirical claims as judgments indexed by an evidence model and a validity context. The statement would record what was observed, how the observation relates to theoretical quantities, what uncertainty remains, and what competing explanations are excluded. The exact formalism remains open, but the central prohibition is clear: empirical support must not be silently promoted into proof, and proof inside a model must not be silently promoted into observation.

6.5 From formal truth to scientific warrant

Executable science requires a stronger distinction than the familiar separation between theorem and experiment. Scientific conclusions often sit in chains that alternate between deduction, computation, measurement, statistical inference, calibration, and modeling judgment. A formalism that recognizes only the deductive parts can verify local mathematics while remaining blind to the points at which the argument acquires empirical meaning. The outfinite programme therefore needs a theory of scientific warrant that is precise without pretending that empirical support is the same thing as proof.

Consider a simple parameterized physical model. The differential equations may be derived deductively from assumptions. Parameters may be estimated from data. A numerical solver approximates the equations. Sensors have calibration uncertainty. The final prediction is compared with observations. At least four different relations are present: logical consequence inside the model, statistical estimation, numerical approximation, and empirical agreement. If the workflow is represented as one undifferentiated chain of formulas, the final conclusion can appear more certain than any individual bridge justifies.

The proposed foundation should instead treat each bridge as a scientific hypothesis with its own evidence and failure conditions. A calibration bridge links instrument readings to a physical quantity. A model bridge links a theoretical variable to a phenomenon. A numerical bridge links an exact equation to an implemented approximation. A statistical bridge links observed samples to a population or generative model. The composition of the workflow is trustworthy only to the degree that the relevant bridges are trustworthy in the regime where they are used.

This view gives provenance a mathematical role. Provenance is not merely bibliographic metadata attached after the fact. It records which transformations, datasets, assumptions, theory versions, and instruments support a conclusion. If an assumption changes, the dependency graph identifies which derived claims require reconsideration. If a dataset is retracted or a calibration is corrected, the system can locate affected conclusions. Such dependency tracking is routine in build systems and software package managers; scientific knowledge lacks an equally universal formal infrastructure.

Evidence also has direction. A successful prediction can support a model without proving that the model's theoretical entities exist exactly as represented. A counterexample may refute a universal claim while leaving neighboring models viable. A failed measurement may reflect an instrument problem rather than a theory failure. The formalism must therefore preserve the object of refutation. Executable science becomes scientifically dangerous if the system is allowed to upgrade local mismatch into global falsification without representing alternative explanations.

A mature framework will probably require several uncertainty dimensions rather than one confidence number. Measurement uncertainty, statistical uncertainty, model uncertainty,

logical openness, approximation error, and source credibility are not interchangeable. Combining them indiscriminately creates false precision. The outfinite programme can contribute by providing typed interfaces between these forms of uncertainty and by requiring explicit conversion rules when they are aggregated.

The relevance to AI is substantial. Current language models can summarize evidence chains, but they may compress distinct warrants into a fluent narrative. A machine-readable scientific theory could force an AI reviewer to ask whether each transition is licensed. The system could detect that an exact theorem is being applied to a floating-point implementation without a numerical bridge, that a population-level estimate is being used as an individual prediction, or that a model validated in one parameter range is being extrapolated outside it. These are precisely the kinds of errors that can remain rhetorically plausible while being scientifically invalid.

The research challenge is to formalize enough of this structure to be useful without claiming to mechanize scientific judgment in full. Human interpretation will remain necessary, especially in the construction of models and in deciding which evidence matters. The goal is narrower: once a scientific argument declares its models, data, bridges, and evidence relations, machines should be able to check consistency, dependency, applicability, and many forms of mismatch. If these checks cannot be defined with domain-independent semantics, executable science may need to remain a collection of domain-specific systems rather than a unified outfinite layer.

6.6 Research obligations and falsifiability

This chapter creates a demanding experimental programme. Scientific case studies should be chosen in which the same mathematical structure has different empirical interpretations. For each case, the formalization should separate internal theorem, empirical evidence, and bridge. Reviewers should then introduce controlled failures, such as a changed calibration, an invalid sampling assumption, or a numerical representation bug. A successful dependency system should invalidate exactly the conclusions affected by the change.

The evidence calculus should also be compared with existing provenance and formal-science systems. If a conventional knowledge graph with typed edges provides the same error localization and proof integration, then the outfinite layer should reuse that technology rather than inventing a duplicate. The research claim is not that provenance is new; it is that provenance becomes semantically entangled with existence and proof in a way that warrants a common foundation.

The most difficult falsification test concerns empirical bridges. Independent scientists should be able to disagree about a bridge while agreeing on the formal consequences conditional on that bridge. If the framework hides substantive scientific judgment inside opaque formal declarations, it has failed to improve transparency.

Chapter 7. The theorem as a typed finite justification

7.1 Why proof must be reconstructed without being weakened

The phrase executable proof can be misunderstood in two opposite ways. One interpretation reduces proof to testing: if a program has been executed many times without failure, the property is treated as established. That is not acceptable. The other interpretation insists that only traditional deductive derivations count as formal justification, thereby excluding sound static analyses and mechanically checked abstraction arguments that can establish global properties without enumerating cases. The outfinite programme should avoid both mistakes.

A formal proof is best understood by the guarantee it provides relative to a precise semantics. If a method produces a finite certificate that a small checker can verify and if the checker is sound for the declared semantics, the result is a formal justification of the corresponding claim. The internal route by which the certificate was discovered may involve search, heuristics, AI, symbolic execution, abstract interpretation, rewriting, or human insight.

This view shifts attention from the method's historical label to the structure of the result. A theorem should identify the claim, the theory, the assumptions, the domain or coverage, the semantics of relevant types, and the evidence object that justifies the conclusion. If the claim is bounded, the bound is part of the theorem. If the proof uses an abstraction, the soundness relation of that abstraction is part of the trusted argument. If the result depends on an unverified solver, the trusted base is larger than if the solver emits a certificate checked independently.

The finite nature of proof objects is central to outfinitism. Even when a theorem concerns a theoretical infinite object, the realized proof is a finite inscription or finite machine representation. This observation does not trivialize classical proof. It makes explicit the difference between the potentially unbounded domain of a theorem and the finite structure that justifies a general statement about that domain.

7.2 Parametric generality instead of imagined enumeration

One recurring misunderstanding about finite or constructive foundations is that a universal theorem must be proven by checking every element. That would make almost all interesting mathematics impossible. Outfinite mathematics instead relies on parametric generality.

A proof of a formula for natural numbers need not enumerate all natural numbers. It can reason about an arbitrary finite value presented with the properties guaranteed by the relevant domain. An invariant can establish that every transition of a process preserves a property, without enumerating every possible execution trace. A symbolic identity can be normalized for arbitrary variables. A type can ensure that any inhabitant satisfies a constraint.

The key distinction is between a finite representation of a rule and a completed collection of all applications of that rule. A proof can be finite because it reasons about structure. This is already how mathematics and program verification achieve generality. The outfinite programme emphasizes the point because it allows universal reasoning without requiring the ontological claim that all possible instances are physically or computationally realized.

Induction is an instructive example. It would be misleading to claim that outfinite mathematics has abolished induction simply because a verifier checks an invariant. An invariant-preservation argument has inductive structure. The more accurate claim is that induction can often be operationalized as a finite verification of a base condition and a preservation rule, with the domain and transition semantics explicit. This is conceptually useful for machine reasoning because it ties the general theorem to an executable proof obligation.

Reduction to contradiction should be treated similarly. A satisfiability solver can prove that assumptions combined with the negation of a goal are unsatisfiable, but the strength of the result depends on the encoded theory and the solver's certificate. If the solver returns unknown or reaches a timeout, no contradiction has been established. Outfinite proof statuses preserve that distinction.

7.3 Counterexamples, witnesses, invariants, and residuals

The theorem calculus should recognize several forms of finite evidence. A constructive witness establishes an existential claim. A counterexample refutes a universal claim. An invariant can justify a safety property over all traces generated by a transition system when initialization and preservation are proven. An unsatisfiability certificate establishes the absence of a model for a formula in a specified logical theory. An equivalence certificate establishes that two representations preserve a declared behavior. An interval or error certificate can establish that a computed result encloses an ideal quantity.

Not every method can produce every kind of evidence. Symbolic execution is excellent at producing path conditions and counterexamples. Abstract interpretation is excellent at producing inductive invariants or safe over-approximations. Bounded model checking can produce finite traces and bounded absence results. Deductive verification can discharge general proof obligations. Equivalence checking can establish refinement or behavioral identity. Taint and dataflow analyses propagate abstract properties whose meaning depends on a formal transfer semantics.

An outfinite theorem should also be able to carry a residual: what has not been justified. This is especially important for partially completed analyses. A symbolic executor may have proven some paths safe, found no failure in others, and left a set of unexplored branches. A numerical method may have reduced error to a known interval but not to the requested tolerance. A scientific model may explain observations in one regime and have no validated bridge outside it.

The residual is not a weakness to be hidden. It is a structured description of the current frontier. In research practice, residuals can guide the next computation, experiment, abstraction refinement, or proof search. For AI, they provide a more honest action space than a forced binary answer.

7.4 Proof composition

Complex claims rarely depend on a single proof method. A realistic safety theorem may combine a type-system guarantee, an abstract interpretation invariant, a solver certificate for a nonlinear condition, a bounded state exploration for a finite controller, and an empirical assumption about hardware timing. The foundational challenge is to compose these results without losing the conditions attached to each one.

Proof composition should therefore be typed. An evidence object can be used only where its conclusion, context, theory, and existence modes match the premise expected by the next step. If a method proves a bounded claim, composition must preserve the bound unless a separate argument lifts it. If a bridge is approximate, downstream claims must inherit the approximation or prove that it is irrelevant. If two components rely on incompatible models, composition should fail rather than silently choose one.

This is where category-theoretic and algebraic perspectives may become useful. Evidence objects and theory morphisms can potentially form compositional structures in which associativity and identity are explicit. Yet the programme should earn this abstraction through working examples. Category theory should simplify composition, not merely decorate it.

7.5 A typed calculus of proof outcomes

The most important correction to a binary idea of proof is not to weaken mathematical rigor but to type the scope of rigor. A sound theorem prover, an abstract interpreter, a bounded model checker, a symbolic executor, a numerical certifier, and an empirical analysis can all produce mathematically meaningful artifacts, but they justify different claims. A common framework should make those differences compositional.

The word soundness is central but frequently overstated. In formal logic, a proof system is sound when derivable conclusions are true in the intended semantics whenever the premises are true. This is a conditional guarantee. It does not certify that the intended semantics is a perfect model of hardware, biology, economics, or laboratory practice. A program proof can therefore be completely sound with respect to a source-language semantics while a compiler bug or hardware fault invalidates the deployed behavior. The appropriate response is not skepticism about formal proof; it is to make the trusted chain explicit.

Completeness is a separate property. An analysis can be sound and still fail to prove many true properties. Abstract interpretation often chooses this tradeoff deliberately: it over-approximates behavior to avoid missing relevant executions, accepting false alarms when the abstraction is too coarse. Symbolic execution often makes the opposite local tradeoff: it reasons precisely about explored paths while leaving other paths unexplored. A theorem prover may have a sound kernel while its automated search is incomplete. Conflating soundness with completeness produces misleading classifications of methods.

Termination is separate again. Some mathematically well-defined analyses may not terminate on every input. In ordinary tooling, this appears as a timeout or resource exhaustion. Outfinite semantics should elevate the event into a frontier result. The residual proof obligations remain part of the artifact, allowing later analysis to continue or another engine to take over. This is especially important for AI agents, which need to distinguish "the theorem is false" from "this tactic did not finish" and from "this bounded search found no counterexample."

Coverage supplies another dimension. A bounded model checker can be exhaustive over all traces up to a depth, even though it is not exhaustive over unbounded behavior. Symbolic execution may cover all branches of a loop-free finite program and thereby prove a postcondition globally for the specified input domain. Concolic testing can produce excellent empirical path coverage without a completeness guarantee. A proof artifact should state

which region has actually been closed and what argument, if any, promotes local coverage to a universal theorem.

The nature of the returned artifact is equally important. A concrete counterexample is often stronger than the global completeness of the search strategy that discovered it. A fuzzer may be heuristic, yet a reproducible input that violates a universal safety claim is a decisive witness. Conversely, the absence of a discovered counterexample is weak unless the method has an independent coverage argument. This asymmetry is foundational for a calculus that combines exploratory and deductive methods without confusing them.

The outfinite theorem can therefore be viewed as a finite justification package. It contains a claim, the theory and semantics in which the claim is interpreted, its assumptions, the existence modes of relevant entities, the coverage and frontier of the analysis, and a certificate or witness. The certificate may be a proof term, an unsatisfiability proof, an inductive invariant, a trace, a numerical enclosure, or another artifact recognized by the theory. This is not a proposal to make every theorem verbose on the page. Most of the information can be inferred or stored internally. The requirement is that it remain recoverable and machine-checkable.

Proof composition then becomes a typed operation. Two certificates can be combined only when their theories, assumptions, frontiers, and semantic bridges are compatible. A global theorem about mathematical integers cannot be applied directly to an implementation using fixed-width arithmetic unless the relevant range has been proved. A proof about an abstract model cannot be transported to a physical device without a refinement or conformance argument. A statistical guarantee cannot be treated as a logical necessity. These restrictions encode common scientific cautions as formal composition rules.

This calculus would also provide a natural role for AI. Coding agents and theorem-proving agents can search for invariants, generate lemmas, choose abstractions, propose counterexamples, and call solvers. Their search strategies can be probabilistic and fallible because the final artifacts are checked independently. The AI becomes a powerful producer of candidate justifications rather than the ultimate source of trust. The programme should consider this division of labor a core design principle.

The proposal is falsifiable in a straightforward way. A typed proof calculus should allow heterogeneous methods to cooperate while preventing invalid promotion of bounded, approximate, or empirical results into stronger forms of certainty. Benchmarks can be constructed in which naive orchestration makes exactly these mistakes. If an existing proof framework can capture the same distinctions more simply, or if the typed calculus adds bureaucracy without preventing real errors, then this part of the outfinite foundation has not justified itself.

7.6 Proof as a finite circuit of justified transformations

A useful implementation model is to regard a proof not as a linear paragraph but as a finite directed structure of claims and transformations. Human exposition often presents reasoning sequentially because prose is sequential. Formal reasoning is naturally graph-shaped: several premises feed a lemma, one lemma is reused in several branches, and different engines may discharge independent obligations. An SSA-like or circuit representation makes this dependency explicit without requiring the surface mathematical language to look like code.

Each node in such a proof circuit would produce a finite value: a proposition, witness, invariant, abstract state, counterexample, approximation, or bridge certificate. Each edge would record why the output of one step is admissible as input to another. The graph preserves provenance and allows incremental rechecking. If one assumption changes, only downstream nodes need to be reconsidered. This property is essential for scientific theories, which evolve more frequently than traditional textbook theorems.

The circuit view also clarifies the role of open proof states. A proof need not be either globally finished or useless. Some branches may be certified while others terminate at frontiers. The residual nodes form an explicit research agenda. An AI agent can attempt to close them with a different method, strengthen an invariant, refine an abstraction, or request additional evidence. Human researchers can inspect the same graph and understand where the uncertainty lies.

This representation should remain semantically conservative. A graph does not create new logical power merely by being executable. Every transformation must correspond to a rule justified in the relevant theory. The advantage is architectural: proof search, analysis, and scientific computation can share a common dependency substrate. The research question is whether this substrate supports enough methods without becoming an unmanageable universal workflow language.

A direct falsification test is to encode the same nontrivial proof both as an ordinary proof-assistant development and as a compositional outfinite proof circuit. The circuit should preserve theorem strength while enabling at least one additional capability, such as cross-engine evidence, incremental revalidation, explicit frontier tracking, or automatic planning. If the representation only reproduces the original proof with more metadata, it should not be treated as a foundational contribution.

7.7 Research obligations and falsifiability

The proof calculus can be falsified by cross-tool examples. The same property should be established using several techniques and then represented as evidence objects in a common framework. Independent checkers should confirm that bounded, abstract, deductive, and equivalence results retain their precise meaning after composition. Any case where a bounded guarantee is accidentally promoted to an unbounded theorem is a direct failure of the design.

The programme should also formalize representative textbook proofs and software-verification arguments. If the typed theorem representation is too cumbersome for ordinary mathematics, that is evidence that the core should remain hidden below a higher-level interface. If even the hidden representation cannot express conventional proofs without excessive duplication, the foundational design is wrong.

A particularly important benchmark is proof repair under changing assumptions. When an axiom, bridge, or abstraction changes, the system should identify exactly which certificates remain valid. This is both a logical and an economic test of the proposed evidence architecture.

Chapter 8. Formal analysis methods as complementary evidence-producing engines

8.1 Why a binary classification of methods is misleading

Formal methods are often classified by asking whether a technique is or is not a "formal proof." That question is too coarse for the goals of outfinite mathematics. Formality, soundness, completeness, termination, coverage, and certificate production are different properties.

A method is formal when its semantics and transformations are mathematically specified. It is sound for a property when any positive guarantee it produces is valid relative to that semantics. It is complete for a class of questions when it can, in principle under the stated conditions, decide every question in the class or derive every valid result of the relevant kind. It terminates when its analysis procedure is guaranteed to finish. Coverage describes which states, paths, inputs, or behaviors the result applies to. Certificate production concerns whether the result can be independently checked without trusting the entire analysis engine.

These axes explain why familiar slogans are often inaccurate. Symbolic execution is not inherently non-formal; it is a formal execution semantics over symbolic values. In practice it is often incomplete because path explosion forces exploration to stop. When it explores all relevant paths of a finite program or is combined with verified summaries, it can contribute to a complete proof. Abstract interpretation is not automatically a proof merely because it uses lattices; its guarantee depends on a sound abstraction and transfer semantics. Bounded model checking is a full formal proof of a bounded statement, and it can establish an unbounded property when a completeness threshold is independently known. Dataflow and taint analyses can be sound formal analyses for precisely defined properties even if they are commonly used as compiler or security tools rather than theorem provers.

Outfinite mathematics benefits from this multidimensional view because it aligns method classification with typed evidence. The method need not be assigned a permanent status. Its particular run produces a result with a particular guarantee and frontier.

8.2 Deductive verification and theorem proving

Deductive verification begins with specifications such as preconditions, postconditions, invariants, and contracts, following the tradition of axiomatic program semantics (Hoare 1969). The program or mathematical object is translated into proof obligations. A theorem prover or SMT solver attempts to establish those obligations. When the obligations are proven in a sound formal system, the resulting guarantee can be general and unbounded with respect to the modeled inputs.

The method is powerful because it reasons symbolically about all cases described by the specification rather than exploring them individually. Its limitations are equally important. Specifications can be wrong or incomplete. Program semantics can fail to model the deployed environment. Automation may require difficult invariants. The trusted base may include a large solver unless certificates are independently checked.

In an outfinite architecture, deductive verification is a major evidence producer but not the sole definition of proof. Its role is clearest when its assumptions and semantic bridge to implementation remain explicit.

Dependent type systems move some proof obligations into type checking, continuing a constructive tradition exemplified by Martin-Löf type theory (Martin-Löf 1984). A value can inhabit a type only if associated logical constraints are satisfied. The propositions-as-types correspondence provides an elegant route from mathematical specification to checkable artifacts. For outfinitism, dependent types are especially promising because existence modes and frontiers can potentially be indexed directly in types. The research question is how far such indexing can go before the type system becomes unusably complex.

Equivalence checking addresses another foundational need. Scientific and engineering workflows constantly transform representations: a specification becomes code, code becomes optimized code, a mathematical expression becomes a numerical algorithm, a high-level theory becomes a simulation model. Equivalence or refinement checking can establish that the transformation preserves a declared semantics. This is central to the outfinite idea of explicit bridges.

8.3 Abstract interpretation and dataflow analysis

Abstract interpretation replaces exact execution states with abstract properties that safely summarize families of states (Cousot and Cousot 1977). Interval analysis may record that a variable lies between two bounds without knowing its exact value. Sign analysis may record that it is positive, negative, or unknown. More sophisticated domains track relational constraints, shapes, resources, or information flow.

The method is particularly congenial to outfinitism because it makes approximation explicit and can prove global properties without enumerating concrete states. It also raises a foundational difficulty: classical abstract interpretation is often presented using complete lattices and fixpoints that may rely on infinite mathematical structures. An outfinite reconstruction should distinguish the ideal theory of the abstract domain from the finite analysis actually performed.

The practical insight is that a global safety proof often requires only a sound inductive post-fixpoint, not the material realization of a completed infinite computation. If an abstract state contains all initial states, is closed under the abstract transition, and entails the desired property, it functions as a finite certificate of safety. Widening can force convergence but may lose precision. That loss should be recorded rather than hidden.

Dataflow analysis can be understood as a structured family of abstract interpretations over control-flow graphs. Reaching definitions, liveness, constant propagation, and available expressions propagate abstract information until a stable solution is reached. The analysis can be fully formal and sound for its defined property even though its primary use is compiler optimization. The outfinite lesson is that "used for optimization" and "capable of producing a formal guarantee" are not mutually exclusive categories.

Taint analysis is another instance of abstract propagation. Inputs are marked with an information-flow property, and the analysis approximates where that property can reach. A taint analysis may miss implicit flows or reflection if its semantics does not model them, but that is a limitation of the declared abstraction, not evidence that taint analysis is inherently informal. A sound taint model can prove absence of flows within its semantics.

8.4 Symbolic execution, bounded model checking, and CEGAR

Symbolic execution runs a program over symbolic inputs (King 1976). Branches produce path conditions that characterize families of concrete executions. When a path condition is satisfiable and the path reaches an error, a solver can construct a concrete input that witnesses the failure. This makes symbolic execution one of the strongest sources of finite counterexamples.

Its difficulty is path explosion. Loops, recursion, concurrency, and environmental interaction can create more paths than can be explored. An outfinite engine should never convert exhaustion of its search budget into a universal safety theorem. The unexplored symbolic states become a frontier object that can later be summarized, abstracted, prioritized, or transferred to another method.

Concolic execution combines concrete execution with symbolic tracking. A concrete input drives one path while symbolic constraints are collected and modified to generate new inputs. This is highly effective for bug discovery and automated testing. Its normal output is coverage and concrete witnesses rather than a proof of absence. Yet the method still belongs in the same evidence architecture because the traces and constraints it produces can be formally meaningful.

Bounded model checking unrolls a transition system to a finite depth and translates the existence of a bad execution into a SAT or SMT problem (Biere et al. 1999). If the solver finds a model, the model is a bounded counterexample. If the formula is proven unsatisfiable, no counterexample exists within that bound. If a completeness threshold has been established, the same bounded result can become a global safety proof. This conditional promotion is exactly the kind of transition the outfinite theorem calculus should represent explicitly.

Counterexample-guided abstraction refinement, or CEGAR, demonstrates how methods can cooperate (Clarke et al. 2000). An abstract analysis produces a potential counterexample. If the trace is spurious, information from the failed concrete check refines the abstraction. The loop continues until a real counterexample is found, a proof is obtained, or a frontier is reached. CEGAR is almost a model of the outfinite research attitude: reason with a finite abstraction, refine when evidence demands it, and do not assume the process must terminate for every problem.

8.5 Partial evaluation, symbolic interpretation, and transformation evidence

Partial evaluation specializes a program with respect to known inputs. It is primarily a transformation technique rather than a proof technique. Nevertheless, its correctness can be stated as an equivalence theorem: the specialized program should behave like the original program under the fixed inputs and the remaining dynamic inputs. The transformation can therefore participate in a proof chain if its equivalence is certified.

Symbolic interpretation is a broader term for rewriting and evaluating expressions according to algebraic or semantic rules without necessarily executing concrete code. Computer algebra systems, interpreters for domain-specific languages, simplifiers, and compiler front ends all perform forms of symbolic interpretation. Their outputs are useful only to the extent that the rewrite rules preserve the intended semantics. In a high-assurance outfinite system,

important symbolic transformations should either be performed inside the trusted core or emit evidence that the core can check.

Fuzzing belongs at the exploratory end of the spectrum. A successful fuzzing run that finds an input causing a failure produces a valuable witness. A large number of successful tests does not generally prove absence of failure. Coverage-guided fuzzing can still be integrated because outfinite mathematics explicitly distinguishes positive evidence, counterexample, coverage, and proof.

8.6 A common profile for analysis methods

The programme should characterize every analysis engine along a small set of semantic dimensions. The following table is deliberately two-dimensional: the left column names the dimension, while the right column states the question that a tool integration must answer.

Dimension	Question required by an outfinite integration
Semantics	What concrete or theoretical system does the method actually analyze?
Guarantee	What is logically justified when the method reports success or failure?
Coverage	Which inputs, paths, states, bounds, or abstractions are included?
Frontier	What remains unexplored, imprecise, unsupported, or resource-limited?
Trusted base	Which components must be correct for the guarantee to hold?
Certificate	Can the result be checked independently of the engine that produced it?

This profile is more informative than a label such as proof, testing, or optimization. It also offers a common interface through which heterogeneous tools can cooperate.

8.7 Formal methods should be classified by the claim they close, not by their marketing category

A mature outfinite proof architecture should resist labels such as "formal proof method," "testing method," or "optimization method" when those labels are used as if they determined logical status. The same family of algorithms can participate in very different justification patterns. What matters is the semantics of the analyzed object, the statement the engine is entitled to return, the coverage argument, and the certificate that a checker can validate.

Deductive verification offers the clearest proof-oriented case. A program is associated with preconditions, postconditions, invariants, frame conditions, and other contracts. Verification-condition generation reduces correctness to logical obligations. A theorem prover or SMT solver is then asked to discharge them. When the translation is correct and the obligations are proved in a sound logic, the result can quantify over all modeled inputs without enumerating them. Yet even here several layers of trust remain. The source language has a semantics, the verification-condition generator must preserve that semantics, the solver

operates in a logical theory, and the theorem being proved may omit aspects of the environment. The formal guarantee is strong precisely because its boundary can be stated.

Dependent types push some of the same reasoning into the type system. A value can carry not merely a representation but a proof that it satisfies an invariant. This is attractive for outfinite mathematics because existence conditions, generation witnesses, bounds, and bridge obligations can become part of type checking rather than external annotations. The cost is that dependent types can make elaboration and proof search difficult, especially when scientific models contain approximate values, evolving state, or evidence that is not deductive. The research question is not whether dependent type theory should be "used," but which outfinite distinctions can be encoded cleanly in it and which require a different layer.

Equivalence checking is particularly important for executable science. A theory is frequently transformed: symbolic equations become numerical code, source programs become optimized programs, circuits are synthesized from specifications, a high-level model is discretized, and an AI may rewrite a theory into a different representation. Every transformation creates the possibility of semantic drift. Equivalence or refinement checking can certify that the transformed artifact preserves a chosen behavior. The crucial phrase is chosen behavior. Exact state-by-state equality is often too strong; observational equivalence, trace equivalence, simulation, bisimulation, numerical closeness, or property preservation may be the relevant relation. The outfinite layer should require that relation to be explicit.

Proof-carrying code generalizes the same idea. Instead of trusting the producer, the consumer receives an artifact accompanied by evidence that it satisfies a policy. The evidence can be checked more cheaply than it was discovered. This producer-checker separation is an excellent model for AI-generated mathematics. A generative model may invent a proof strategy, an invariant, or an implementation, but the consuming system should only accept the result if a small checker validates the corresponding certificate. The epistemic role of the AI is then discovery, not authority.

8.8 Abstract interpretation, dataflow, information flow, and the logic of sound approximation

Abstract interpretation is central to an outfinite foundation because it demonstrates how finite representations can justify claims about families of concrete behaviors. The key idea is not merely that the analyzer uses intervals, signs, congruences, polyhedra, or other abstract values. The guarantee comes from a mathematically specified relation between concrete and abstract semantics. An abstract transfer function must conservatively represent the effect of the concrete operation. If this relation is sound, an invariant established in the abstract domain can rule out concrete behaviors that would violate the property.

This is a powerful alternative to explicit enumeration. Suppose a variable starts between zero and ten and a loop only increases it while maintaining an upper guard. An interval analysis can propagate a range rather than explore each concrete value. More sophisticated relational domains can track constraints among variables. The analyzer can reach an inductive post-fixpoint that contains every reachable concrete state represented by the abstraction. If the bad states are disjoint from this abstract invariant, safety follows. The proof does not require an infinite execution of the loop; it requires a finite certificate that the invariant contains the initial states, is closed under transition, and entails the desired property.

The classical theory of abstract interpretation often uses complete lattices and fixpoint results. An outfinite reconstruction must decide which of those structures are regarded as theoretical objects and which are required computationally. A practical analyzer does not materialize an infinite lattice. It manipulates finite abstract elements and uses iteration strategies such as widening to force convergence. This makes the technique an ideal test of the distinction between theory-level infinity and realized computation. The mathematical lattice may exist as a theoretical object, while the implemented analyzer follows a finite trace that terminates with a finite abstract state and an explicit precision loss.

Widening illustrates why a proof certificate must record more than success. A widening operator deliberately sacrifices precision to obtain termination. The resulting invariant can remain sound, but it may include unreachable states and generate false alarms. The analysis is therefore not "less formal" because it is approximate; it is a sound proof of a weaker abstraction. An outfinite system should preserve the abstraction relation and the precision frontier so that later symbolic reasoning can refine only the suspicious region.

Dataflow analysis belongs naturally inside this picture. Classical reaching-definitions, liveness, constant propagation, available-expressions, and similar analyses operate over control-flow graphs with ordered information states and transfer functions. They are often introduced as compiler optimizations, which can obscure their formal status. A dataflow analysis can be sound for its chosen property even when the property is modest. The fact that its industrial use is optimization does not turn it into a non-formal technique. The outfinite classification should distinguish the strength of the claim from the mathematical rigor of the method.

Taint and information-flow analyses require the same care. A dynamic taint tracker may provide only evidence about observed executions. A static taint analysis may over-approximate possible flows and prove that certain sources cannot reach sensitive sinks within its model. Security type systems and lattice-based information-flow models can establish stronger properties such as noninterference. Their weakness is not that they are called taint analysis; it is that unmodeled implicit flows, reflection, concurrency, side channels, or environment behavior can lie outside the guarantee. A typed evidence profile should state exactly which flows and semantics are covered.

The outfinite interpretation of abstract analysis is therefore broader than software verification. It provides a general pattern for scientific abstraction. A macro-model of a collective, a coarse-grained physical model, or a statistical summary can be treated as an abstract state if there is a justified relation to the underlying system. The crucial question is whether conclusions drawn at the abstract level are sound for the concrete phenomena they claim to represent. This connection may allow abstract interpretation to become one of the mathematical bridges between typed existence and executable science rather than remaining merely a program-analysis technique.

8.9 Symbolic execution, concolic search, bounded model checking, and the exact status of exploration

Symbolic execution replaces concrete input values with symbolic expressions and follows the operational semantics of a program. Each branch adds a path condition, so one symbolic state can represent many concrete executions. The method is often described as bug hunting because real programs generate too many paths, but this practical limitation should not be

confused with its logical nature. On a loop-free finite program, or on a state space for which all symbolic branches can be closed, symbolic execution can be exhaustive and can support a global proof. On a large open-ended program it is usually incomplete, yet each explored path remains formally meaningful.

The asymmetry between proof and refutation is especially important. A symbolic executor can stop after examining only a tiny fraction of paths and still produce a decisive counterexample if it finds a satisfiable path condition leading to an error. One finite witness is enough to refute a universal safety claim. The absence of a discovered error, by contrast, says little without coverage. This is why exploratory methods can be first-class participants in a proof calculus even when they cannot establish global absence of bugs.

Concolic execution strengthens practical exploration by combining concrete runs with symbolic tracking. The concrete execution handles environment details and difficult operations, while the symbolic conditions are systematically modified to generate new tests. The method is typically incomplete because the search space is enormous and environmental behavior may be only partially modeled. Its evidence is nonetheless structured. Each run is a reachability witness, each symbolic condition describes a family of nearby runs, and the campaign as a whole has a measurable coverage frontier. An outfinite system can preserve this frontier rather than converting a successful campaign into a false universal claim.

Fuzzing should be interpreted similarly. Random or coverage-guided input generation is not a proof strategy in the deductive sense, but a concrete crashing input is a formal refutation of a universal statement about the tested implementation. Property-based testing can additionally expose shrinking witnesses that make failures easier to analyze. AI-guided fuzzing may become a valuable source of adversarial examples for executable theories. The search heuristic can remain probabilistic because the validity of the witness is checked independently.

Bounded model checking provides an exact theory of bounded exploration. A transition system is unrolled to a finite depth, and the existence of a bad trace is reduced to SAT or SMT. If the resulting formula is satisfiable, the model gives a counterexample within the bound. If it is unsatisfiable and the reduction is correct, then no such trace exists within that horizon. This is a genuine theorem about all modeled traces up to the chosen depth, not a sample of the first few executions. The frontier is mathematical: the theorem says exactly what the bound permits it to say.

A bounded result becomes global only with an additional lifting argument. A known completeness threshold may show that all relevant states have been covered. K-induction can combine a finite base with an inductive step. An independently discovered invariant can close the transition system. Property-directed methods such as IC3 or PDR can construct inductive frames that exclude bad states. The outfinite proof calculus should represent these lifting arguments explicitly so that increasing a bound is never confused with proving an unbounded property.

CEGAR provides a particularly attractive model of progressive formal reasoning. An abstract system is checked first. If it satisfies the property under a sound abstraction, the concrete system is safe for the relevant claim. If the abstract checker returns a counterexample, the trace may be real or an artifact of over-approximation. A concretization phase attempts to realize it. A real counterexample refutes the property. A spurious one explains where the

abstraction lacks precision and guides refinement. The cycle may not terminate, but every iteration produces finite artifacts and a clearer residual obligation.

This architecture resembles scientific inquiry. A coarse theory makes a prediction, an apparent counterexample is investigated, and the model is either rejected or refined. The analogy should not be overstated, because scientific model revision involves empirical judgment, but CEGAR shows that incomplete formal reasoning need not be epistemically shapeless. A sequence of refinements can preserve provenance and expose why the current frontier exists. This is exactly the kind of structure that AI agents could exploit when planning verification work.

8.10 Transformation methods as proof steps when preservation is certified

Partial evaluation and symbolic interpretation are frequently categorized as optimization or simplification techniques rather than proof methods. This is correct if they are considered in isolation. A partial evaluator specializes a program with respect to known inputs and produces a residual program. A symbolic simplifier rewrites expressions according to algebraic rules. Neither operation, by itself, proves an arbitrary safety or mathematical theorem. Yet both can become certified proof steps if the transformation is accompanied by an equivalence or refinement argument.

This distinction matters because modern automated reasoning depends heavily on transformations. Theorem provers normalize expressions, compilers rewrite programs, solvers preprocess formulas, and AI systems may translate between natural language, symbolic forms, and executable code. If transformations are excluded from the proof architecture merely because their immediate purpose is optimization, the system loses the ability to reason about most of its own pipeline. Instead, the outfinite framework should treat a transformation as producing a new obligation: show that the transformed artifact preserves the relevant meaning of the original.

Normalization is the simplest example. Ring normalization can prove polynomial identities by reducing both sides to a canonical form. Constant folding can preserve program behavior under a specified arithmetic semantics. Partial evaluation can be justified by showing that the residual program is observationally equivalent to the original program under the fixed parameters. Compiler passes can be validated independently, as in translation validation, rather than trusted wholesale. Each case turns a transformation into evidence when the preservation relation is checked.

This principle is crucial for AI-generated formalizations. A language model may produce a mathematically plausible reformulation that is easier for a solver, but the central risk is semantic drift. The reformulation should not be accepted because it looks reasonable. The system should generate or demand a preservation certificate relating the source claim to the target representation. The relation may be exact equivalence, one-way refinement, approximation within tolerance, or preservation of a named property. The chosen relation becomes part of the theorem.

The same approach clarifies the role of symbolic interpretation. A domain-specific interpreter that evaluates expressions according to explicit rules can serve as a proof engine if its rules are sound and its trace is checkable. A purely heuristic simplifier remains only a producer of

candidate expressions. The category of the tool is secondary; the evidence contract determines how its result may be used.

This leads to a practical architecture for the research programme. Engines should not be integrated by declaring them trusted proof tools. They should be integrated through adapters that specify the statement they can close and the certificate they can emit. A theorem prover may emit a proof term. A SAT solver may emit a refutation certificate. A symbolic executor may emit a concrete trace and path condition. An abstract interpreter may emit an invariant and closure evidence. A fuzzer may emit a reproducible witness. A partial evaluator may emit a residual program plus an equivalence obligation. The kernel only needs to know how to validate the corresponding evidence types.

Such an architecture is technically ambitious, but it provides a clear falsification target. If heterogeneous engines cannot be reduced to a manageable family of evidence contracts, or if certificate checking becomes nearly as complicated as trusting each engine directly, the proposed unification loses its advantage. If the contracts can be kept small and compositional, the outfinite proof layer could become a useful substrate for automated mathematics, formal verification, and scientific AI.

8.11 Certificates, trusted bases, and proof interchange

The technical programme should ultimately be organized around certificate interchange rather than around a hierarchy of analysis tools. Each engine should be free to use the algorithm best suited to its task, but the rest of the system should not be forced to trust the engine's internal implementation. The engine earns a place in the proof architecture by exporting an artifact whose meaning can be checked by a smaller and more stable verifier.

This principle already appears in several mature areas. Proof assistants reduce complex tactic search to proof terms checked by a kernel. SAT solvers can emit resolution-style refutations. SMT systems increasingly support proof traces for important fragments. Model checkers can return counterexample traces or inductive invariants. Translation validation checks each produced artifact instead of proving an optimizer correct once and for all. Proof-carrying code moves verification evidence to the consumer. The outfinite programme should treat these as instances of a common architectural pattern.

The trusted base must itself be typed. A proof about a source-language program depends on the formal semantics of that language. A proof about compiled machine code depends on the compiler bridge if the source proof is transported downward. A numerical certificate depends on rounding semantics and library functions. A scientific conclusion may depend on data ingestion and calibration. Listing trusted components is not an admission that formal proof is weak; it is the exact statement of where the guarantee lives.

Certificate checking also provides a route to heterogeneous composition. Suppose an abstract interpreter produces an invariant that bounds a variable. A deductive verifier can consume the invariant as a lemma. A symbolic executor can focus on the residual states where the invariant is too weak. A bounded model checker can search a finite horizon for violations. If the search closes the residual region, the final theorem is supported by multiple certificates. No tool needs to understand the internal algorithm of the others; they exchange typed claims and witnesses.

The architecture should distinguish certificate generation from certificate validation. Generation can be computationally expensive, heuristic, parallel, and AI-assisted. Validation

should be deterministic whenever practical and should have a substantially smaller trusted surface. This asymmetry is particularly valuable in an AI context because it allows aggressive automated search without assigning epistemic authority to the generative agent.

Not every useful analysis will be able to emit a compact certificate. Some abstract domains depend on sophisticated numerical libraries; some theorem provers have large trusted kernels; some model transformations are difficult to replay independently. The programme should therefore define levels of trust rather than pretend that all engines can immediately fit a proof-carrying architecture. A result may be fully independently checkable, replayable only by the same verified engine, or dependent on a named external trusted component. These levels should affect how far the result can be transported.

The proof-interchange problem is also a standardization problem. A common representation must encode claims, assumptions, theories, frontiers, and evidence without embedding every solver-specific proof language in the core. One possible strategy is to allow evidence adapters that translate native certificates into a small family of generic proof obligations. Another is to keep native evidence but require each adapter to prove a soundness theorem once. The right tradeoff between a universal certificate language and certified adapters is an empirical systems question.

This part of the programme has a straightforward benchmark. The same safety property should be attacked with at least three engines, and their outputs should be composed without manual reinterpretation. A second implementation of the checker should independently validate the final artifact. The benchmark should then alter one assumption or representation and verify that only certificates depending on the change are invalidated. If evidence interchange requires bespoke human reasoning at every tool boundary, the proposed architecture has not solved the integration problem it was designed to address.

8.12 Research obligations and falsifiability

The classification should be tested by integrating real tools rather than only describing them. A small benchmark suite should contain properties that can be proven deductively, properties for which abstract interpretation is natural, bounded failures found by model checking, path-specific counterexamples found by symbolic execution, information-flow questions handled by taint analysis, and transformations requiring equivalence validation. Every result should be translated into the common evidence profile and independently checked where certificates are available.

The programme fails if the common representation erases method-specific information or forces every engine into an unnatural interface. It also fails if the representation is so generic that it provides no additional safety compared with ordinary log files. The standard of success is compositional usefulness: a result from one method should become a precise premise for another without overstating its guarantee.

A particularly revealing experiment is to deliberately stop symbolic execution or bounded model checking before the depth at which a defect occurs. The framework must preserve the analysis frontier and prevent the partial result from being interpreted as global safety. This is a simple, falsifiable demonstration of whether frontier typing has real value.

Part III. Reinterpreting mathematical structure

Chapter 9. Arithmetic, finite witnesses, and algorithmic forms of proof

9.1 Natural numbers as values and as an open generative domain

Arithmetic is the first serious test of the programme because a proposed foundation that cannot recover elementary number theory has little value. The challenge is to reconstruct familiar results without quietly returning to the assumption that the natural numbers must first be given as a completed physical or computational collection.

Outfinite arithmetic begins by distinguishing an individual natural value from the domain rule that generates natural values and from any theoretical totality introduced by a stronger theory. An individual natural number is finitely represented whenever it is actually used in a computation or proof artifact. A generative account of the natural domain specifies a starting value and a successor rule. The rule has no internal maximum, but each realized numeral and each realized derivation remains finite. A classical set-theoretic theory may additionally introduce the completed set of all natural numbers as a theoretical object. These descriptions can coexist if their existence commitments remain explicit.

This separation does not prevent general arithmetic. A theorem can quantify parametrically over any natural value satisfying the domain contract. The proof need not enumerate a completed collection. It reasons about an arbitrary finite witness and the structural laws of the domain. In this respect, outfinite arithmetic is closer to constructive and type-theoretic traditions than to naive finitism.

The point is not to pretend that the idea is historically new. Recursive definitions, primitive recursion, induction, type-theoretic naturals, and proof assistants already provide rigorous accounts of finite representations and general reasoning. The research question is whether placing this generative interpretation inside a broader ontology of existence and frontiers produces new value when arithmetic is connected to implementations, measurements, and open processes.

9.2 Arithmetic operations as theory-dependent structures

The familiar symbols for addition and multiplication hide multiple semantics in real systems. Addition over mathematical integers is exact and unbounded in the ideal theory. Addition over fixed-width machine integers may wrap modulo a power of two, trap on overflow, saturate, or invoke undefined behavior depending on the language and machine model. Floating-point addition is rounded and generally not associative. Interval addition returns an enclosure. Modular addition belongs to a finite algebraic structure with different order properties.

A foundation intended to support executable science should not allow these operations to share a symbol while silently changing laws. The operation must belong to an algebra or representation whose semantics is part of the theorem context. A claim such as associativity is then a theorem of the appropriate structure rather than an unqualified law of the visual plus sign.

This simple discipline has practical importance for AI. Language models frequently manipulate mathematical expressions by pattern, and the same notation is reused across

exact and approximate arithmetic. Explicit structure can prevent a symbolic transformation valid over a ring from being applied to floating-point code where the corresponding identity does not hold exactly. The problem is well known in formal verification; the outfinite contribution would be to make the distinction part of a general bridge between ideal mathematics and computational realization.

9.3 Invariants as executable arithmetic proofs

Many elementary theorems can be reformulated as invariant proofs. Consider the formula for the sum of the first n positive integers. A conventional proof may use induction. An algorithmic version maintains a running index and a running sum. The central invariant states that twice the current sum equals the current index multiplied by its successor. Initialization is checked at zero. A symbolic transition shows that if the invariant holds before adding the next integer, it holds afterward. A decreasing measure establishes termination for every finite input n . When the loop exits, the invariant yields the familiar closed form.

Nothing in this reformulation requires enumerating all natural numbers. Nor does it eliminate induction in a deep logical sense. The invariant proof is an operational form of inductive reasoning. Its advantage for the research programme is that the process, the preservation condition, and the termination argument are visible and machine-checkable. It also connects directly to software verification, where the same reasoning can certify an implementation rather than only an abstract equation.

Euclid's proof concerning prime numbers offers an even more characteristic outfinite formulation. Given any finite collection of known primes, form a number related to their product and show that at least one prime divisor lies outside the supplied collection. The theorem states an extension property: no finite presented collection exhausts the prime numbers. A classical theory can summarize this extension property by saying that there are infinitely many primes. The outfinite version preserves the constructive content and records the fact that every witness produced is finite.

This is not offered as a replacement for the classical theorem in ordinary mathematics. It is a model of how a theorem can express unbounded generativity without postulating the physical realization of a completed infinite collection.

9.4 Divisibility, Euclid's algorithm, and finite descent

The greatest common divisor is a useful example because both the mathematical property and the computation admit compact certificates. Euclid's algorithm repeatedly replaces a pair of nonnegative integers by a smaller remainder relation. The key invariant is that the set, or more cautiously the predicate, of common divisors is preserved. A natural-valued measure decreases at every nonterminal step, so the process terminates for any finite input.

This pattern generalizes. Finite descent proves termination by identifying a well-founded measure that must decrease. In an outfinite framework, such a measure is an explicit argument that a particular process is not outfinite in the relevant dimension: although the ambient domain has no logical maximum, the computation itself is forced toward a terminal region.

The distinction between open domain and terminating process is important. The natural numbers can be generated without an internal upper bound, yet an algorithm operating on a particular pair of naturals can have a finite proof of termination. This prevents the concept of

outfiniteness from becoming a vague synonym for any computation involving natural numbers.

9.5 What arithmetic contributes to the foundation

Arithmetic reveals several general principles. Values and domains should not be conflated. A generator can define an open domain without materializing a totality. Operations belong to declared structures. Generality can be parametric. Inductive reasoning can be compiled into finite proof obligations. Termination and open-endedness are properties of processes under measures, not of notation alone.

These principles also clarify where the programme must remain compatible with classical mathematics. Many number-theoretic results use nonconstructive methods, completed infinite objects, or higher-order principles in productive ways. Outfinite mathematics should permit such results inside a theoretical layer rather than insist that every theorem be converted into an algorithm. The research task is to preserve the difference between a theorem's theoretical existence and the finite artifacts by which it is represented or applied.

9.6 Arithmetic as the first reconstruction test

Arithmetic is the natural place to test whether outfinite foundations add substance rather than vocabulary. The domain is simple enough that classical results are well understood, yet rich enough to expose the difference between a rule, a value, a representation, and a completed totality. A credible research programme should reconstruct a substantial fragment of elementary arithmetic without assuming that the physical realization of the natural numbers is an actually infinite collection.

The basic distinction is between an individual natural value and the generative principle that produces successors. Any value manipulated in a proof or computation is finitely represented. The successor rule can be described finitely and need not contain a largest element. Generality can then arise parametrically: a theorem is checked for an arbitrary finite natural witness satisfying the generation contract, rather than by imagining that the proof iterates through a completed set of all naturals.

This perspective does not eliminate induction by decree. Invariants and well-founded descent often encode the same logical structure in an executable form. To prove the familiar formula for the sum of the first n naturals, one can define the summation process and identify an invariant relating the loop index to the accumulated sum. The proof checks the initial state, preservation by one transition, and the exit condition. The universal theorem follows because every admitted finite execution preserves the invariant. The outfinite contribution is to make the process and its witness structure explicit rather than to deny the legitimacy of inductive reasoning.

Euclid's theorem on primes offers an even more revealing formulation. The ancient statement can be read constructively: given any finite collection of primes, one can produce a prime not contained in it by examining a divisor of one plus their product. The theorem does not require a physically completed collection of all primes. It proves the extendability of every finite collection by a finite construction. This is almost a canonical outfinite theorem because it distinguishes an open generative property from an actually realized infinite object.

Euclid's algorithm for the greatest common divisor provides a complementary case. The execution terminates because a nonnegative measure decreases under each transition. A

finite descent certificate proves termination for arbitrary finite inputs. The proof is both mathematical and computational: it establishes preservation of common divisors and the decrease of the remainder. Such examples show that the proposed foundation can preserve classical rigor while using language naturally aligned with programs and symbolic execution.

The harder arithmetic problem concerns representation. The expression "addition" can denote addition on mathematical integers, modular addition on fixed-width words, saturating arithmetic, floating-point addition, or an interval operation. These structures satisfy different laws. Associativity holds exactly for integers and modular arithmetic but can fail for floating point because of rounding. A range theorem can sometimes bridge the fixed-width implementation to ordinary integer arithmetic by proving that overflow cannot occur. This is a concrete example of the programme's central thesis: the ideal operation, representation, and implementation should not be silently identified.

The arithmetic reconstruction should therefore produce two kinds of result. The first is conservative: familiar theorems should be recoverable with proof complexity comparable to established systems. The second is discriminating: the outfinite formulation should expose implementation-sensitive failures that an idealized theorem leaves outside its scope. If arithmetic becomes dramatically more cumbersome without yielding clearer bridges to computation, the foundation has failed an early and important test.

9.7 Research obligations and falsifiability

Arithmetic provides a straightforward benchmark for conservativity. A library of elementary and intermediate number theory should be formalized in the proposed core and translated to an established proof assistant. The outfinite representation should recover ordinary results without changing their mathematical content. If routine proofs become dramatically more difficult with no corresponding gain in semantic information, the design should be simplified.

A second benchmark should compare ideal arithmetic with machine arithmetic. The same high-level expressions should be interpreted over mathematical integers, fixed-width integers, modular arithmetic, floating point, and interval arithmetic. The system should correctly identify which algebraic laws survive each bridge. Deliberately applying an invalid law, such as exact associativity of floating-point addition, should be rejected or qualified automatically.

A third test concerns AI. Agents should be asked to derive and implement arithmetic theorems under mixed numeric semantics. If explicit algebra and representation typing reduces invalid rewrites relative to a baseline, that supports the claim that the foundation can improve practical AI rigor.

Chapter 10. Algebraic structures, lattices, and the algebra of information

10.1 From carrier sets to algebraic protocols

Algebra is unusually compatible with the outfinite project because it already emphasizes structure over the internal nature of elements. A group, ring, field, monoid, semiring, lattice, or module is characterized by operations and laws. The conventional definition typically begins with a carrier set. Outfinite mathematics asks whether the carrier must always be interpreted as a completed set, or whether a more general notion of domain can support the same structural reasoning.

The proposed answer is that an algebraic structure should be understood as a protocol consisting of a domain specification, operations, laws, representations, and bridge conditions. The domain may be a finite collection, a generative domain, a recognized class, a theoretical whole, or a family of process states. The algebraic laws remain precise, but their status depends on the semantic layer in which the operations are interpreted.

This reinterpretation is conservative when the carrier is an ordinary set. It becomes informative when the same algebra has several realizations. Mathematical integers form a ring with exact operations. Fixed-width machine integers form finite rings under wraparound semantics, but not under trapping semantics where addition can become partial. Floating-point values do not form a field in the ordinary algebraic sense because rounding and exceptional values break familiar laws. Interval arithmetic supports inclusion properties that matter more than exact equality. A programmable foundation should let these structures share high-level concepts without conflating their laws.

Calling such a structure an algebraic protocol is useful because it emphasizes that operations come with obligations. Closure may hold in an ideal theory but fail in an implementation because of overflow. An inverse may exist theoretically but be unavailable within a numerical tolerance or resource frontier. The framework can therefore distinguish algebraic totality from operational totality.

10.2 Monoids, groups, rings, fields, and semirings under outfinite interpretation

Monoids are a natural starting point because they require an associative binary operation and an identity. Many computational processes form monoids: strings under concatenation, traces under composition, transformations under sequencing, and costs under accumulation. In an outfinite setting, the identity and associativity laws can be proven parametrically over any realized values in the domain, while the domain itself need not be physically completed.

Groups add inverses. Here the distinction between theoretical and operational existence becomes more visible. A group theorem may establish that every theoretical element has an inverse. A concrete numerical implementation may represent only a subset of elements or may not be able to compute the inverse within a resource bound. The inverse exists in the theory, while the operation reaches a computational frontier. These are not contradictory statements.

Rings and fields are central because much of classical mathematics assumes their laws silently. Outfinite mathematics should make the relevant structure explicit whenever a theorem is transported to computation. A symbolic algebra engine can then justify transformations from the declared ring or field laws. A numerical backend must prove or approximate the corresponding bridge. This could reduce a common class of engineering errors in which identities derived over exact reals are implemented using arithmetic that does not preserve them exactly.

Semirings deserve special attention because they unify diverse computational structures. Shortest-path algorithms, provenance polynomials, weighted automata, dynamic programming, probability-like calculations, and program analyses often instantiate semiring patterns. An outfinite foundation could use semiring abstractions to make algorithmic theories portable across domains while preserving the meaning of the operations. The scientific value would not come from inventing semirings but from integrating them with existence types, proof certificates, and resource-aware realizations.

10.3 Lattices and abstract interpretation

Lattices are particularly important because they organize information and approximation. In static analysis, abstract values are often ordered by precision or information content. Joins combine information from control-flow paths. Fixpoints represent stable solutions to recursive equations. Complete lattices provide elegant existence theorems for fixpoints and form part of the classical theory of abstract interpretation.

This creates a productive tension for outfinitism. Complete lattices can be infinite theoretical objects. The programme should not simply ban them, because doing so would discard powerful mathematics. Instead, it should distinguish the ideal lattice in which semantic results are stated from the finite representation used by an analyzer. A finite abstract domain is straightforward. An infinite symbolic domain can still be represented by finite expressions and manipulated by finite rules. A widening operator can force stabilization of a computation even when the ideal ascending chain could continue indefinitely.

An outfinite reading of a fixpoint theorem therefore separates three questions. The ideal theory may prove that a least fixpoint exists. The analyzer may compute a finite post-fixpoint that soundly contains all reachable states. The actual computation may stop because a widening reached stability, because a convergence criterion was met, or because a frontier was encountered. These are different kinds of results.

This separation is not merely philosophical. It clarifies a frequent misconception about static analysis. A sound abstract interpreter does not need to enumerate an infinite set of concrete states. It manipulates finite representations that denote or over-approximate families of states. The finite representation, the abstraction relation, and the transfer functions constitute the operational evidence.

10.4 Bilattices, four-valued logics, and information order

Bilattices and four-valued logics are attractive for a system that must reason with incomplete and contradictory evidence. One ordering can represent truth-like status while another represents information content. A proposition can be supported, refuted, both supported and refuted by conflicting sources, or neither established nor refuted.

This structure is useful for executable science, where contradictory measurements or models should not cause logical explosion. It is also useful for AI review. An agent can encounter two sources that make incompatible claims. Rather than choosing one prematurely, the formal system can preserve the conflict and attach provenance to both sides. Deductive modules that require classical consistency can then operate only after an explicit reconciliation step.

The important caution is that a four-valued status is not the same as probability, fuzzy degree, or confidence. These notions answer different questions. A statement may be highly probable but unsupported by direct evidence; it may have graded membership without uncertainty; it may be both supported and contradicted by sources. A modular foundation should resist the temptation to compress all uncertainty into one numerical scale.

10.5 Algebra as an implementation discipline

Algebraic structures can also organize the implementation of outfinite mathematics. Proof evidence can have composition operations. Theory translations can form compositional structures. Costs can be accumulated in semirings. Information states can be joined in lattices. Process traces can form categories or monoids. The value of algebra is that these patterns come with laws that can be tested and proved.

An implementation should therefore expose algebraic contracts rather than hard-code assumptions. An analyzer that relies on a join operation should require proof or a trusted declaration of the relevant lattice laws. A transformation pipeline that relies on associativity should identify the algebra in which associativity holds. AI agents could search for suitable algebraic structures, but a checker would verify that the required laws are satisfied.

10.6 Structure, representation, and implementation as separate algebraic layers

The reinterpretation of algebra should not attempt to replace groups, rings, fields, modules, semirings, or lattices. These are among the most successful abstractions in mathematics. The outfinite contribution is to distinguish the abstract laws from the form in which a structure is represented and from the finite mechanism that realizes its operations.

An algebraic structure can be understood as a protocol consisting of a class of admissible values, operations, and laws. The carrier need not be imagined first as a completed set. For finite carriers, enumeration is possible. For generative domains, values can be admitted by witnesses. For theory-local completed objects, classical set-based semantics can be imported. The same algebraic laws can then be studied across these different modes of existence.

This perspective is especially useful for semirings and monoids, which already appear throughout computer science. Path problems, parsing, dynamic programming, provenance, weighted automata, probability, and cost accumulation can often be formulated by choosing suitable algebraic operations. In an outfinite environment, the algebra can also describe how evidence combines. One operation may represent alternative justifications, another sequential composition. Different proof engines can then participate in a common algebra of results as long as the combination laws are stated explicitly.

Rings and fields reveal the importance of representation. The rational numbers can be represented exactly by finite numerator-denominator pairs, although computations may grow costly. Real-number fields are ideal theoretical structures whose exact arbitrary

elements are not finitely materialized in ordinary computation. Floating-point formats approximate field operations but fail some algebraic identities. Interval arithmetic replaces exact values with enclosures and changes the interpretation of equality. Treating all of these as "numbers" without a typed algebraic layer invites invalid theorem transport.

The correct bridge is not to declare approximate arithmetic defective but to specify a refinement relation. A floating-point computation may be shown to approximate a real-valued expression within a certified error. A fixed-width integer computation may refine ordinary integer arithmetic on a proven range. A symbolic algebra system may preserve exact ring identities while a numerical backend implements them approximately. The structure-preservation theorem becomes part of the application.

Lattices occupy a particularly important place because they organize partial information. Abstract interpretation uses ordered domains in which one element can represent less or more precise information than another. Security policies use lattices to describe information-flow levels. Four-valued and bilattice logics can separate degrees of support from degrees of truth or knowledge. These structures fit naturally with an outfinite epistemology in which evidence may accumulate, contradict, or remain incomplete.

The programme should investigate whether an algebra of frontiers and evidence can be defined using such ordered structures. A frontier can be refined when more paths are explored, more precision is available, or more observations are collected. Evidence states can be ordered by informativeness without pretending that they are probabilities. If these orderings compose well, algebraic structure may provide a mathematically clean implementation layer for open proof states and scientific knowledge.

The falsification criterion is again comparative. Existing algebraic and type-theoretic frameworks may already express all necessary distinctions. The outfinite proposal earns its place only if separating abstract structure, representation, and realization enables more reliable theorem transport, better automation, or clearer interoperability among formal engines. Algebra should be one of the places where the programme demonstrates continuity with mainstream mathematics rather than rhetorical rupture.

10.7 Research obligations and falsifiability

The outfinite reinterpretation of algebra is credible only if it does more than replace the word set with domain. The programme should implement the same algebraic theorem across several carriers: finite enumerations, symbolic generative domains, exact theoretical structures, and resource-limited implementations. It should demonstrate that the structural theorem is reused while the bridge obligations differ.

Lattices provide another test. A classical abstract interpretation using an infinite lattice should be represented together with a finite analyzer. The framework must clearly separate the theorem that justifies the abstraction from the operational strategy that computes a post-fixpoint. If this distinction cannot be represented without duplicating the entire theory, the proposed layering is too weak.

Finally, the programme should investigate whether algebraic protocols improve AI-generated code and proofs. Agents can be asked to optimize or transform expressions under several numeric algebras. The system should prevent transformations that rely on laws absent from the selected structure. A measurable reduction in invalid transformations would support the practical value of the approach.

Chapter 11. Category theory as a structural grammar for an outfinite foundation

11.1 Why category theory is a natural ally

Category theory is one of the most promising external frameworks for the outfinite programme because it begins not with membership but with objects, morphisms, composition, and structural preservation. This orientation is well matched to a research programme concerned with theory bridges, multiple views, process refinement, and transformations between representations.

A set-theoretic perspective asks what an object contains. A categorical perspective often asks how the object relates to other objects and which transformations preserve the structure relevant to the problem. Outfinitism similarly resists treating internal membership as the universal foundation of existence. A collective is defined by organization and persistence, a process by transitions, a theory bridge by preserved meaning, and an implementation by its relation to a specification. These are inherently relational questions.

Category theory also provides mature languages for abstraction, beginning with the structural framework of Eilenberg and Mac Lane (1945) and later foundational developments such as Lawvere's work. Functors translate between categories while preserving composition. Natural transformations compare functors coherently. Adjunctions capture pairs of transformations that are optimal in a precise structural sense. Monoidal categories express composition of systems. Coalgebras provide powerful accounts of state-based and potentially ongoing behavior. Categorical logic connects logical systems with structural semantics.

This makes category theory attractive as a grammar for a family of outfinite theories. A theory could be represented as a category or a related structured object, while bridges become functors or more general morphisms. The same scientific system could have a detailed category and an abstract category connected by a functor that forgets irrelevant information. A numerical realization could be connected to an ideal algebraic theory through a semantics-preserving translation. A process and its observational view could be related coalgebraically.

The programme should nevertheless avoid the fashionable mistake of assuming that category theory automatically solves foundational problems. Standard category theory is fully compatible with actual infinity and often quantifies over large collections of objects and morphisms. Universal properties are usually stated without operational resource bounds. A category can therefore be as infinitary as a set-theoretic construction. Category theory fits the structural goals of outfinitism, but it must itself be interpreted through typed existence and finite representation.

11.2 What an outfinite category might mean

An outfinite category should not be defined simply as a finite category. Many useful categories are not finite, and forcing finiteness would destroy much of the theory. A more promising notion is representational: every realized categorical construction is finite, while the theory may describe open generative families of objects and morphisms or theoretical wholes under explicit existence modes.

This distinction permits several kinds of categories. A finite computational category may enumerate all objects and arrows. A generative category may provide constructors that produce new objects and morphisms as needed. A symbolic category may represent families of morphisms through finite expressions. A theoretical category may contain ideal infinite structures as theory-local objects. The categorical laws of identity and associativity are then interpreted in the relevant domain.

Universal properties require special care. To say that an object has a universal property is normally to quantify over every relevant object and morphism in the category. An outfinite interpretation can preserve such theorems inside a theoretical category. For executable use, however, one may also represent a universal construction by a parametric constructor and a finite proof that any supplied admissible instance factors in the required way. This is analogous to parametric generality in arithmetic: universal reasoning does not require enumerating the universal domain.

The important research question is whether this distinction leads to a genuinely useful concept of an outfinite category or merely restates constructive category theory. The answer should be determined by examples involving resource frontiers and theory bridges, not by terminological preference.

11.3 Functors as theory translations

The most immediate application of category theory is translation between theories. Suppose one category represents ideal arithmetic and another represents a concrete implementation. A functor-like bridge should map values and operations while preserving the composition relevant to the specification. Where exact preservation is impossible, the bridge may land in an enriched structure that records error or approximation.

Similarly, a detailed physical model can be mapped to an abstract model by a functor that forgets microscopic information. If two sequences of transformations in the detailed model become the same observable behavior after abstraction, the functor expresses precisely which distinctions the higher-level theory ignores.

This is closely related to the meta-rational programme. Different theories need not be merged. They can remain separate semantic worlds connected by maps whose preservation properties are explicit. A failed bridge is then a local scientific result rather than a contradiction in a global ontology.

Natural transformations may represent coherent changes between two translation strategies. For example, two numerical discretizations may implement the same ideal transformation with different error behavior. Rather than demanding literal equality, a natural transformation or a related structure can express a systematic comparison across all objects in the relevant family.

11.4 Adjunctions, abstraction, and approximation

Adjunctions are particularly suggestive for the relationship between concrete and abstract semantics. Abstract interpretation is classically connected to Galois connections, which are order-theoretic structures closely related to adjunctions between posets viewed as categories. This suggests that the movement between detailed states and abstract properties can be integrated into a categorical account of theory bridges.

An outfinite interpretation would emphasize that the abstract and concrete sides may have different existence modes and representation costs. The concrete theory may itself be symbolic rather than physically enumerated. The abstraction function may be computable only for finite representations. The concretization relation may denote a theoretical family that is never materialized. The adjunction remains an ideal structural theorem, while the analyzer realizes finite portions of it.

The programme has previously considered ideas such as frontier-indexed adjunctions. Such terminology should remain explicitly provisional until a formal definition demonstrates value. The intuitive objective is clear: abstraction quality and available transformations may depend on a frontier such as precision, memory, time horizon, or observation scale. A family of adjunction-like relations indexed by such frontiers could organize refinement. Whether this construction is mathematically natural is an open research problem.

11.5 Coalgebras and the primacy of process

Coalgebra is perhaps even more closely aligned with outfinitism than ordinary algebra because it emphasizes observable behavior and state evolution (Rutten 2000). Algebra often describes how complex objects are constructed from simpler ones. Coalgebra often describes how a state can be observed and how it evolves. Automata, transition systems, streams, and many reactive systems admit coalgebraic descriptions.

This suits the outfinite view of processes. A process need not be identified with a completed set of all future states. It can be specified by a finite observation-and-transition rule. Behavioral equivalence can be expressed through bisimulation or related notions. Two implementations may have different internal states yet be equivalent under an observational view.

A service with retry behavior provides a simple example. A high-level specification may say that a request eventually returns a valid result or an explicit unavailability frontier. The implementation may pass through several invisible retry and backoff states. Step-by-step state equality would declare the systems different. A weak behavioral equivalence can ignore internal transitions and compare only externally visible outcomes. This is precisely the kind of identity relation needed in a process-centered foundation.

Coalgebra also exposes the distinction between safety and liveness. A safety theorem may establish that no invalid response is produced, even if the process can continue retrying. A liveness theorem that some response eventually occurs requires additional assumptions about fairness, timeouts, or the environment. Outfinitism should never infer liveness from the mere absence of a logical stopping condition.

11.6 Monoidal composition and scientific systems

Many scientific and engineering systems are built by composing subsystems. Monoidal categories and related compositional frameworks offer a disciplined way to connect local models. If the outfinite programme is to support executable science, compositionality is essential: a theory of a sensor, a controller, a physical plant, and a communication channel should combine without requiring a monolithic global model.

The frontier concept adds a distinctive question. When two components are composed, how should their validity regions, resource budgets, uncertainty models, and existence bridges combine? Sometimes the relevant frontier is the minimum of two ranges. Sometimes errors

accumulate. Sometimes one component changes the representation needed by another. A categorical framework enriched with annotations or effects may help make these combinations systematic.

Again, this is a research direction rather than a result already obtained. The programme should be wary of treating enriched categories as a universal answer. The value must be demonstrated through concise formal models in which compositional proofs become simpler or more reusable than they would be in a less structured framework.

11.7 Indexed categories, bridges, and the geometry of theory translation

Category theory may provide the most natural high-level language for organizing a pluralistic outfinite foundation because it focuses on structure-preserving relationships rather than on the internal construction of every object. This is closely aligned with the meta-rational ambition of maintaining several legitimate theories of the same reality while making their translation conditions explicit.

A theory can be treated as defining a category of objects and admissible morphisms, or more generally as a structured semantic environment. A bridge between theories can then be modeled by a functor when it preserves the relevant composition and identity structure. This gives a precise meaning to the idea that one mathematical description is translated into another without arbitrary reinterpretation. The functor need not be invertible. Many scientific bridges are lossy: a detailed physical model may map to a coarse statistical model, while the reverse direction reconstructs only a family of compatible detailed states.

Natural transformations are relevant when two translations are themselves compared. Scientific workflows often contain competing discretizations, approximations, or coordinate systems. Instead of asking whether two translations are syntactically identical, one can ask whether there is a coherent transformation relating their outputs across all objects of interest. This may provide a principled way to represent alternative formalizations produced by AI and to test whether they preserve the same semantics.

Adjunctions are especially suggestive because abstraction and concretization often come in asymmetric pairs. Abstract interpretation already contains a closely related order-theoretic pattern. A coarse theory forgets details to obtain tractable reasoning; a concretization maps abstract information back to the family of concrete states it represents. An adjunction can formalize the best approximation compatible with the chosen abstraction. In an outfinite framework, this structure may help explain why a result proved at one level transports soundly to another even when information is lost.

Coalgebra strengthens the process orientation of the programme. Algebra is naturally associated with ways of constructing values from operations, while coalgebra often describes how systems expose observations and transition to future states. Automata, state machines, streams, and reactive systems fit this pattern. Outfinite processes are fundamentally about finite observations of potentially open behavior, making coalgebra a promising semantic basis for process identity and behavioral equivalence.

Bisimulation is particularly relevant to the problem of identity over time. Two systems may have different internal states but remain behaviorally indistinguishable under a chosen observation interface. This offers a rigorous alternative to naive equality for processes and implementations. A protocol can be equivalent to a simplified model because each visible

action can be matched, even though their internal step structure differs. A collective can perhaps be studied through a related notion when member-level changes preserve macro-observables.

The outfinite twist is to index these structures by frontiers or observation regimes. Two processes may be bisimilar within a finite observational horizon and distinguishable later. A functor may preserve structure only for a validated parameter region. An abstraction may be sound under one resolution and invalid under another. Indexed or fibred categorical constructions could offer a language for these dependencies, although the research programme should not commit to sophisticated category theory unless it produces simpler and more powerful results than direct type-theoretic encodings.

Category theory also offers a disciplined answer to the fear that the programme will become a collection of unrelated logics. Instead of forcing every domain into one logic, the framework can seek compositional translations among local formal systems. Classical logic, probabilistic models, temporal logics, numerical abstractions, and empirical evidence structures may remain distinct while sharing categorical interfaces. Meta-rationality then acquires mathematical content: plural theories coexist, but their relationships are not informal.

The decisive test is constructive. The programme should formalize several nontrivial bridges, such as exact real analysis to interval computation, abstract program semantics to concrete execution, or a process model to a finite-state abstraction, and compare categorical formulations with simpler alternatives. If category theory merely redescribes translations after the work has already been done elsewhere, it should remain explanatory background. If it yields reusable composition theorems, automatic checking of bridge compatibility, or principled abstraction mechanisms, it may become a central layer of the foundation.

11.8 Research obligations and falsifiability

Category theory should face a particularly strict test because it can easily produce impressive vocabulary without operational benefit. The first benchmark should formalize several theory bridges both directly and categorically. The categorical version should make preservation laws, composition, or proof reuse materially clearer. If the result is longer and no more informative, category theory should remain a secondary semantic tool rather than part of the core.

A second benchmark should concern abstraction. A known abstract interpretation can be expressed order-theoretically and categorically. The outfinite extension should show whether frontier-indexed refinement adds a theorem, algorithm, or compositional law absent from the standard account. If it does not, the new terminology should be abandoned.

A third benchmark should use process equivalence. Specification and implementation models with hidden internal steps should be related through a coalgebraic or categorical semantics and checked against real finite-state examples. The formalism must distinguish safety from liveness and preserve environmental assumptions. Successful reuse across software protocols, agent systems, and collective processes would provide stronger evidence that category theory is genuinely structural for the programme.

Chapter 12. Non-classical logics, algebraic logic, and controlled pluralism

12.1 Why classical logic remains necessary but not sufficient

Classical logic is indispensable and should remain available inside outfinite theories. The programme does not seek novelty by rejecting excluded middle or ordinary implication wherever they are useful. The problem is that a single classical truth relation does not naturally encode every distinction required by processes, constructive existence, uncertain evidence, graded phenomena, resource consumption, and contradictory information.

The appropriate response is modularity. A theory should declare the logic used for a particular class of judgments. A deductive arithmetic module may be classical. A constructive existence claim may use intuitionistic rules. A process specification may use temporal or dynamic logic. An evidence repository may use a paraconsistent logic. A fuzzy classifier may assign degrees of membership. A resource-sensitive proof may use ideas from linear logic. The meta-layer must ensure that conclusions are not moved between these logics without explicit translations.

This is another expression of meta-rationality. Rationality does not require one logic to simulate every possible mode of reasoning if several precise logics can coexist and their bridges are controlled. What it forbids is opportunistic switching in which a result proven under one semantics is treated as if it had been proven under another.

12.2 Intuitionistic logic and constructive existence

Intuitionistic logic is a natural partner for existence typing because an existential claim is closely associated with a construction or witness. In a type-theoretic interpretation, a proof of existence contains an object and evidence that the object satisfies the property. This fits the outfinite preference for finite witnesses.

The programme should nevertheless avoid equating all existence with constructive existence. Theoretical modules may legitimately introduce ideal objects nonconstructively. Empirical existence is not the same as constructive mathematical existence. Collective existence may be established through a recognizer and evidence rather than by constructing the collective from first principles. The value of intuitionistic logic lies in providing one precise mode of justification, not a universal ontology.

Constructive reasoning can also clarify computation. If a proof produces a witness, an implementation may extract a program. Yet the computational realization still has frontiers. A constructive existence theorem does not guarantee that the extracted program is efficient enough for a particular machine. Existence type and resource realization must remain separate.

12.3 Modal logic and modes of existence

Modal logic offers a natural language for distinguishing necessity, possibility, knowledge, obligation, or accessibility across worlds or contexts. An outfinite foundation may use modal structure to represent transitions between existence modes or theory contexts. For example,

what is necessary inside a theoretical model need not be physically necessary. What is computationally reachable under one resource profile may become unreachable under another.

The danger is overloading modal operators. It would be easy to label every distinction with a new modality and produce an unreadable logic. The programme should instead identify a small number of semantically justified accessibility relations. A theory bridge can often be modeled more transparently as a typed map than as a modal operator. Modal logic should be used where reasoning about accessibility itself is central.

Epistemic modalities may be relevant for machine review. A system may distinguish what follows deductively from what is known from a source, what is merely reported, and what remains possible under unresolved evidence. Yet epistemic logic cannot replace provenance. Knowing that a claim is attributed to a source is different from knowing why that source should be trusted.

12.4 Temporal and dynamic logics for processes

Processes require a logic of change. Temporal logic allows statements about what always holds, what may eventually hold, what holds until another condition occurs, and how properties relate across a trace (Pnueli 1977). Dynamic logic reasons about programs or actions and their effects.

These logics fit the outfinite focus on process identity. A process can satisfy a safety property over every realized and admissible transition even while its state changes. A liveness property can state that a desired condition eventually occurs, but such a theorem must expose the assumptions required for eventuality. If the environment can block progress forever, an unconditional liveness theorem is false even if the transition rule itself has no terminal state.

Temporal reasoning also gives a precise language for frontiers. A bounded temporal theorem applies over a finite horizon. A monitor may establish that a property has held so far without claiming that it will hold forever. A model checker may establish an unbounded temporal property for a finite-state abstraction if the abstraction and completeness argument are sound. The evidence type should reveal which case applies.

12.5 Fuzzy logic and graded membership

Fuzzy logic is useful when a concept is genuinely graded by definition (Zadeh 1965). Collective membership, organizational cohesion, similarity between phenomena, and qualitative physical regimes can sometimes be represented more faithfully by degrees than by sharp thresholds.

The key outfinite requirement is semantic hygiene. A fuzzy degree is not a probability that a statement is true. It is not automatically a confidence level. It is not the percentage of evidence supporting the statement. It expresses degree of membership or truth according to a specified many-valued semantics. If an institution has a membership degree of 0.7 under a particular collective model, this does not mean there is a 70 percent probability that it exists.

This distinction is particularly important for AI because natural-language systems routinely blur confidence, likelihood, similarity, and truth. A typed logical framework can prevent these values from being compared or combined unless a bridge defines how.

12.6 Paraconsistency and four-valued evidence

Scientific and institutional data can be contradictory. Two laboratories may report incompatible measurements. Two databases may contain conflicting records. A model may predict one outcome while an observation suggests another. Classical logic can handle contradiction by revising assumptions, but a raw repository that immediately applies explosion would become useless.

Paraconsistent and four-valued systems allow contradictory information to be represented without deriving every statement (Belnap 1977). Four-valued systems distinguish support, refutation, both, and neither. This is well suited to an evidence layer in which provenance is preserved. Contradiction becomes a localized object requiring reconciliation rather than a catastrophic property of the entire knowledge base.

The programme should be clear that paraconsistency is not a license to maintain inconsistent mathematical proofs. A deductive module can still require consistency. Paraconsistent reasoning is valuable at boundaries where multiple sources or theories meet and conflicts are expected.

12.7 Linear logic and resource sensitivity

Linear logic is relevant because it treats assumptions as resources that cannot necessarily be copied or discarded freely (Girard 1987). This is conceptually attractive for outfinitism, which wants resource and process limits to be visible. Yet the relation should not be overstated. Physical memory, energy, and time are not automatically linear-logic propositions.

The research question is whether resource-sensitive proof systems can provide a clean semantics for consumable capabilities, tokens, permissions, or budgets in executable theories. In security protocols and distributed systems, such structures are already natural. In numerical science, ordinary quantitative cost models may be more appropriate. Linear logic should therefore be one tool in a modular family rather than the universal logic of limits.

12.8 Algebraic and categorical logic as the organization layer

Once several logics coexist, the foundation needs a theory of their relationships. Algebraic logic studies how logical consequence can be represented by algebraic structures. Boolean algebras correspond to classical propositional logic; Heyting algebras to intuitionistic logic; other lattices and algebras represent modal and many-valued systems. This can give OCL a uniform method for declaring and comparing local logics.

Categorical logic offers another level of organization through the correspondence between logical systems, type theories, and categories. The Curry-Howard-Lambek tradition shows deep relationships among propositions, types, proofs, programs, and categorical morphisms. These relationships are highly relevant to a foundation in which theorem proving and executable transformation are closely linked.

The outfinite contribution would be to add existence modes, theory bridges, and frontiers to this modular picture. Rather than building one enormous logic, the meta-layer could record which algebraic or categorical semantics a local logic uses and what translations preserve which judgments.

This architecture may turn out to be the most mathematically elegant path for outfinitism, but it is not yet justified. The programme must discover whether the extra structure is necessary for actual scientific and computational cases.

12.9 Logic selection as a meta-rational and falsifiable act

The presence of several useful logics does not imply that the outfinite foundation should become logically permissive in an uncontrolled sense. Classical, intuitionistic, modal, temporal, fuzzy, paraconsistent, linear, and probabilistic systems answer different kinds of questions. The foundational problem is to specify when a logic is appropriate, how its conclusions are typed, and how results move between logical regimes.

Classical logic remains the natural choice for many exact mathematical theories. The law of excluded middle and proof by contradiction are extraordinarily powerful and need not be abandoned merely because physical realizations are finite. The outfinite concern is narrower: a classical theorem about a theory-local object should not be confused with constructive production, empirical observation, or bounded verification. Classical truth is one well-defined relation inside a larger architecture.

Intuitionistic logic is valuable when existence should correspond to construction. In a computational setting, a proof of existence can carry a witness and a proof of implication can carry a transformation. This is closely aligned with proof assistants and dependent types. Yet requiring every scientific existence claim to be constructive would be too restrictive, because science routinely reasons with latent parameters, stochastic objects, or inferred mechanisms. The framework should therefore preserve the distinction between constructive and nonconstructive existence rather than legislate one as universally correct.

Modal logic is a natural candidate for existence modes because modalities can express necessity, possibility, knowledge, temporal status, or theory-relative truth. A modal treatment might make it formally impossible to infer physical existence from theoretical existence without an explicit rule. The challenge is to avoid creating a proliferation of modalities whose interaction axioms are opaque. The programme must discover a small and interpretable modal structure if this approach is to be useful.

Temporal and dynamic logics are indispensable for processual existence. They reason about what holds now, eventually, always, after an action, or along a path. Their integration would allow the foundation to state persistence conditions for processes and collectives without encoding time as an incidental extra variable. Again, the frontier matters: a temporal claim may be proved globally in a finite-state model or only within an explored horizon.

Fuzzy logic should be reserved for genuinely graded predicates, not used as a generic substitute for uncertainty. Membership in a vague category, degree of similarity, or cohesion of a collective may be naturally graded. The probability that an event occurs is a different concept. Confidence in a source is different again. Collapsing all of these to numbers between zero and one would undo the very semantic discipline the programme seeks.

Paraconsistent and four-valued logics address another real problem: scientific and AI systems can contain conflicting evidence. Classical explosion makes an inconsistent database unusable if contradictions are treated naively. A paraconsistent layer can localize conflict, allowing the system to represent that a claim has support, refutation, both, or neither while preserving provenance. This is potentially important for literature review, where different experiments or theories may conflict without making every unrelated conclusion derivable.

Linear logic offers a language in which assumptions behave like resources. This resonates with an outfinite sensitivity to resource use, but the analogy must remain controlled. Physical energy and memory are not automatically linear-logic propositions. The useful question is whether linear or substructural logics can represent consumable permissions, experimental samples, one-use capabilities, or proof resources in ways that improve composition.

Algebraic logic and categorical logic may provide the organization layer. They allow logical systems to be studied through algebraic structures and semantic categories, making relationships among logics explicit. Rather than creating an ad hoc switch statement selecting a logic for each theorem, the programme should seek formal embeddings, translations, and conservative extensions. A result imported from one logic should retain enough information to prevent stronger rules from being applied illegitimately.

This is where meta-rationality becomes testable. The ideal is not unrestricted pluralism but controlled selection among partial reasoning systems according to the structure of the problem. A theory should state why a particular logic is used and what is lost or gained by the choice. Comparative benchmarks can then ask whether the multi-logic architecture detects contradictions, vagueness, temporal dependence, or constructive obligations more accurately than a single classical encoding. If the complexity of coordinating logics exceeds the semantic benefit, the programme should reduce the number of primitive logics and express more distinctions through types or modalities inside a smaller core.

12.10 Research obligations and falsifiability

The logic architecture should be tested on problems that genuinely require different semantics. Constructive existence, temporal safety, graded collective membership, contradictory evidence, and resource-sensitive protocol actions should each be formalized in a logic suited to the task. The meta-layer should prevent invalid cross-logic inferences while allowing justified translations.

A critical falsification test is simplification. If all benchmark cases can be expressed clearly and safely in one established dependent type theory with ordinary libraries, the proposed modular logic layer may be unnecessary. The programme should prefer reuse over ideological pluralism.

Another test concerns confusion between fuzzy degree, probability, and evidence. Synthetic scientific datasets can be constructed in which these quantities intentionally diverge. An AI reasoning through the typed logic should avoid invalid numerical combinations. If the formal distinctions do not improve reasoning accuracy, the practical case for typed non-classical semantics is weak.

Part IV. Reconstructing major mathematical domains

Chapter 13. Collections, set-like theories, size, and diagonal reasoning

13.1 Why set theory should be reframed rather than rejected

Set theory gave modern mathematics a remarkably successful common language. Numbers, functions, relations, spaces, algebraic structures, and much more can be encoded as sets. The outfinite programme would gain little by pretending that this expressive achievement is an error. The more precise proposal is that set theory should be one possible theory within a broader foundation rather than the only ontological template through which every form of mathematical and scientific existence is interpreted.

The practical reason is that the word collection hides several different structures. A finite collection can be explicitly enumerated. A generative domain can produce values without ever materializing all of them. A class can be specified by a predicate or recognizer. A completed infinite totality can be admitted as a theoretical object by an axiom. A collective system can contain changing participants without being identical to the instantaneous collection of those participants. Treating all of these as one undifferentiated notion of set can be mathematically harmless inside a sufficiently strong theory, but it obscures distinctions that matter to computation and scientific interpretation.

An outfinite foundation should therefore provide several collection-like primitives or views. The exact choice remains a design question. The minimum requirement is that an explicitly enumerated finite collection, a generative domain, a predicate-defined class, and an ideal theory-whole not be conflated merely because they admit a membership-like relation.

This distinction also clarifies what it means for an AI system to reason over an open domain. It need not imagine a stored container holding all values. It can reason parametrically about any supplied witness generated under a rule. That is a different computational and ontological operation from iterating over a finite database table.

13.2 Membership is not one universal relation

The notation for membership is elegant because it unifies mathematics. In applied ontology, however, several relations deserve separate names. A number can be contained in a finite collection. A value can be generated by a domain rule. An object can satisfy a predicate. A component can be part of a structure. An agent can participate in a collective. A theoretical object can be admitted by a theory.

These relations obey different dynamics. A member can leave a collective while the collective persists. A generated value remains generable regardless of whether it has ever been produced in a particular execution. A physical component can be replaced. A theoretical set can contain an element independently of observation.

The programme should not prohibit a theory from abstracting these relations into ordinary membership. It should require the bridge to be visible when such abstraction is semantically significant. This is particularly important for executable science, where a machine may otherwise treat participation, containment, generation, and observation as interchangeable binary predicates.

13.3 Size as a family of views

Galileo's observation about natural numbers and perfect squares remains a useful illustration. A one-to-one correspondence pairs every natural number with a square. Under Cantorian cardinality, the two infinite sets have the same cardinality. Yet within finite prefixes the density of squares tends to zero. There is no contradiction because the two comparisons ask different questions.

Outfinite mathematics should generalize this lesson. Size need not be a single primitive for every domain. Finite cardinality remains exact for finite collections. Generative domains can also be compared by pairability, growth rate, density under a chosen enumeration, representation cost, generation cost, coverage at a frontier, or other application-specific measures.

Cantorian cardinality remains available inside theories that admit completed infinite sets. Outfinitism should not claim that prefix density is a superior replacement. Pairability and density capture different invariants. The advantage of the outfinite meta-language would be to make the chosen view explicit and to prevent a result under one view from being interpreted as a result under another.

This has practical analogues beyond arithmetic. Two state spaces can be equinumerous but radically different in reachability under a process. Two model classes can have similar parameter counts but very different effective complexity. Two networks can contain the same number of nodes but differ in communication cost and resilience. A science of limits should prefer measures that preserve the operational question being asked.

13.4 Diagonalization and the limits of enumeration

Diagonal arguments are among the deepest uses of infinite reasoning. Cantor's diagonal argument, the construction of uncomputable functions, and related results in logic demonstrate that certain enumerations cannot capture all objects of a specified theoretical class. An outfinite programme must engage these results directly rather than avoiding them.

There are at least two possible interpretations. In a theory admitting completed infinite totalities, the classical diagonal theorem can be imported as a theoretical result. In a more generative setting, diagonal reasoning can be interpreted as a rule that, given any proposed finite description of an enumerator with specified capabilities, constructs or specifies an object that escapes the enumeration under the relevant semantics.

The second interpretation does not automatically recover every classical diagonal theorem. Some arguments depend essentially on quantification over completed function spaces or power sets. The programme should not assume in advance that these can all be translated without loss. Their treatment is one of the places where the natural boundary of outfinite mathematics may emerge.

The useful methodological point is that failure of enumeration is itself a frontier phenomenon. A finite machine can manipulate a finite description of an uncountable theoretical space without enumerating its elements. The theorem is about the relation between representations and the larger theoretical object. This is exactly the kind of distinction typed existence is designed to preserve.

13.5 Self-reference, paradox, and typed collection formation

Naive unrestricted collection formation leads to familiar paradoxes. Modern set theories avoid them through disciplined axioms. Type theories avoid many self-membership problems by stratifying objects. Outfinite mathematics should reuse these lessons rather than claiming that finiteness by itself solves paradox.

A predicate-defined class should not automatically be promoted to a realized collection or a completed theory-whole. The act of collecting requires a constructor authorized by the local theory. This prevents a recognizer from being mistaken for an object containing every thing it recognizes. It also aligns with programming practice: a predicate over a type is not the same thing as an allocated list of every satisfying value.

This distinction is valuable for AI because natural language frequently shifts between extension and intension. A phrase such as "all systems satisfying condition P" can mean a logical class, a database query result, a theoretical set, or a generative search space. A typed collection theory would force the intended meaning to be declared.

13.6 Collection primitives, generated domains, and theory-wholes

One of the strongest methodological commitments of the programme should be to avoid asking a single primitive notion of set membership to carry several intuitively different relations. A finite collection that has been explicitly enumerated, a generative domain whose elements are produced by a rule, a class recognized by a predicate, a population sampled through observation, and a completed infinite totality admitted in a theory all support different operations and different evidence. Classical set theory can encode all of them, but an outfinite application layer may benefit from preserving the distinctions rather than compiling them away immediately.

For a finite collection, cardinality is an exact executable quantity. Membership can be decided by the representation when the elements are explicitly stored. For a generated domain, the relevant relation may instead be whether a supplied value carries a generation or admission witness. The domain need not be materialized in full. For a predicate-defined class, membership can be an executable or semi-decidable recognition problem. For a theory-whole, quantification may be licensed by the axioms of the imported theory even though the object is not physically enumerated. These are different epistemic acts under the same ordinary phrase "x belongs to A."

This separation gives a more precise reading of classical examples. The natural numbers can be represented as a generative domain when the goal is computation or constructive arithmetic, while a Cantorian theory may additionally introduce the completed totality of naturals as a theoretical object. Neither representation needs to invalidate the other. The research question is which theorems require the completed totality and which can be reconstructed from finite witnesses and parametric rules.

Galileo's comparison between natural numbers and squares illustrates why size should also be typed. There is a one-to-one pairing between each natural number and its square. From the Cantorian viewpoint this supports equality of cardinality. Within finite prefixes, however, squares become increasingly sparse. A computational view can additionally compare generation cost or representation size. These statements are not contradictory because they answer different questions about size. An outfinite framework should make the chosen view

explicit rather than declaring one notion universally privileged outside the theory that motivates it.

The same discipline applies to diagonal arguments. Diagonalization is a profound and valid method inside theories that quantify over suitable totalities. The outfinite programme should not dismiss it on physical grounds. Instead, it should ask what existence assumptions the diagonal construction uses. When a theorem states that no enumeration covers a completed domain, the conclusion is theory-local unless a bridge identifies that domain with a physical or computational system. For actually stored finite objects, enumeration behaves differently and the theorem may have a finite combinatorial counterpart.

Typed collection formation also offers a clean response to certain paradoxes. A property or recognizer does not automatically generate an object containing every entity that satisfies it. The transition from a predicate to a collection is an explicit construction with its own admissibility rules. This is compatible with lessons already learned in type theory and axiomatic set theory; the outfinite contribution is to make the construction relevant to broader existence modes, including observational populations and generated process states.

A serious foundation must nevertheless demonstrate that these primitives do more than rename familiar distinctions. One benchmark should translate ordinary finite set theory, a fragment of classical infinite set theory, and practical data-collection tasks into the proposed system. The framework should preserve standard results where appropriate while making illegal certain category errors, such as treating an observed sample as the complete population or treating a generated domain as if all of its values had been realized. If users must manually manage multiple nearly identical collection constructs without gaining clearer inference, the design should be simplified.

The research programme should also examine interoperability with category and type theory. A finite collection may be an ordinary finite type. A generated domain may be represented by an inductive family or a coalgebraic process. A theory-whole may be hosted by an imported classical universe. The objective is not to create a rival to set theory but to design a semantic layer in which the role played by a collection is visible enough for executable science and AI reasoning.

13.7 Research obligations and falsifiability

The set-like layer should be evaluated through formal translations. Standard finite set theory, a fragment of classical set theory, and a generative domain calculus should all be represented in the core. The framework should make clear which theorems are conservative translations and which depend on stronger existence assumptions.

Diagonal arguments are a particularly important stress test. The programme should formalize representative classical diagonal theorems and determine which parts can be reformulated constructively or outfinally. If the framework avoids major results only by reclassifying them as external classical theorems, its claim to broad mathematical coverage must be reduced accordingly.

The practical benchmark should test AI reasoning over ambiguous collection language. Problems can be designed in which a finite list, a predicate-defined class, a database relation, and an ideal set share the same informal wording. If typed collection semantics reduces invalid operations and category errors, it provides measurable support for the approach.

Chapter 14. Analysis without implicit physical infinity

14.1 Exact theoretical objects and finite information

Mathematical analysis concentrates many of the conceptual pressures that motivate outfinitism. Real numbers can carry infinite information in their standard mathematical representation. Limits quantify over arbitrarily small tolerances or arbitrarily large indices. Infinite series, integrals, derivatives, and function spaces are central objects. Any programme that simply excludes these constructions would abandon a large part of modern science.

The more productive strategy is to separate the exact theoretical object from the finite information available about it in computation or measurement. An exact real number may remain a legitimate theoretical entity. A physical measurement returns finite information constrained by instrument resolution and noise. A numerical algorithm returns a finite representation, perhaps with an error certificate. A symbolic expression may denote an exact real value without being numerically expanded.

This separation is already familiar in computable analysis, constructive analysis, interval arithmetic, numerical analysis, and proof assistants. The outfinite hypothesis is that the same distinction should become part of a general ontology rather than a domain-specific convention.

One useful computational model is a refinement object. Instead of representing a real by an actually materialized infinite digit sequence, a program provides a procedure that, when given a finite tolerance, attempts to return a finite enclosure or approximation certified to that tolerance. The theoretical value and the refinement procedure are related but not identical. The procedure may reach a resource frontier even when the theoretical value is perfectly well defined.

14.2 Limits as stabilization contracts

The classical epsilon-style definition of limit is already structurally close to an outfinite view. It does not require executing a process at infinity. It states a finite conditional relationship: for every requested tolerance there is a finite threshold after which the specified relation holds. The notational reference to infinity in a limit such as a sequence tending to a value is therefore often an abbreviation for a quantified stabilization property.

Outfinitism can exploit this structure while making the implementation bridge explicit. A theoretical theorem can remain exactly the classical theorem. An executable version adds a procedure or certificate that constructs a usable threshold for a given tolerance, where such construction is available, and records the representation and resources needed to verify the bound.

This distinction matters because not every classical existence proof yields an efficient modulus of convergence. A sequence can be proven to converge while the rate needed for computation is unknown or expensive. An outfinite executable theorem should distinguish existence of a theoretical limit from availability of a constructive stabilization bound.

The geometric series provides a simple example. Finite partial sums admit an exact residual relation. Rather than imagining that an actual computation completes infinitely many additions, one proves that the residual can be made smaller than any requested positive tolerance by a sufficiently advanced finite stage. The exact sum remains a theoretical invariant; the computation returns certified finite information.

14.3 Differentiation as local model plus residual

The derivative is often introduced through a limit, but many practical uses interpret it as the coefficient of a local linear approximation. An outfinite formulation can emphasize this approximation contract.

For a simple polynomial such as the square function, symbolic algebra shows exactly how the finite difference quotient decomposes into the candidate derivative plus a residual depending on the step size. The derivative is the theoretical local coefficient, while a finite numerical estimate carries a residual. For more complex functions, Taylor-type results, interval methods, automatic differentiation, or verified numerical analysis can provide corresponding certificates.

This approach has two advantages. It keeps the exact analytical theorem intact while making the operational approximation explicit. It also prevents a measurement with a finite step from being treated as if it had literally realized an infinitesimal.

The concept of an infinitesimal can still be admitted in theories that use it, including nonstandard analysis. Outfinitism does not need to legislate a single acceptable foundation for calculus. It needs to type the existence mode of the infinitesimal object and distinguish it from finite measurement resolution.

14.4 Integration as certified aggregation

Integration offers a similar distinction. An exact integral can exist inside a theory. A numerical integrator partitions a finite interval, evaluates a finite number of points or subregions, and produces an approximation. Verified integration algorithms can return upper and lower bounds or error estimates.

An executable scientific theory should retain both levels. The symbolic integral may support algebraic derivation. The numerical realization may provide evidence that a particular computed quantity lies within a certified interval. A bridge connects the ideal definition to the algorithm through convergence and error theorems.

The frontier is often informative. If the integrator cannot meet the requested tolerance within available resources, the correct result is not an arbitrary number but a finite enclosure together with the residual uncertainty or a report that the required guarantee was not obtained. This is a general pattern for outfinite computation.

14.5 Continuity, compactness, and harder problems

Continuity can be formulated as a local stability relation between input and output tolerances. In many practical contexts, the important object is a modulus or a locally computable bound. Yet classical analysis includes theorems whose standard proofs use compactness, choice principles, or nonconstructive existence in ways that do not automatically yield algorithms.

This is an important boundary for the programme. Outfinitism should not promise that every classical existence theorem will acquire a finite witness of equal strength. Some results may remain purely theoretical. Others may require stronger principles admitted in a local theory. The foundation is successful if it makes the difference visible, not if it forces a constructive interpretation where none is known.

Compactness itself is a particularly revealing test because it often turns local information into a global existence result. An outfinite account may distinguish several levels: a classical compactness theorem inside an ideal theory, a constructive compactness result with explicit finite subcovers, and a computational algorithm that can find such a cover under additional assumptions. These are related but distinct artifacts.

14.6 Reconstructing analysis as finite challenge-response mathematics

Mathematical analysis is a decisive test because many of its central concepts are traditionally expressed through infinite sequences, completed real-number structures, or arbitrarily fine limiting processes. If the outfinite programme can only reformulate discrete computation, it is not yet a serious candidate foundation for broad mathematics. At the same time, analysis already contains a powerful finite pattern that the programme can exploit: the epsilon-style statement of a limit is a challenge-response relation between a requested tolerance and a finite condition sufficient to meet it.

The outfinite interpretation does not need to abolish exact real numbers. It can allow them as theoretical objects while distinguishing them from finite information available in computation or measurement. A real quantity can be approached through a refinement procedure that returns a certified interval when given a tolerance and a resource context. The exact object and the interval are not rival definitions of the same existence. They are linked representations at different levels.

This distinction clarifies many practical calculations. A root-finding algorithm does not produce an infinitely precise real number; it produces an enclosure or approximation whose residual can be bounded. A numerical integral returns a value together with discretization and rounding error estimates. A differential-equation solver produces a finite trace with a stability and error model. The mathematical ideal remains valuable because it defines the target, while the outfinite layer formalizes the bridge from target to finite evidence.

Limits can then be phrased as stabilization contracts. Given any admissible positive tolerance, the theory constructs a finite condition after which all admissible finite stages lie inside the requested error. Nothing is gained by speaking of a literal computational step at infinity. The universal force of the theorem comes from the parametric rule that accepts an arbitrary finite tolerance challenge and produces a finite response.

The geometric series is an elementary example because the residual is explicit. Each finite partial sum is related to the target by a remainder that decreases geometrically. The classical statement that the infinite sum equals two and the outfinite statement that every positive tolerance can be met after a computable finite stage are closely related but not identical in interpretation. The first is concise inside a theory of infinite series. The second exposes the information required by computation. A mature foundation should allow both and formalize their bridge.

Differentiation can be analyzed similarly. The derivative is an exact theoretical local linearization, but numerical use depends on a finite neighborhood, finite precision, and a

residual term. For a polynomial such as x squared, the difference quotient can be simplified symbolically to show that the local slope differs from two x by a term proportional to the step size. The result is a finite identity that explains the limiting derivative and also exposes the approximation error. More complex differentiability theorems can be treated as proofs that a residual becomes controllably small under explicit conditions.

Integration provides a complementary aggregation problem. Exact integrals may be theoretical objects, while numerical quadrature partitions a domain finitely and produces upper and lower estimates or other certified bounds. Adaptive refinement is an outfinite process: no logical final partition is required, yet every computed partition is finite and the procedure may stop when tolerance is met or when a frontier such as precision, time, or singular behavior is reached.

The harder questions involve compactness, completeness, and existence theorems. Classical analysis often proves that an object exists without constructing a practical witness. The outfinite programme should not assume in advance that every such theorem must be discarded. Instead, it should classify the theorem as theory-level existence and investigate constructive, approximate, or finite-witness companions when applications require them. Some classical results may have efficient outfinite translations; others may expose genuine limits of the approach.

This makes analysis a natural falsification domain. A representative corpus should include limits, continuity, differentiation, integration, root existence, fixed points, compactness arguments, and basic differential equations. Researchers should compare classical formalizations, constructive formalizations, computable analysis, interval methods, and outfinite versions. The programme succeeds only if its existence types and frontiers explain or automate something that the neighboring traditions do not already provide as cleanly. It must be prepared for the result that large parts of analysis are better imported from existing constructive or classical foundations rather than rebuilt.

14.7 Research obligations and falsifiability

Analysis should be one of the hardest mathematical benchmarks. The programme should build a small verified library covering real refinement, convergence, continuity, differentiation, and integration. The library should interoperate with an established proof assistant so that the outfinite layer does not rely on unverified claims.

A useful falsification experiment would compare the outfinite representation with computable and constructive analysis. If every concept maps directly to existing frameworks and the existence/frontier layer adds no new compositional benefit, the programme should explicitly adopt those frameworks rather than claim originality. Conversely, if the layer clarifies translation from exact theorem to numerical implementation and measurement, that bridge should be demonstrated in executable examples.

The strongest practical benchmark is error propagation across theory, algorithm, and data. A model with an exact analytical relation should be connected to finite-precision computation and finite-resolution observation. The framework should calculate which parts of the final uncertainty arise from approximation, measurement, and model assumptions. Failure to keep these sources separate would undermine the programme's central epistemic claim.

Chapter 15. Probability, statistics, and the separation of uncertainty types

15.1 Probability is not evidence, confidence, or vagueness

Probability is another domain in which a single numerical form can hide several meanings. A probability may be a parameter of an ideal stochastic model, a long-run frequency interpretation, a subjective degree of belief, a predictive distribution, or a physical propensity depending on the theory adopted. A confidence interval is not the probability that a fixed parameter lies in the realized interval under the ordinary frequentist interpretation. A fuzzy membership degree is not a probability. A model's predictive uncertainty is not the same thing as uncertainty about whether the model is appropriate.

Outfinite mathematics should therefore type probabilistic quantities by interpretation and evidence context. The intention is not to settle philosophical debates about probability. It is to prevent an executable theory from combining quantities that happen to lie between zero and one while representing different semantics.

A probabilistic model can be perfectly well defined as a theoretical object. Its parameters may or may not correspond directly to observed frequencies. Data provide finite evidence. Statistical inference connects data to model parameters under assumptions about sampling and model structure. The bridge between model and data is therefore as important as the internal probability calculus.

15.2 Finite probability spaces and generated experiments

Finite probability spaces fit naturally into an outfinite foundation because their outcomes can, at least in principle, be enumerated. More interesting scientific processes involve repeated or open-ended experiments. A stochastic process can be specified by a finite generative rule without materializing an infinite sequence of realized trials.

A realized dataset always contains finitely many observations. The model may reason about arbitrary finite sample sizes or about ideal limiting behavior. An outfinite representation should preserve this difference. It can state finite-sample concentration bounds, exact combinatorial probabilities, or Bayesian posterior calculations without requiring an actual infinite frequency sequence.

This is scientifically valuable because practical decisions are always based on finite evidence. Asymptotic theorems remain useful because they summarize how finite behavior improves as sample size grows. The outfinite layer asks for the finite threshold or error statement when the theorem is applied.

15.3 Large-sample theorems as families of finite guarantees

The law of large numbers is often informally described by saying that empirical frequency becomes the true probability as the number of trials tends to infinity. This language can invite a misleading picture of an actual experiment reaching an infinite stage. The mathematically meaningful content can instead be expressed through finite probability bounds that become tighter as sample size increases.

Concentration inequalities are especially compatible with this interpretation. For any finite sample size and error tolerance, they provide a finite upper bound on the probability of a deviation. The asymptotic statement summarizes the behavior of this family of finite bounds.

An outfinite scientific system would prefer the finite bound when supporting a practical decision. It may also retain the limiting theorem as a theoretical result. The two levels complement rather than replace each other.

This distinction is potentially useful for AI systems that otherwise cite asymptotic consistency as if it guaranteed performance for a particular finite dataset. A typed theorem can force the agent to identify the finite regime or acknowledge that only an asymptotic claim is available.

15.4 Statistical inference and model boundaries

Statistical conclusions are conditional on assumptions about data generation, independence, sampling, missingness, measurement, and model family. In practice, many severe errors occur because a correct statistical formula is applied under invalid assumptions. This is exactly the kind of boundary problem outfinitism wants to formalize.

A statistical result should therefore record the model under which it was derived, the evidence used, the uncertainty statement produced, and the diagnostics or bridge assumptions connecting the model to the data. A change in the data-generating process should not leave the old conclusion silently valid.

The empirical layer also needs to distinguish model uncertainty from parameter uncertainty. A posterior distribution conditional on a model can be mathematically exact while uncertainty about whether the model is appropriate remains outside that distribution. Outfinite theory bridges could make this separation explicit.

15.5 AI, automated statistics, and semantic discipline

AI can automate large parts of statistical analysis: data cleaning, model proposal, code generation, diagnostic testing, and report production. This increases the risk that a workflow becomes mechanically fluent while semantically wrong. An agent can fit a model that violates causal assumptions or report a confidence measure under an inappropriate interpretation.

Outfinite typing could provide guardrails. An agent proposing a statistical result would have to identify the model and evidence relation used. A review agent could search for missing assumptions. A proof or calculation engine could verify algebraic derivations. None of this solves the scientific problem of choosing the right model, but it makes the choice visible and reviewable.

15.6 Statistical evidence, finite guarantees, and falsification boundaries

Probability and statistics are particularly vulnerable to semantic conflation because the same numerical language is used for theoretical probabilities, empirical frequencies, confidence procedures, posterior beliefs, predictive scores, and sometimes vague informal uncertainty. An outfinite foundation should not invent a new probability calculus unless necessary. Its contribution should be to preserve the type of uncertainty and the bridge that justifies moving from one interpretation to another.

A probability measure inside a mathematical model is a theoretical object. An observed relative frequency is a finite statistic. A confidence interval is the output of a procedure with a

repeated-sampling coverage claim under assumptions. A Bayesian posterior is conditional on a prior and likelihood model. A machine-learning confidence score may have no calibrated probabilistic semantics at all. Treating these as interchangeable because they are numbers between zero and one is precisely the kind of category error the programme aims to prevent.

Large-sample results can be restated in a way that makes their finite content central. Instead of narrating the law of large numbers as if an actual experiment reaches infinity, one can emphasize families of finite bounds. For every required error and failure probability, the theory can derive a sample size sufficient under the model assumptions. Concentration inequalities are naturally outfinite in this sense: they produce finite guarantees indexed by sample size, variance assumptions, dependence structure, and tolerance.

This viewpoint does not reduce asymptotic statistics to elementary bounds. Asymptotic theory is extremely powerful for understanding estimators and limiting distributions. The outfinite question is how an asymptotic theorem is connected to the finite dataset in front of the scientist. The bridge may be justified by explicit finite-sample bounds, simulation, higher-order approximations, or empirical diagnostics. If no bridge is available, the use of the asymptotic result should remain marked as an approximation rather than becoming an exact theorem about the data.

Model boundaries matter as much as sample size. A statistical guarantee usually assumes a data-generating structure, independence pattern, stationarity condition, missing-data mechanism, or distributional family. Violating these assumptions can be more damaging than using a small sample. A machine-readable theory should therefore attach the guarantee to the model and preserve evidence for whether the assumptions are plausible in the present dataset.

This creates a direct link to scientific falsification. A failed prediction can refute a probabilistic model only relative to a statistical decision rule; rare events are not logical contradictions. Conversely, a model that assigns high probability to observed data is not thereby proven true. Outfinite scientific reasoning should distinguish deductive refutation from statistical incompatibility and from model comparison. The distinction is essential for AI reviewers, which otherwise risk turning probabilistic evidence into categorical language.

The programme should test whether typed uncertainty improves automated analysis. Benchmarks can deliberately mix probabilities, frequencies, confidence levels, measurement error, and fuzzy membership. Systems should be evaluated on whether they refuse invalid arithmetic or inference across these types unless an explicit conversion is supplied. A second benchmark can test whether AI-generated scientific conclusions preserve the assumptions and finite-sample domain of statistical theorems. If the typed layer adds little beyond established probabilistic programming and statistical metadata, the contribution should be narrowed accordingly.

15.7 Research obligations and falsifiability

Probability and statistics offer excellent adversarial benchmarks. Test problems should deliberately mix model probability, empirical frequency, Bayesian degree of belief, fuzzy membership, and confidence measures. The system should prevent invalid substitutions while allowing explicitly declared bridges. Human and AI performance can then be measured with and without typed uncertainty semantics.

A second benchmark should compare asymptotic reasoning with finite guarantees. AI systems can be given practical questions where an asymptotic theorem is true but useless at the available sample size. An outfinite representation should make the finite frontier visible and reduce unjustified claims.

A third benchmark should use dataset shift. A model is trained or calibrated under one distribution and then applied to another. The formal dependency graph should show which guarantees no longer transfer. If the framework cannot express this common failure mode more clearly than ordinary documentation, its scientific value is limited.

Chapter 16. Geometry, topology, optimization, and dynamical systems under finite observation

16.1 Geometry: ideal object, construction, and measurement

Geometry illustrates the difference between theoretical and physical existence with unusual clarity. A mathematical point has no extension in the usual ideal theory. A physical mark has finite size. A straight line can be unbounded in a theory while every physical construction is finite. An exact angle can be defined algebraically while an instrument reports an interval.

None of this invalidates geometry. It suggests that executable geometry should distinguish at least three layers: the ideal geometrical object, the finite computational representation, and the measured physical realization. Bridges between them carry tolerances and calibration assumptions.

This distinction is already central in computational geometry and computer-aided design, where numerical tolerances can determine whether two objects are treated as intersecting. An outfinite foundation would generalize the discipline so that approximate equality is never silently substituted for exact equality.

The same approach can be applied to space-time models, manifolds, and coordinate systems. Infinite or continuous structures can remain theoretical objects. Finite observations and simulations interact with them through explicit maps.

16.2 Topology as a theory of distinguishability and refinement

Topology is traditionally formulated using open sets, neighborhoods, continuity, convergence, and related structures that may be infinite. The outfinite programme can reinterpret part of this theory through distinguishability under observation.

Two states may be mathematically distinct yet indistinguishable at a given resolution. Refining an observation can separate states previously treated as equivalent. This suggests a family of observational topologies or uniform structures indexed by resolution, information, or measurement capability.

Such ideas are not new in topology, domain theory, formal topology, constructive analysis, or information-based semantics. The research question is whether they can provide the appropriate bridge between ideal topology and finite observation in executable science. A topological statement inside a theoretical model should not be confused with what a finite sensor can distinguish.

This may be particularly useful in AI and data analysis. Learned representations often treat nearby points as similar under a metric that is itself model-dependent. An outfinite view would require the metric or topology to be part of the declared observation theory rather than an implicit property of reality.

16.3 Optimization: ideal optimum versus certified gap

Optimization exposes a common ambiguity between existence and attainment. A mathematical problem may have a global optimum in an ideal domain. A numerical solver may find a candidate solution without proving optimality. It may produce a lower or upper

bound that certifies an optimality gap. It may terminate at a local optimum or at a resource frontier.

An outfinite result should preserve these statuses. The phrase "optimal solution" should be reserved for cases with an appropriate certificate relative to the model. Otherwise the result should be described as the best witnessed solution under the search performed, together with whatever bound is known.

This discipline is especially relevant for AI planning and scientific design. Generative models can propose high-quality candidates but rarely justify global optimality. A formal optimization layer can separate proposal from certificate. Coding agents can invoke exact solvers, relaxations, interval methods, or branch-and-bound procedures and report the frontier honestly.

The theoretical optimum can still exist as an ideal object even when no feasible computation finds it. Again, theoretical existence and computational realization remain distinct.

16.4 Dynamical systems and the horizon of prediction

Dynamical systems bring together process, state, stability, reachability, approximation, and time. A differential equation can define an exact flow in a theory. A simulation approximates trajectories over finite time. A model checker explores reachable states under a discrete abstraction. Observations constrain only part of the state.

The notion of a prediction horizon is naturally outfinite. A model may be well validated and numerically stable over a finite region while long-term behavior becomes sensitive, uncertain, or computationally inaccessible. The correct conclusion is not that the underlying theory ceases to exist, but that the bridge from theory to reliable prediction has reached a frontier.

Chaos provides an obvious example. Deterministic equations can generate trajectories whose long-term prediction requires exponentially increasing precision in initial conditions. A finite observer and finite computer therefore possess a practical horizon even when the theoretical evolution is well defined. Outfinitism can treat that horizon as a mathematical object attached to the observation and computation model.

Reachability analysis offers another example. A finite-state abstraction may prove that an unsafe region is unreachable. An interval or zonotope analysis may over-approximate continuous dynamics. A bounded simulation can produce counterexamples but not prove global safety. These methods should be integrated through the typed evidence calculus described earlier.

16.5 Boundary-sensitive geometry, topology, optimization, and dynamics

The domains grouped in this chapter share a common theme: ideal mathematical structure is often used to reason about systems observed or computed only at finite resolution. The outfinite programme should avoid the simplistic conclusion that ideal geometry or topology is somehow illegitimate. The more productive aim is to formalize the relation between the exact theory and the finite observational or computational interface.

Geometry makes the distinction vivid. A mathematical point has no spatial extension, an exact line has zero width, and Euclidean constructions can rely on ideal incidence. A

measured location instead comes with resolution, coordinate uncertainty, instrument calibration, and sometimes a probability distribution. The mathematical point may be the correct theoretical object even though no measurement instantiates it exactly. A bridge maps finite observational regions to ideal geometric entities under a model of error.

Topology can be reinterpreted operationally through distinguishability and refinement. Two physical states may be indistinguishable under a given sensor resolution but distinguishable after refinement. Neighborhoods can be related to what perturbations or observations the theory treats as equivalent for a purpose. This does not replace classical topology; it suggests an indexed observational topology whose resolution is part of the scientific context. A limit or continuity theorem can then be transported to the observational layer only through a bridge that respects the chosen topology.

Optimization provides an especially practical distinction between an ideal optimum and a certified solution. Real solvers often return a feasible candidate, a lower or upper bound, an optimality gap, and a termination status. Calling the candidate "the optimum" erases useful information. An outfinite optimization result should preserve the gap and the frontier that prevented further improvement. In convex problems strong duality may provide excellent certificates. In combinatorial problems branch-and-bound already produces a natural finite proof object when the optimality gap closes.

Dynamical systems add time and prediction horizons. An exact differential equation may define behavior for an ideal continuum of time, while a numerical trajectory covers a finite interval with bounded error. Chaotic systems show that longer computation does not necessarily mean proportionally better prediction because sensitivity to initial conditions amplifies measurement uncertainty. A frontier-sensitive theorem should distinguish existence of a mathematical trajectory from the horizon over which observations support a particular prediction.

Reachability analysis ties these fields back to formal verification. The question whether a dynamical system can enter an unsafe region can be attacked through abstractions, interval methods, symbolic methods, barrier certificates, or finite-horizon search. Each method returns a different evidence profile. The outfinite framework could become useful if it allows these certificates to compose with discrete controller proofs and sensor models without hiding where approximation enters.

These domains therefore provide valuable cross-disciplinary benchmarks. A successful foundation should be able to represent an ideal model, a finite approximation, an observational interface, and a certificate of the relation between them. It should help an AI distinguish exact geometry from measurement, global optimization from a current best solution, and mathematical existence of a trajectory from reliable physical prediction. If the framework cannot handle these distinctions without domain-specific exceptions at every step, its claim to provide a reusable foundation should be weakened.

16.6 Research obligations and falsifiability

Geometry and topology should be tested on problems where ideal and measured equality diverge. The framework must avoid both false precision and arbitrary tolerance. A bridge should justify why a tolerance is appropriate for the measurement or computation. If tolerances remain informal constants with no provenance, the formalization has not solved the problem it claims to address.

Optimization offers a clear benchmark because solver certificates are already available in many settings. The system should distinguish exact optimum, certified gap, local optimum, heuristic candidate, and search frontier. AI-generated optimization reports can be evaluated for whether they overstate their results with and without the outfinite layer.

Dynamical systems provide the hardest test. A model with known numerical instability or chaotic sensitivity should be formalized together with its solver and observation precision. The framework should compute or at least represent a prediction horizon and prevent conclusions beyond it from inheriting the same confidence. Success here would strongly support the claim that a mathematics of frontiers adds value to executable science.

Chapter 17. Processes, phenomena, and collective existence as mathematical primitives

17.1 Why process should not always be derived from static objects

Traditional foundations can represent processes perfectly well by encoding time-indexed states, functions, relations, or transition systems. The outfinite programme does not claim otherwise. Its stronger philosophical proposal is that processes deserve primitive status because change is not a peripheral feature of the domains we increasingly want mathematics to describe.

Software systems execute. Biological organisms metabolize and adapt. Markets evolve. Institutions persist through changing membership. AI agents learn and interact. Scientific theories themselves develop through revision. In such domains, treating the stable object as primitive and process as a derived sequence can produce poor intuitions even when the formal encoding is technically adequate.

A process-centered foundation begins with state, transition, observation, continuity, and frontier. An object can then emerge as an abstraction that remains stable under a chosen view. This reverses the conventional explanatory direction without denying the usefulness of object-based mathematics.

Coalgebra, transition-system semantics, temporal logic, dynamical systems, process calculi, and event-based formalisms already provide much of the required mathematics. The outfinite contribution would be to integrate them with typed existence and theory bridges.

17.2 Phenomena as recognized patterns in process

A phenomenon such as turbulence, an epidemic wave, a market panic, or a social movement is not well represented by one immutable object. It is recognized through patterns in evolving data. The mathematical primitive is therefore not only the process but a recognizer that maps process histories or observations into a phenomenon judgment.

The recognizer itself has a domain of validity. A turbulence criterion may depend on scale and variables. An epidemic wave may be defined by incidence patterns and reporting practices. A social phenomenon may depend on measurement constructs that change over time. A machine-readable theory must therefore attach the phenomenon definition to a view and evidence model.

This is a natural place for graded or probabilistic reasoning, but the choice must be explicit. A phenomenon may have graded membership because its definition is fuzzy, or uncertain detection because data are noisy. Those are different semantic reasons for non-binary results.

17.3 Collective existence and persistence under member change

Collectives are among the strongest examples motivating typed existence. A swarm, corporation, scientific consortium, community, state, or distributed software system can persist while constituent members change. The collective is not a mystical entity detached from participants, but neither is it identical to a static set of them.

A formal account should identify the relations and invariants that make the collective operationally meaningful. These may include communication, shared memory, coordination mechanisms, decision processes, role continuity, boundary maintenance, and externally recognized capabilities. Different theories may choose different invariants because they answer different questions.

A corporation, for example, can have legal existence defined by documents and jurisdiction, economic existence defined by transactions and obligations, physical existence through people and assets, and collective functional existence through coordinated activity. These modes can diverge. A legally registered shell may remain a legal object while no longer functioning as an operational collective.

Meta-rationality is important here because there is no need to force one definition to be the only true essence of the corporation. The framework should let several theory-local definitions coexist and specify bridges where they interact. A legal contract may cause a change in economic behavior. A loss of operational coordination may eventually trigger a legal procedure. These are processes connecting existence modes.

17.4 Robustness and collective frontiers

Collective identity is often robust rather than exact. A swarm can lose some agents and remain cohesive. A distributed service can lose nodes and continue operating. A scientific collaboration can survive changes of personnel. This suggests robustness theorems: the collective persists under specified perturbations.

Graph theory, control theory, reliability, game theory, and network science provide appropriate tools. Abstract interpretation can summarize a large network by ranges of connectivity or participation. Symbolic execution can search for adversarial removal sequences. Bounded model checking can verify resilience up to a number of failures. Probabilistic models can estimate risk under stochastic failure assumptions. Fuzzy logic can describe graded participation. Paraconsistent logic can retain conflicting reports about member status.

The results should remain separated. A combinatorial proof that a network remains connected after any two node failures is not the same as a probabilistic estimate that two nodes are likely to fail. A fuzzy cohesion score is not a probability of existence. A legal record is not evidence of operational activity. Typed evidence prevents these distinctions from collapsing.

17.5 AI and collective systems

AI systems themselves increasingly appear as collectives: multiple agents, tools, human reviewers, memories, planners, and execution environments cooperate to solve a task. The identity of such a system is not necessarily the identity of one model checkpoint. A workflow can persist while individual agents are replaced.

Outfinite mathematics could therefore contribute to a theory of AI systems as processual collectives. Safety properties can be stated at the workflow level. Provenance can identify which agent produced a claim. A collective invariant can require that high-risk actions pass through an independent verifier. A frontier can be reached when the system lacks sufficient evidence or resources to proceed.

This is a concrete economic use case because industrial AI increasingly relies on orchestrated systems rather than isolated models. A mathematical foundation that represents workflow identity, tool authority, evidence, and bounded reasoning could support audit and assurance.

17.6 Collective identity as a stable abstraction rather than a mystical whole

Collective existence is one of the most distinctive and also one of the riskiest claims in the programme. It is distinctive because many sciences reason about entities whose components change while the higher-level entity persists. It is risky because a careless theory can turn any convenient grouping into an allegedly real collective. The concept therefore needs a disciplined mathematical interpretation.

A promising approach is to define a collective through a stable abstraction of interacting processes. The members and their local states form a concrete system. A macro-level description maps many concrete configurations to a smaller set of collective states. The collective earns explanatory status when the macro-description supports invariants, predictions, interventions, or causal accounts that remain robust under changes in individual membership. The abstraction should not be regarded as a metaphysical substance; it is a theory-relative level of organization with explicit persistence conditions.

A swarm is an intuitive example. Individual agents can enter or leave, yet the swarm may preserve cohesion, direction, and coordinated response. A distributed protocol can replace nodes while retaining a replicated state and consensus rules. A company can replace employees and assets while preserving legal identity, organizational memory, contracts, and decision procedures. An ecosystem can undergo turnover of organisms while maintaining recognizable interaction patterns. These cases differ substantially, but they share the problem of identity through changing components.

The outfinite framework should therefore treat membership, participation, and part-whole relations as distinct. A person can participate in an organization without being a mere element in the set-theoretic sense that exhausts the organization's identity. A legal entity can persist after functional coordination has collapsed. A network can remain topologically connected while losing the behavior that justified calling it a collective. Multiple existence types may therefore apply simultaneously, and they can diverge over time.

This is precisely where meta-rationality becomes useful. Different theories can describe the same organization legally, economically, socially, computationally, or physically. None of these perspectives is automatically the privileged ontology for every question. A legal theory may say that the company exists, while an operational theory says that no coordinated decision process remains. The outfinite aim is not to choose one answer but to maintain the distinction and to specify the bridges under which conclusions from one view can inform another.

Abstract interpretation offers a mathematical analogy. A collective-level state is an abstraction of many microstates. If a collective theorem is meant to guarantee something about all compatible microstates, the abstraction relation must be sound for that purpose. If the abstraction is used only for prediction, empirical validation may be more appropriate. The same macro-object can therefore have different justification modes depending on the claim being made.

Collective persistence also has frontiers. A swarm may remain coherent under loss of a fraction of agents but fragment beyond a threshold. A distributed system may tolerate a

bounded number of Byzantine failures. An organization may maintain function under gradual turnover but not after a critical loss of expertise. These thresholds are natural outfinite objects because they describe where an identity relation ceases to be justified under a chosen view.

AI makes the problem more urgent. Multi-agent systems, autonomous organizations, human-AI teams, and distributed research systems will increasingly exhibit collective behavior that is not reducible to one agent's state. A foundation capable of representing collective invariants, responsibility boundaries, and failure thresholds could support formal analysis of such systems. Yet the claim must remain falsifiable: if collective variables do not improve prediction, intervention, or compression relative to member-level models, the collective ontology should not be imposed.

The empirical programme should therefore compare alternative abstractions of the same systems. Candidate collective identities can be tested for robustness to member replacement, predictive stability, intervention relevance, and transfer across contexts. A theory of collective existence becomes credible only when it can tell us when a collective stops existing in a particular sense and when two competing abstractions produce measurably different explanatory value.

17.7 Research obligations and falsifiability

Collective existence is vulnerable to arbitrariness, so it should be treated as a stringent test. The programme should formalize several collectives with clearly different persistence criteria, such as a replicated software service, a swarm simulation, and an institution. Independent domain experts should evaluate whether the formal identity conditions match the phenomenon they care about.

Robustness claims should be linked to measurable behavior. If a collective model declares persistence after perturbations that domain experts regard as organizational collapse, the abstraction is wrong or the view is inappropriate. The framework must make such disagreement traceable to the declared theory rather than present the formal result as universal truth.

The AI workflow case offers an operational benchmark. A multi-agent system can be instrumented so that agents fail, are replaced, disagree, or reach tool limits. The theory should predict which failures preserve system-level invariants and which cross a collective frontier. If the formal model cannot outperform ordinary workflow monitoring in explanation or assurance, the proposed collective ontology has limited practical value.

Part V. Executable science and artificial intelligence

Chapter 18. Outfinite mathematics as a foundation for executable science

18.1 From scientific text to executable theory

Scientific theories are currently distributed across prose, equations, diagrams, code, datasets, conventions, tacit disciplinary knowledge, and references to experimental procedures. A human expert reconstructs the theory by combining these sources. A machine generally sees fragments. Even when the mathematical part is formalized, the relation between the formal object and the scientific referent often remains outside the proof system.

Executable science, in the sense intended here, is the attempt to turn the operational core of a scientific theory into a machine-manipulable object while preserving the distinctions that make scientific interpretation legitimate. Such an object would not replace the paper, the experiment, or the scientist. It would provide a formal layer beneath them.

A sufficiently rich executable theory would identify its entities and their existence modes, define its state variables and processes, record units and representation assumptions, state its mathematical relations, name its domains of applicability, connect theoretical quantities to observations, record uncertainty and provenance, and identify which conclusions are deductive, computational, statistical, or empirical. It would also be able to say that a question is outside its validated scope.

The phrase "near-complete formalization" should therefore be interpreted carefully. The programme does not assume that scientific concepts can be generated automatically from raw reality or that scientific judgment can be replaced by deduction. Concept formation, relevance, interpretation, experimental ingenuity, and theory choice remain open-ended human and machine activities. The tractable target is the part of a theory that scientists already regard as sufficiently explicit to support reproducible calculation, prediction, simulation, and argument.

If that layer can be formalized, computers can do more than execute equations. They can review the theory as a dependency structure. They can ask whether a conclusion uses an assumption that is absent from the current experimental context. They can detect that a numerical solver implements a different arithmetic from the symbolic derivation. They can identify when a claim is supported only by a theoretical model rather than observation. They can recompute consequences when a parameter or bridge changes.

This is where outfinite mathematics may differ from a conventional proof assistant. A proof assistant excels at establishing consequences inside a formal theory. Executable science requires an additional semantics for the boundary between theory, computation, and empirical evidence. The hypothesis is that typed existence and explicit frontiers provide the missing organizing layer.

18.2 Scientific theories as versioned, revisable objects

A scientific theory is not usually a frozen axiom system. It evolves. Parameters are re-estimated, models are refined, experimental anomalies accumulate, terminology changes, and different communities use partially incompatible variants. Executable science therefore needs versioned theories and explicit dependency management.

This fits the meta-rational approach. A new model need not erase the old model. The older theory can remain valid in a restricted regime. A theory bridge can state where the new model refines the old one and where it diverges. Scientific progress can then be represented as a network of theories and bridges rather than as a linear sequence of replacements.

Such a representation could support unusually precise review. When a new observation contradicts a bridge assumption, the system can identify which conclusions depend on that bridge. When a numerical implementation changes, equivalence checking can determine whether previously certified results transfer. When a model expands its domain, new obligations can be generated automatically.

This is a promising target for AI-assisted coding agents. Much of the work resembles software maintenance: dependency discovery, refactoring, test generation, regression checking, and documentation synchronization. Agents can perform these tasks at a scale difficult for human teams, provided that the formal artifacts remain independently verifiable.

18.3 Scientific review as machine-supported adversarial analysis

Peer review today relies heavily on human expertise and sampling. Reviewers rarely reproduce every calculation, verify every cited dependency, inspect all code paths, or trace every model assumption. Executable theories would make a different workflow possible.

A machine reviewer could search for inconsistent units, hidden assumptions, invalid theory bridges, unsupported extrapolation, unbounded use of bounded verification, parameter mismatches, stale dependencies, and discrepancies between equations and code. Another agent could deliberately seek counterexamples or alternative interpretations. A proof assistant could check formal derivations. Numerical tools could reproduce calculations under controlled environments. Statistical tools could reevaluate reported uncertainty.

This would not automate scientific judgment in the strong sense. The system might establish that a conclusion follows from the declared assumptions while a human reviewer disputes whether those assumptions are scientifically reasonable. That disagreement is valuable because the formalization has localized it. Instead of arguing over an opaque chain of reasoning, reviewers can identify the bridge or assumption under dispute.

The same architecture can support post-publication revision. A paper can be re-evaluated when a dataset is corrected or a software dependency changes. Claims whose proof objects remain valid need not be recomputed. Claims dependent on the changed artifact can be flagged. This transforms scientific literature from static documents into partially executable knowledge structures.

18.4 The relation to digital twins and scientific software

Digital twins and simulation platforms already represent aspects of executable science, but they typically encode a model rather than its full epistemic status. A simulation can compute how a model behaves without formally distinguishing which parts are theoretical assumptions, which are empirically calibrated, and which guarantees apply to the numerical method.

Outfinite mathematics could serve as a semantic envelope around such software. The simulator remains an external engine. The outfinite layer records what theory the engine

implements, what approximation and solver assumptions it uses, what evidence calibrates the model, and what conclusions are justified by a particular run.

This separation is economically attractive because it does not require replacing mature scientific software. The research programme can aim for interoperability. Existing solvers, proof assistants, model checkers, symbolic algebra systems, statistical packages, and simulation environments become evidence-producing components connected through typed interfaces.

18.5 A realistic notion of scientific machine understanding

The phrase "a computer understands a theory" can easily become vague. The programme should use an operational criterion. A machine demonstrates useful understanding of a theory when it can preserve the theory's distinctions while performing transformations that experts judge semantically valid.

Such transformations include deriving consequences, finding missing assumptions, translating between equivalent representations, identifying applicability limits, proposing counterexamples, comparing theory versions, generating experiments that discriminate between hypotheses, and explaining which evidence supports a claim. None of these requires philosophical consciousness. They require a sufficiently structured representation.

An outfinite foundation may improve machine understanding because it makes explicit some distinctions that are difficult to recover reliably from natural language. The claim remains empirical. It must be tested against strong baselines using current knowledge representations and proof systems.

18.6 An executable-science workflow from literature to revisable theory

The vision of executable science becomes credible only when it is expressed as a workflow rather than as a slogan. A realistic workflow begins with scientific material that was written for humans: papers, protocols, equations, datasets, source code, calibration descriptions, and domain conventions. The first task is not theorem proving but semantic reconstruction. The system must identify the entities discussed by the theory, the processes that connect them, the mathematical relations, the observational procedures, the idealizations, and the domain in which the claims are intended to hold.

The reconstruction should produce a theory graph rather than a flat translation. Definitions depend on other definitions, equations depend on modeling assumptions, parameters depend on measurements or prior literature, and conclusions depend on chains of deduction and estimation. The graph should preserve source spans and provenance so that every formal object can be traced back to the scientific claim that motivated it. This traceability is essential for human review and for detecting semantic drift.

The second task is typing. The system must distinguish theoretical entities from observations, computed approximations, latent parameters, processes, phenomena, and collectives. It must also distinguish exact relations from empirical fits and statistical dependencies. Many scientific errors emerge not because a local equation is wrong but because an entity changes role silently. A parameter estimated from one population becomes treated as a universal constant, or a simulated state becomes described as observed behavior. Typed existence provides a formal place to detect such transitions.

The third task is executable elaboration. Mathematical relations are connected to solvers, simulators, theorem provers, or numerical methods. Experimental claims are connected to data and preprocessing. Verification obligations are generated for the bridges. If a differential equation is solved numerically, the workflow records the discretization and error model. If a statistical model is fitted, the assumptions and finite dataset are linked to the resulting parameters. If a theorem is imported from another theory, its hypotheses and domain of validity are checked against the current context.

The fourth task is adversarial review. Rather than asking only whether the theory can execute, the system actively searches for failure. It can generate boundary cases, test unit consistency, challenge assumptions, vary parameters, compare alternative numerical representations, seek symbolic counterexamples, and check whether conclusions survive reasonable model perturbations. AI is well suited to generating such attacks because it can propose many candidate objections, while formal tools can decide which objections become genuine counterexamples or unresolved frontiers.

The fifth task is revision. Scientific theories change. An executable theory should therefore be versioned like software but with richer semantics. When a parameter definition, dataset, assumption, or bridge changes, the system should identify dependent results and classify them as preserved, invalidated, or requiring revalidation. This capability could make replication and review far more efficient because a revision would expose exactly which parts of the theory changed and what new evidence is required.

The resulting artifact is not intended to replace the paper. Human scientific communication remains necessary because theories have motivations, analogies, historical context, and interpretive arguments that resist complete formalization. The executable artifact is a parallel representation of the operational core. It provides a machine-checkable substrate for the claims that can be made precise and records the frontier where formalization stops.

This conception also clarifies what "understanding" should mean for a scientific AI. The system need not possess human-like phenomenological understanding. A useful operational criterion is that it can identify the objects and assumptions of a theory, derive authorized consequences, detect invalid transfers, execute relevant computations, explain provenance, and state the conditions under which its conclusions cease to be justified. This is a much stronger and more testable notion than fluent question answering.

The most direct validation experiment is to build executable versions of small but complete scientific theories. Candidate domains should include one deterministic physical model, one stochastic model, one computational protocol, and one biological or social process with competing levels of description. Experts can compare the executable representation with the source material and score semantic fidelity. Errors can be seeded deliberately. The system should identify incorrect units, missing hypotheses, unsupported extrapolation, numerical instability, and confusion between model objects and observations. If these tasks cannot be automated more reliably than with existing scientific software and general-purpose AI, the executable-science claim should be reduced.

18.7 Research obligations and falsifiability

Executable science should be validated on complete, small scientific theories rather than isolated equations. Candidate domains should contain a mix of theory, computation, and data. A mechanical oscillator, a reaction-kinetics model, a population model, a small

epidemiological model, and a simple control system would provide useful early cases. More demanding later cases could include quantum models, continuum mechanics, and biological networks.

For each case, independent experts should define a set of review questions whose answers are known. The executable representation should support machine answers to questions about assumptions, validity regions, numerical approximations, provenance, and consequences. Controlled defects should then be introduced: a wrong unit, invalid parameter range, changed calibration, hidden approximation, inconsistent equation, or unsupported extrapolation. Detection rate and false alarms can be measured.

A comparison with direct formalization in an existing proof assistant is essential. If the outfinite layer does not improve semantic fidelity or reviewability, then executable science may be better served by extending existing systems directly. The programme should welcome that outcome as useful negative evidence.

Chapter 19. Artificial intelligence as co-developer, user, and beneficiary

19.1 Why AI is unusually well matched to the development problem

A new mathematical foundation faces a classic bootstrapping problem. It needs libraries, examples, translations, proofs, benchmarks, and tooling before users can evaluate it, but producing those artifacts manually is expensive. AI coding agents may substantially change this constraint.

The tasks required by the programme are highly structured. Existing mathematical texts must be analyzed for definitions and dependencies. Candidate formalizations must be generated. Failed type checks and proof attempts must be repaired. Theories must be translated into solver inputs. Counterexamples must be searched. Documentation must remain synchronized with formal declarations. Benchmarks must be expanded and regression-tested. These tasks resemble the long-horizon code-generation and compiler-interaction workflows in which agentic AI is increasingly capable.

The opportunity is not to allow an AI to declare new mathematics correct. It is to use AI to explore the design space cheaply while maintaining a hard verification boundary. An agent may propose a definition of outfinite process, but the metatheorems derived from that definition must be checked. An agent may generate a proof, but the proof object must pass a trusted kernel. An agent may propose a theory bridge, but independent tests and expert review must establish that the bridge preserves intended meaning.

This division of labor can accelerate research without weakening rigor. It can also make the programme unusually transparent because every AI-generated artifact can retain provenance and review history.

19.2 Tasks that coding agents could automate

A realistic automation programme would begin with formalization support rather than autonomous discovery. Agents can extract candidate entities, assumptions, units, equations, and validity statements from papers. They can map these to a provisional ontology and ask a human to resolve ambiguous existence modes. They can generate formal definitions and translate them into Lean, Isabelle, Coq, SMT-LIB, temporal logics, or specialized analyzers. They can use compiler and solver feedback to repair syntax and proof obligations.

At the proof level, agents can search for invariants, propose induction measures, select abstract domains, generate lemmas, decompose difficult obligations, and orchestrate several verification engines. They can interpret counterexamples and propose refinements. They can produce equivalence obligations when code is transformed or a numerical method replaces an exact specification.

At the scientific level, agents can compare formal theory statements with source literature, detect semantic drift, search for omitted assumptions, and generate adversarial examples. They can maintain dependency graphs across theory versions and rerun affected analyses. They can generate explanatory prose from verified artifacts so that human readers do not need to inspect the low-level proof terms.

At the language-design level, coding agents can help discover which surface notations are natural by observing recurring patterns in formalized theories. This is a better strategy than inventing a controlled natural language prematurely. The semantic core should be stabilized through experiments; a human-friendly surface language can then be designed around demonstrated needs.

19.3 AI formalization and the problem of semantic drift

A central risk in AI-assisted formalization is that the output becomes formally valid while no longer meaning what the source meant; recent work on scientific autoformalization has characterized several forms of this semantic drift (Meadows, Zhang, and Freitas 2026). A model may replace a difficult physical concept with an easier mathematical abstraction, collapse distinct notation, strengthen or weaken an assumption, or formalize only the tractable fragment of a statement. The proof checker will correctly verify the resulting theorem, but the scientific translation will be wrong.

This problem is especially relevant to outfinite mathematics because the programme claims that typed existence and explicit bridges can reduce precisely this form of drift. An AI translating a physical statement should have to state whether an entity is theoretical, observed, measured, simulated, or computational. A reviewer can then compare that typing with the source. A translation from exact to approximate quantities must introduce an explicit bridge rather than silently change the type.

The system should also support back-translation. After formalization, a separate agent or deterministic renderer can regenerate a human-readable statement from the formal object. Experts can compare that statement with the source. Discrepancies become review targets. This round-trip process does not guarantee semantic fidelity, but it makes drift easier to detect.

19.4 AI reasoning with explicit applicability boundaries

The benefit to AI extends beyond formalization. A model reasoning over outfinite theories would have access to structured applicability information. Instead of retrieving a theorem and applying it based on semantic similarity, the agent could check whether the current context satisfies the theorem's bridge assumptions and frontier conditions.

This has implications for retrieval-augmented generation and scientific assistants. Scientific knowledge bases today often store papers, passages, formulas, and metadata. An executable outfinite knowledge base would store theory-local claims and evidence relations. Retrieval would therefore return not only a conclusion but the conditions under which it can be used.

The agent could also distinguish failure modes. If a theorem is inapplicable because a required assumption is false, the agent should not search longer for a proof. If the theorem might apply but a parameter is unknown, the correct action is to request evidence. If a symbolic analysis has reached a frontier, the agent can allocate more resources or choose another method. If the relevant theories disagree, it can present the disagreement rather than merge them.

This is a more rigorous model of tool use than asking an LLM to decide everything in natural language. It gives the agent a formal action space shaped by the semantics of the problem.

19.5 AI as a source of pressure on mathematical notation

Traditional mathematical notation is optimized for human visual compression. It uses overloaded symbols, context-sensitive conventions, typography, implicit coercions, and domain-specific shorthand. Experts navigate these conventions efficiently. Machines can parse much of them, but semantic interpretation remains difficult because the same expression can mean different operations in different structures.

Outfinite mathematics creates an opportunity to separate human presentation from machine semantics. The internal representation can be explicit, typed, and verbose. The surface representation can remain elegant and familiar. AI can mediate between the two, generating explanations and notation appropriate to the reader while preserving the verified core.

This is why the programme should not make ASCII syntax or a controlled natural language part of its philosophical identity. A surface language is an interface problem. It is important, especially for adoption, but it should be derived from the semantic foundation rather than used as evidence that the foundation exists.

19.6 AI risk: formal confidence without scientific understanding

The same automation that makes the programme feasible creates a new danger. AI can produce enormous volumes of formally checked but scientifically trivial or misdirected material. A theorem prover can validate a statement that is irrelevant, based on assumptions no scientist accepts. Formal correctness can create an illusion of epistemic strength.

The programme therefore needs explicit review layers for significance, modeling adequacy, and empirical bridge quality. These judgments may remain partly human for a long time. AI can assist by comparing alternatives and searching evidence, but the existence of a certificate should never be presented as proof that the formalized question was the right scientific question.

This limitation is not a reason to avoid formalization. It is another reason to type the status of claims. Formal validity is one property among several required for scientific usefulness.

19.7 Coding agents as experimental co-developers of the foundation

AI is not only a future consumer of outfinite mathematics. It can change the methodology by which the foundation itself is developed. Formal foundations traditionally require a large amount of painstaking engineering: definitions must be encoded, proof obligations generated, libraries organized, examples formalized, countermodels searched, and implementations kept synchronized with specifications. Coding agents can automate a significant fraction of this work, but only if the research process is designed around verification rather than trust.

One class of agent can act as a formalization engineer. Given a chapter of mathematical or scientific prose, it proposes definitions, theorem statements, existence types, and dependencies. Another can attempt alternative formalizations and search for semantic discrepancies among them. A third can generate tests and counterexamples. A fourth can search for invariants or proof decompositions. A fifth can translate certificates into a conventional proof assistant for independent checking. These agents need not be individually reliable if the workflow makes disagreement and failed verification visible.

A second class of agent can operate at the level of implementations. It can build symbolic executors, abstract domains, bridge checkers, parsers, and visualization tools from

specifications. The agent can then generate regression suites that compare the implementation against a reference semantics. Because the programme is itself about executable knowledge, its software-development process should exemplify the same principles: every generated component has an explicit contract, a test or proof boundary, and provenance.

A third class of agent can act as an adversarial scientific reviewer. It can search for assumptions that are used but not declared, identify places where a theorem changes existence type, look for missing boundary conditions, and propose experiments that would discriminate between rival theories. Formal engines can filter these suggestions. A solver-generated counterexample becomes a strong objection; an unverified verbal concern remains a candidate question. This division allows generative breadth without confusing plausibility with proof.

A fourth class can perform automated theory refactoring. As the outfinite core evolves, thousands of formalized results may need migration. Agents can propose mechanical translations, but equivalence or refinement checking should validate them. This is an area where AI and category-theoretic bridge semantics could reinforce each other: the agent proposes a functorial or representation-level translation, and the formal system checks that the relevant structure is preserved.

This development methodology could substantially reduce the cost of exploring alternative foundations. Instead of committing early to one syntax or proof calculus, researchers can ask agents to implement several candidate cores and translate a common benchmark corpus into each. The alternatives can then be compared empirically on proof burden, semantic expressiveness, automation, performance, and comprehensibility. AI makes foundational design more experimental because prototype languages and libraries can be produced and discarded more cheaply.

The danger is that AI can also create an illusion of rapid progress. A large corpus of machine-generated formal text may compile while encoding the wrong mathematical meaning. This is the semantic-drift problem in its most dangerous form: syntactic correctness can hide conceptual failure. Human experts must therefore review meaning at carefully chosen checkpoints, and independent translations should be compared. The programme should prefer smaller validated corpora over vast unreviewed formalizations.

The benefit to AI research is reciprocal. A model trained or prompted to operate in a system with explicit existence types, frontiers, proof statuses, and theory bridges may become better at practical reasoning. Instead of merely producing a theorem, it must track whether the theorem applies to the represented implementation, whether an approximation is within tolerance, and whether an empirical bridge has support. This creates a benchmarkable notion of rigor that is closer to engineering and science than formal theorem proving alone.

A strong experimental design would compare AI agents working directly with prose and conventional mathematical notation against agents using an outfinite intermediate representation. The tasks should contain traps involving overflow, incomplete search, hidden model assumptions, uncertainty types, or shifts from theoretical to empirical entities. Success would be measured not only by final-answer accuracy but by the ability to identify when a conclusion is unsupported and to generate a checkable residual obligation. If explicit outfinite structure does not improve reliability enough to compensate for the extra representation cost, the AI argument for the programme weakens substantially.

19.8 Research obligations and falsifiability

The AI hypothesis should be tested through controlled autoformalization benchmarks. Scientific and mathematical source texts should be formalized both directly into an established system and through an outfinite intermediate representation. Independent experts should score semantic fidelity, completeness of assumptions, amount of human correction, and formal validity. The outfinite approach succeeds only if it provides a measurable advantage.

Coding-agent productivity should also be measured. The programme should record the time and computational cost required to add a theory, generate proofs, repair failures, and maintain a library under change. If agents require so much supervision that the formalization cost remains prohibitive, the economic argument weakens.

Finally, the benefit to downstream AI should be evaluated separately. Agents answering scientific questions from a typed theory base should be compared with agents using text retrieval and conventional symbolic tools. Tests should focus on scope errors, model-reality conflation, asymptotic-to-finite mistakes, invalid theory transfer, and uncertainty-type confusion. These are the errors the foundation specifically claims to reduce.

Chapter 20. A future mathematical language and implementation architecture

20.1 The language is a research output, not the starting axiom

Earlier conceptual work on outfinitism experimented with a controlled natural-language mathematical notation and with names such as Outfinite Mathematical Language. Such experimentation is useful, but a credible research programme should reverse the dependency. The language should not define the mathematics. The formal semantics should define the mathematics, and the language should provide one usable surface representation of that semantics.

This distinction protects the programme from premature commitment. We do not yet know the minimal set of existence modes, the best proof calculus, the appropriate bridge semantics, or the correct relation to dependent types and category theory. A polished surface syntax designed before these questions are answered would encode accidental assumptions and make later revision expensive.

The current role of language examples is therefore illustrative. They show that the intended concepts can be expressed in a form understandable to programmers and scientists without relying on specialized typography. They should not be interpreted as a normative language specification.

Once the semantic core stabilizes, a mature interface should support several presentations. Mathematicians may prefer conventional notation. Scientists may prefer equations integrated with units and model declarations. Software engineers may prefer specification-language syntax. AI agents may prefer a normalized abstract syntax tree or graph representation. These presentations can compile to the same core if their semantics is unambiguous.

20.2 Requirements for a machine-native intermediate representation

Although the surface language is provisional, the programme can already state requirements for the internal representation. It must be typed, immutable at the level of individual values, explicit about state transitions, capable of representing theories and bridges, and suitable for independent checking.

A static single-assignment style is attractive because each computed value is produced once and state change becomes an explicit transition to a new value. This mirrors the ontological distinction between identity of a value and persistence of a process. Yet SSA should be treated as an implementation strategy, not a metaphysical axiom. Other intermediate representations may prove more suitable for higher-order mathematics or continuous models.

The representation should also carry provenance and proof obligations. When a transformation changes representation, it should identify the equivalence or approximation theorem required. When an analyzer produces a result, the result should point to a certificate or a checkable evidence object. When a frontier is reached, the residual state should be serializable so that another tool or agent can continue the analysis.

The representation should be extensible without making the trusted core large. External analyzers can operate as plugins. A symbolic executor, abstract interpreter, theorem prover,

model checker, numerical verifier, probabilistic analyzer, or AI agent may be sophisticated and independently developed. The trusted interface should specify what evidence the plugin must produce before its claim is accepted.

20.3 Small trusted kernel and certificate checking

The architecture should follow a proof-producing model wherever practical. The kernel checks a small foundational language and a family of certificate types. Powerful search procedures remain outside the trusted base.

For deductive proofs, the certificate may be a proof term. For SAT or SMT results, the solver may emit a proof trace when supported. For abstract interpretation, the analyzer can emit an invariant and enough evidence for the checker to validate initialization, preservation, and property implication. For bounded model checking, the engine can emit a counterexample or an unsatisfiability certificate for the finite encoding. For equivalence checking, it can emit a relation or proof object accepted by a specialized checker.

Not every external tool currently produces independently checkable certificates. The programme should therefore support graded trust. A result can be marked as depending on a trusted external engine when necessary, while research gradually reduces the trusted base. The crucial point is that trust is explicit.

20.4 Interoperability before replacement

A realistic implementation should integrate existing formal ecosystems. Reimplementing theorem provers, SMT solvers, numerical libraries, computer algebra, model checkers, and statistical packages would consume the programme without testing its central ideas.

The first prototype should therefore act as an orchestration and semantic layer. It should translate local theories to existing engines, collect their results, preserve frontiers, and verify certificates where possible. Established proof assistants can host metatheory and validate foundational definitions. SMT solvers can discharge decidable obligations. Model checkers can analyze finite transition systems. Numerical libraries can provide verified enclosures. AI agents can coordinate the workflow.

Success would mean that the outfinite layer adds semantic integration not already present in the tools individually. Failure would mean that users gain little beyond what a conventional build system and typed API already provide.

20.5 Human-readable rendering

Human accessibility remains essential. Formal systems fail socially when their internal precision is purchased at the cost of making review impossible for domain experts. The implementation should therefore generate explanations, equations, dependency diagrams, and proof summaries from the verified core.

AI can help here without entering the trusted base. A language model can explain a certificate in domain-specific prose, summarize which assumptions matter, or translate a low-level counterexample into an intelligible scenario. Because the underlying result is structured, the explanation can be checked against the artifact rather than generated from scratch.

This architecture also supports education. Students and researchers can see the same theorem at several levels: intuitive explanation, conventional mathematics, formal

dependencies, and executable certificate. Outfinite mathematics could therefore become usable by humans even if the low-level verification is impossible to perform mentally.

20.6 A semantic intermediate representation before a mathematical controlled language

The programme is likely to need a human-readable mathematical language, but that language should be a later engineering consequence of the semantic core. The first implementation target should be a machine-native intermediate representation capable of preserving the distinctions that the research programme claims are foundational. The representation should be canonical enough for independent tools to exchange theories and flexible enough to host several mathematical and logical sublanguages.

A useful intermediate representation would treat theories as versioned modules with explicit imports and bridges. Within a theory, definitions, claims, processes, evidence, and transformations would be represented as typed nodes in a dependency graph or an SSA-like circuit. State changes would create new values rather than silently mutating old ones. This is not because SSA is philosophically fundamental, but because immutable dataflow makes provenance, dependency, and transformation easier to audit.

The representation should separate the meaning of a claim from the engine used to analyze it. A safety property does not become a symbolic-execution property merely because a symbolic executor is called. The same claim may be attacked by deductive verification, abstract interpretation, bounded model checking, and testing. Their outputs should attach distinct evidence objects to the common claim. This decoupling is essential for a modular ecosystem.

The trusted kernel should validate a small family of primitive certificate relations. More complex engines can remain untrusted. A theorem prover may export a proof term or a proof trace. A SAT or SMT solver may export a certificate of unsatisfiability or a model. An abstract interpreter may export an invariant and evidence that it is inductive. A symbolic executor may export a path trace and a concrete witness. A numerical engine may export an interval enclosure. A scientific data pipeline may export provenance and calibration evidence. The long-term research problem is to determine how small this family can remain.

Interoperability should precede replacement. The system should translate to and from existing proof assistants, SMT languages, numerical libraries, probabilistic languages, and scientific data formats. If a claim can already be proved effectively in Lean or Isabelle, the outfinite layer should reuse that capability rather than reimplement theorem proving. If an SMT solver can discharge an arithmetic obligation, the kernel should validate the resulting certificate or rely on a well-defined trusted interface. The new contribution lies in orchestration and semantic typing across tools.

A controlled natural-language mathematical notation can then be built on top. Its purpose would be readability and AI interaction. It should allow a scientist to state a process, assumption, theorem, evidence relation, or frontier in prose-like form while remaining deterministically elaboratable into the intermediate representation. The earlier experimental syntax is useful as an intuition, but it should not be frozen before the semantic design is validated. Different surface notations may coexist for different communities.

The elaboration step is a natural place for AI assistance. A language model can infer omitted types, propose theory imports, identify likely bridges, and translate conventional notation into the core. However, every inferred element must remain visible for review. An AI should

never silently invent an assumption merely to make a theorem type-check. Ambiguity should produce an explicit unresolved obligation when it cannot be resolved from context.

Human-readable rendering is equally important. A researcher should not be forced to inspect low-level certificate graphs. The system should generate conventional mathematical exposition, dependency diagrams, boundary summaries, and explanations of why a theorem applies. This creates a reversible workflow: human prose is elaborated into formal structure, machines analyze the structure, and the results are rendered back into human mathematical language with provenance.

The implementation can be tested before the full foundation is settled. Several candidate intermediate representations can encode the same benchmark theories. Researchers can compare translation complexity, certificate size, incremental recompilation, support for multiple logics, and resistance to semantic drift. A design should be rejected if it becomes a monolithic language that must natively understand every domain. The architectural goal is a small semantic spine supporting extensible theories and evidence engines.

20.7 Research obligations and falsifiability

The language and implementation should be developed through competing prototypes rather than one monolithic design. A direct embedding in Lean, a standalone typed intermediate representation, and a graph-based representation should be compared on the same corpus. The winning architecture should be determined by semantic clarity, interoperability, certificate size, performance, and ease of authoring.

The claim that a controlled natural-language interface is beneficial should be tested only after the core is stable. Domain experts should author the same theory through conventional mathematical notation, a structured editor, and a controlled-language interface. Error rate, authoring time, readability, and AI autoformalization performance can be measured. If controlled language does not help, it should not be imposed.

The trusted-kernel architecture is also testable. Independent checkers should validate the same certificates. Adversarial malformed certificates and corrupted provenance should be rejected. The system should document the exact trusted computing base for every accepted theorem.

Part VI. Validation, scientific trajectory, and potential impact

Chapter 21. A falsifiable research programme

21.1 The programme must be able to fail

A foundational proposal is easy to protect from criticism by moving every objection into philosophy. That would make outfinitism scientifically unproductive. The programme should instead commit to concrete hypotheses whose failure changes the theory.

The first hypothesis is semantic: typed existence should prevent or expose invalid inferences that arise when theoretical, computational, physical, processual, and collective existence are conflated. The second is formal: outfinite processes and frontier-sensitive theorems should support a useful calculus not reducible to trivial bookkeeping. The third is computational: heterogeneous proof and analysis methods should compose more safely when their coverage and frontier are typed. The fourth is scientific: executable theories built on this foundation should preserve source meaning and applicability conditions better than strong alternatives. The fifth is economic: AI-assisted construction and review should reduce the cost of creating and maintaining such formal theories enough to justify their use.

Each of these hypotheses can fail independently. The programme should publish negative results rather than redefine success after the fact.

21.2 Benchmark families

Validation should use several benchmark families because no single domain tests the whole foundation. Pure mathematics tests conservativity and proof expressiveness. Software verification tests frontiers and method composition. Numerical science tests bridges between ideal and finite representation. Empirical science tests evidence and model validity. Collective systems test non-static identity. AI formalization tests usability and semantic preservation.

The mathematical benchmark should include arithmetic, algebra, elementary analysis, probability, and a selection of theorems whose standard proofs use infinite structures. The goal is not to compete with mature libraries by size. It is to identify where outfinite reformulation is straightforward, where it adds useful structure, and where it becomes unnatural.

The software benchmark should contain fixed-width arithmetic, loops, recursive processes, finite-state protocols, information flow, and numerical code. Deliberate defects should appear just beyond bounded verification thresholds so that frontier handling can be tested. Equivalent and non-equivalent program transformations should test bridge certificates.

The scientific benchmark should consist of complete small models with equations, units, parameters, numerical methods, and data. The benchmark should include known validity boundaries and controlled model mismatch. This is where executable science either becomes real or remains a metaphor.

The collective benchmark should include a distributed software service, a simulated swarm, and an institutional process. It should test persistence under member change and the separation of legal, functional, and processual existence.

21.3 Comparative baselines

Every benchmark needs strong baselines. Dependent type theories and proof assistants already support rich formalization. Specification languages can express preconditions and effects. Scientific workflow systems track provenance. Knowledge graphs represent typed relations. Model checkers and abstract interpreters already expose some bounds. Numerical verification provides certified errors.

The relevant question is therefore not whether outfinitism can perform a task at all. It is whether the integrated foundation makes the task more semantically faithful, compositional, auditable, or economical.

Comparisons should avoid straw men. A baseline should be configured by experts in the competing method. If a direct Lean formalization, for example, can represent existence modes elegantly through ordinary types and type classes, the outfinite programme must show what additional structure it contributes. If a knowledge graph handles scientific provenance better, the programme should integrate it rather than duplicate it.

21.4 Quantitative and qualitative evaluation

Several outcomes are measurable. Formal validity is binary when checked by a trusted system. Semantic fidelity can be rated independently by domain experts. Hidden assumptions can be counted. Review time can be measured. Proof and certificate size can be recorded. The rate of detecting seeded defects can be compared. The cost of updating a theory after a changed assumption can be measured.

AI experiments add further metrics: compilation success, percentage of source assumptions recovered, number of semantic corrections required, proof-search cost, and performance on questions requiring correct scope and existence typing.

Qualitative evaluation is also necessary. Experts can judge whether the formal ontology matches scientific practice or forces artificial distinctions. They can identify where theory bridges hide rather than expose modeling judgment. These criticisms should feed directly into revisions of the core.

21.5 Criteria for abandoning or narrowing claims

The programme should define in advance what would count as failure. If existence typing remains subjective and cannot be given stable formal rules, it should be treated as documentation rather than foundational logic. If outfinite process theory produces no useful metatheorems beyond ordinary transition systems with resource parameters, the new terminology should be reduced. If category theory and multi-logic architecture increase complexity without improving composition, they should remain optional libraries.

If executable science can be achieved just as effectively by modest extensions to existing proof assistants and scientific workflow tools, the correct outcome may be an integration project rather than a new mathematics. That would still be valuable, but it would change the philosophical claim.

Most importantly, if AI systems do not reason more reliably about applicability when given the additional structure, the argument that outfinitism is especially important for AI would need to be reconsidered.

21.6 Benchmark governance, preregistered failure, and evidence for the programme itself

A research programme about rigor should apply the same rigor to its own evaluation. The strongest way to avoid retrospective success criteria is to define benchmark families and failure thresholds before the implementation is mature enough to optimize for them. The programme should publish not only demonstrations but also negative results and cases in which existing tools outperform the outfinite approach.

The benchmarks need several layers. Foundational benchmarks should test whether the core semantics prevents illicit inference among existence types and whether theory bridges compose correctly. Mathematical benchmarks should reconstruct established results from arithmetic, algebra, analysis, probability, and process theory. Formal-method benchmarks should compare proof obligations and certificate composition across deductive verification, abstract interpretation, symbolic execution, model checking, and transformation validation. Scientific benchmarks should include real model-data workflows in which applicability boundaries and provenance matter. AI benchmarks should test semantic fidelity and refusal to overclaim.

Each benchmark should include adversarial variants. A safe fixed-width arithmetic example can be modified so that overflow occurs just outside the tested range. A bounded model-checking task can hide a counterexample one step beyond the chosen horizon. A numerical model can cross a stability boundary. A statistical example can violate independence while retaining plausible summary statistics. A collective model can preserve legal identity while losing functional coordination. The evaluation asks whether the representation preserves the boundary rather than merely whether the solver finds a correct answer on the nominal case.

Comparisons must use serious baselines. Dependent type theory, Lean or Isabelle formalization, SMT-based specification, model checking, scientific workflow systems, provenance graphs, probabilistic programming, and ordinary typed programming languages can each cover part of the target. The correct question is not whether outfinite mathematics can do something that no existing tool can do in principle. It is whether the integrated semantics reduces errors, integration cost, or semantic ambiguity enough to justify a distinct research direction.

Quantitative metrics should include authoring effort, proof and certificate size, translation overhead, solver performance, number of assumptions made explicit, precision and recall on seeded semantic errors, rate of invalid theorem transfer, and success of incremental revalidation after changes. Qualitative review by domain experts is also necessary because semantic fidelity cannot be reduced entirely to compilation success. Expert raters should compare the formal artifact with the source theory and identify lost or invented meaning.

The programme should preregister conditions under which claims are narrowed. If typed existence does not reduce semantic errors, it may remain philosophical rather than foundational. If frontier tracking is useful only for software verification, the broader mathematical claim should be abandoned. If a new core logic offers no advantage over an existing modal dependent type theory, the project should adopt the existing theory. If the controlled language creates more ambiguity than conventional formal syntax, it should be removed. If executable-science artifacts are too expensive to maintain, the target should shift to narrower high-value domains.

Replication should be encouraged through a small trusted reference implementation and public benchmark corpus. Independent teams should be able to implement the semantics and check the same certificates. Disagreement between implementations is itself valuable evidence that the specification is underspecified. The research programme becomes credible when outsiders can construct counterexamples to its claims rather than having to accept demonstrations produced by the same system that defines success.

This evaluation discipline also helps with funding and adoption because it converts a broad philosophical vision into staged technical risk. Early phases can fail cheaply. The programme does not need to prove that a new mathematics will become mainstream. It needs to show, one falsifiable step at a time, that its semantic distinctions produce measurable value. Positive results can justify deeper reconstruction; negative results can eliminate unnecessary components before they become institutional commitments.

21.7 Research obligations and falsifiability

This entire chapter is the falsification protocol for the programme, but one further requirement is important: benchmarks and evaluation criteria should be published before the architecture is tuned to them. Otherwise a flexible foundational language can always be adjusted after the fact to appear successful.

A public corpus of adversarial examples should accompany the implementation. External researchers should be encouraged to construct cases in which existence typing is ambiguous, frontiers compose badly, or theory bridges become circular. A foundation that survives only friendly examples has not yet earned credibility.

Chapter 22. Scientific and economic trajectories if the programme succeeds

22.1 The scientific contribution

If the programme succeeds at a modest level, it would provide a clearer formal language for several distinctions already known to specialists. That alone could be useful. If it succeeds at a deeper level, it could create a new research area at the intersection of mathematical foundations, formal methods, philosophy of science, programming languages, and scientific AI.

The strongest scientific contribution would not be the slogan that infinity is problematic. Mathematics has debated infinity for millennia. The deeper contribution would be a formal theory in which existence, applicability, observation, approximation, computation, and proof are typed and composable. The outfinite concept would then serve as one important case within a broader mathematics of boundaries.

Such a foundation could also create new mathematical questions. What structures classify theory bridges? How do frontiers compose? When is an outfinite process equivalent to a classical infinite object for a given family of observations? What conservation theorems hold when resource contexts are extended? Which existence modes admit constructive witnesses? How do approximate categorical translations compose? What algebra governs evidence and residual uncertainty? These questions are interesting independently of any immediate industrial application.

The programme may also sharpen philosophy of science. Instead of discussing idealization, model pluralism, and theory-ladenness only in prose, some of these relations could become executable formal objects. Meta-rationality would acquire a technical counterpart: disciplined movement among theories with explicit preservation and loss.

22.2 The economic contribution

Economic value would arise if formal scientific knowledge becomes reusable infrastructure. Today, much scientific and engineering reasoning is repeatedly reconstructed from documents. Assumptions are rediscovered, code is rewritten, validations are repeated, and AI systems reread the same prose without a shared formal semantic layer.

Executable theories could become assets analogous to software libraries. A validated model could be imported, specialized, connected to new data, and reviewed automatically. A regulatory or safety argument could reuse machine-checkable evidence. A research organization could maintain theory versions with dependency tracking. An AI assistant could operate over the formal knowledge base rather than improvising the meaning of every equation from text.

The strongest business case is likely in domains where wrong applicability is expensive. Safety-critical engineering, cybersecurity, pharmaceutical and biomedical modeling, industrial simulation, financial risk, energy systems, and regulated AI all contain expensive review processes. Outfinite infrastructure would be relevant only if it can reduce that cost or increase assurance without making modeling prohibitively burdensome.

A second economic opportunity lies in tooling. A mature ecosystem could include autoformalization agents, theory repositories, certificate verifiers, model-review services, translators to formal backends, executable scientific notebooks, and domain-specific libraries. The defensible asset would be the combination of formal semantics, curated theory libraries, interoperability, and verification workflows rather than a proprietary syntax alone.

A third opportunity concerns AI development itself. Training and evaluating AI on theory-local, machine-checkable scientific artifacts could provide richer supervision than natural-language answers. Models could learn not only whether a conclusion is correct but why it is applicable, which assumption failed, and where the frontier lies. This may create new benchmarks and training signals for rigorous scientific reasoning.

22.3 Adoption strategy as a scientific constraint

A mathematically elegant foundation that requires users to abandon existing tools is unlikely to succeed. Adoption should therefore begin where the programme offers clear incremental value. The initial system can sit above existing theorem provers, solvers, analyzers, and simulators. Users can benefit from typed bridges and evidence without rewriting their entire stack.

The first human interface should probably resemble ordinary scientific and mathematical writing rather than a new programming language. AI can help translate between human notation and the formal core. Over time, recurring patterns may justify a dedicated language or structured editor.

The same incremental strategy applies to mathematics. Outfinite theories should import classical results wherever appropriate. A researcher should not have to reprove elementary algebra merely because the result was originally developed inside set theory. The foundation gains credibility by interoperating with established mathematics, not by isolating itself.

22.4 Long-term possibility: a machine-native layer for science

The most ambitious outcome is a common intermediate layer for scientific theories, somewhat analogous to the role an intermediate representation plays in compilers. High-level scientific languages, mathematical notation, domain-specific models, and AI-generated representations could compile into a typed semantic core. Verification, simulation, optimization, and inference engines could operate on or translate from that core. Results could then be rendered back into human scientific language with their assumptions intact.

This analogy should not be taken too literally. Scientific theories are not programs, and scientific meaning is not reducible to compiler semantics. Yet the analogy captures an important economic idea: interoperability becomes possible when multiple front ends and back ends agree on a stable intermediate contract.

If outfinitism can provide such a contract, its significance would be much greater than the original critique of infinity. It would become part of the infrastructure through which human and artificial researchers exchange rigorous knowledge.

22.5 Economic pathways and defensible value if the research succeeds

The economic significance of the programme depends less on ownership of a mathematical notation than on whether it creates a reliable infrastructure for machine-mediated scientific

reasoning. Formal languages are difficult to monetize in isolation, and foundational ideas should remain open enough to support scientific scrutiny. The valuable assets are more likely to be the ecosystem of executable theories, validated bridges, domain libraries, review workflows, and high-assurance AI agents that operate on top of the foundation.

One pathway is scientific software assurance. Research organizations increasingly depend on complex computational pipelines in which mathematical models, data processing, statistics, simulation, and machine learning interact. Errors often occur at boundaries between components rather than inside a single algorithm. A theory representation that can track assumptions and validity across these boundaries could support automated review, regulatory evidence, and safer reuse of models.

A second pathway is AI for research and engineering. General-purpose models are strong at generating candidate code and explanations but weak at reliably determining where a result applies. An outfinite semantic layer could support AI systems whose outputs include checkable applicability conditions and residual uncertainty. Such systems would be valuable in domains where an apparently reasonable but inapplicable answer is costly, including drug development, safety-critical engineering, cybersecurity, advanced manufacturing, and high-stakes scientific analysis.

A third pathway is reusable executable knowledge. Scientific models are currently distributed as papers, notebooks, source code, datasets, and informal conventions. A mature executable-theory format could become a new class of research asset. Organizations could maintain validated model libraries in which theory versions, evidence, simulation code, and proof certificates remain linked. Reusing such an artifact would be safer than copying equations from a publication and reconstructing hidden assumptions manually.

A fourth pathway is interoperability. The formal-methods ecosystem is powerful but fragmented. Proof assistants, SMT solvers, static analyzers, model checkers, probabilistic systems, numerical libraries, and scientific workflow platforms each use different representations. A semantic bridge layer that allows their evidence to compose could reduce integration cost. The defensible technical value would lie in verified adapters and theory translations rather than in trying to replace every tool.

The largest long-term opportunity would arise if an outfinite intermediate representation became a common machine-readable layer for scientific theories. Such a layer could support search over theorem applicability rather than merely keyword similarity, automated impact analysis when assumptions change, machine-generated replication plans, and training data in which reasoning steps carry formal provenance. The network effect would come from the corpus: each formalized theory and validated bridge increases the value of the infrastructure for later theories.

These opportunities should not be used to inflate the scientific claims. A programme can be scientifically important without becoming a commercial platform, and a useful software product does not prove that the underlying philosophy is the correct foundation of mathematics. The research should therefore separate theoretical milestones from economic hypotheses. Commercial relevance is evidence that explicit semantic boundaries solve real problems, not a substitute for mathematical validation.

The most credible adoption strategy is incremental. The programme should first solve narrow problems where boundary errors are expensive and formal methods already have institutional acceptance. Successful components can then generalize. If users gain value from

a proof-evidence interchange layer, a theory provenance system, or an AI-assisted formalization workflow, those components can be adopted without requiring agreement on the entire philosophy of outfinitism. Mainstream influence would emerge through utility and interoperability rather than foundational conversion.

22.6 Research obligations and falsifiability

The economic thesis should be tested in realistic workflows rather than inferred from conceptual elegance. Pilot projects should measure the cost of formalizing, reviewing, updating, and reusing models. A successful pilot should show that a verified theory artifact can be reused across more than one tool or project without expensive manual reconstruction.

The AI-training thesis also requires evidence. Models trained or prompted with typed theory artifacts should outperform comparable systems on applicability-sensitive reasoning tasks. If the gain is marginal, the infrastructure may still be useful for assurance but not for general AI reasoning.

Finally, adoption itself is informative. If domain experts consistently reject the formalism as too intrusive, the surface architecture or even the foundational distinctions may be wrong. Usability is not separate from scientific validity when the research goal is a working language for science.

Chapter 23. Integrated exemplars for testing the research programme

23.1 One symbol, several arithmetics

A small arithmetic example reveals why the programme insists on explicit algebra and representation. Consider an expression that visually uses ordinary addition. In pure integer arithmetic, addition is exact, associative, and unbounded in the mathematical model. In a fixed-width implementation with wraparound, the same written operation is addition modulo a machine-dependent power of two. In a trapping implementation, the operation is partial because overflow terminates the computation or raises an exception. In floating-point arithmetic, rounding means that associativity can fail even when no exceptional value occurs.

The underlying engineering fact is familiar. The research question is whether a common mathematical foundation can express it without forcing users to treat every implementation as an unrelated language. An outfinite theory would retain a common structural idea of an additive operation while attaching it to a declared algebra and a representation bridge. A theorem proved from ring associativity can then be transported only to realizations whose bridge preserves the relevant law or to approximate realizations with an explicit error theorem.

This is an excellent benchmark for AI. A coding agent can be given a symbolic derivation and asked to optimize a numerical implementation. The baseline agent may apply exact algebraic transformations without checking the arithmetic semantics. The typed agent must establish that the target representation validates the transformation. The experiment is easy to automate and produces objective counterexamples when the agent is wrong.

The example also tests the distinction between theoretical and computational existence. A mathematical integer resulting from an operation can exist in the ideal theory even when it cannot be represented in the selected machine type. The computational frontier is not a contradiction in arithmetic. It is a failure of the realization bridge for that input and representation.

Research obligations and falsifiability

The experiment should cover wraparound integers, checked integers, arbitrary-precision integers, floating point, and interval arithmetic. A useful outfinite layer must reuse high-level structure while correctly separating laws that differ. If a conventional strongly typed programming language plus existing numeric libraries already prevents the same errors with equal clarity, this example provides no evidence for a new foundation; it instead identifies an existing mechanism that the programme should adopt.

23.2 A loop whose safety requires complementary methods

A second exemplar should combine several proof techniques rather than asking one technique to solve everything. Consider a loop that reads input, updates an index, accesses a buffer, and accumulates a numerical result. The safety questions include whether the index remains inside the buffer, whether an arithmetic accumulator can overflow, whether tainted

input can reach a sensitive operation, and whether the loop terminates under its declared conditions.

Deductive verification can express preconditions, postconditions, and an invariant relating the index to the buffer length. Abstract interpretation can infer numerical intervals and prove that certain runtime errors are impossible. Taint analysis can track whether external input reaches a sensitive sink. Symbolic execution can search unusual branches and produce concrete failing inputs. Bounded model checking can exhaustively analyze a finite abstraction up to a chosen depth. A dependent type can encode some shape or length constraints statically.

A conventional engineering workflow might run these tools separately and produce several reports. The outfinite hypothesis is that a typed evidence layer can compose them. The index invariant is a deductive or abstract certificate. The absence of overflow may be global under a sound interval abstraction or only bounded if established by exploration. A symbolic counterexample is a witness. A taint result is meaningful only relative to the modeled flow semantics. If one analysis stops because of path explosion, the unexplored region remains visible.

The example should also test theory bridges. The mathematical buffer model may assume an abstract array, while the implementation includes pointer arithmetic or language-specific undefined behavior. A proof of the abstract model does not automatically certify the concrete program. Translation validation or a verified semantics must justify the bridge.

Research obligations and falsifiability

The benchmark should contain seeded defects deliberately positioned outside shallow search bounds, defects visible only under aliasing or information flow, and defects caused by arithmetic representation. The common evidence layer succeeds only if it preserves which method established which property and refuses to turn partial exploration into global proof. If the integration adds no more protection than a conventional CI pipeline collecting tool reports, the claim of a unified proof calculus is not supported.

23.3 A retrying protocol and observational equivalence

A third exemplar concerns process identity. A simple service specification may state that a request results in either a valid answer or an explicit unavailability outcome. A realistic implementation may attempt an operation, wait, retry, and eventually return one of the specified observable outcomes. The internal states of the implementation have no direct counterpart in the abstract specification.

Literal state equality is therefore the wrong notion of equivalence. The appropriate relation ignores internal retry and backoff steps while requiring externally visible behavior to refine the specification. Coalgebraic bisimulation, weak simulation, trace refinement, or another behavioral relation can express the contract. Category theory may help organize the observation map that forgets internal events.

This example also separates safety from liveness. Safety can state that every emitted answer satisfies the content specification. It may be provable even if the service can retry forever. Liveness states that the process eventually produces an answer or frontier. That property requires assumptions about retry limits, fairness, timeouts, or eventual environmental

response. The absence of a logical terminal condition must not be mistaken for a guarantee of progress.

Bounded model checking can analyze finite retry limits and queue sizes. If a completeness threshold is known, it can produce a global result for the model. Abstract interpretation can summarize retry counts and state classes. Symbolic execution can construct failure sequences that lead to an unavailability frontier. A temporal logic can express safety and progress distinctly.

Research obligations and falsifiability

The protocol should be implemented in at least two semantically different runtimes, one with a fixed retry limit and one with environment-dependent termination. The formal theory must state exactly where the liveness proof changes. If the framework cannot express the difference without ad hoc annotations, its process semantics is insufficient. If categorical machinery provides no reuse beyond ordinary refinement checking, it should not be made foundational for this class of examples.

23.4 A collective whose legal and functional existence diverge

The final exemplar returns to the philosophical core. Consider a research consortium. It may exist legally because an agreement is in force, administratively because it appears in registries, physically because member organizations and people exist, processually because activities occur, and collectively because coordination, shared memory, and decision capacity persist.

These modes can diverge. A consortium may remain legally registered after practical collaboration has ceased. A partner may remain a formal member while contributing nothing. A smaller subset may preserve the effective collective even while nominal membership is larger. A single Boolean predicate of existence hides these distinctions.

An outfinite theory can represent the legal status as one theory, the activity trace as another, and the functional collective through a persistence criterion. Some aspects of membership may be graded. Evidence can be contradictory: reports may claim active participation while observed traces show none. A paraconsistent evidence layer can preserve both claims without deciding the entire collective inconsistently. A fuzzy model can describe graded participation without pretending that the grade is a probability. Graph-theoretic robustness can state how many member failures the coordination network tolerates.

The point is not to prove a metaphysical essence of the consortium. It is to make explicit which theory answers which question. A legal conclusion follows from legal evidence. A functional conclusion follows from an operational definition and evidence. A bridge may relate them, but the bridge is itself contestable.

Research obligations and falsifiability

This exemplar should be evaluated by domain experts who are not involved in designing the formalism. They should judge whether the proposed existence modes and persistence criteria clarify real distinctions or merely formalize arbitrary thresholds. If expert disagreement cannot be localized to explicit definitions and evidence assumptions, the collective ontology has failed to deliver meta-rational transparency.

23.5 What the integrated exemplars must prove about the programme

The examples in this chapter are not illustrations added after the theory. They should become the first serious experimental objects of the research programme. Each is deliberately chosen because an ordinary mathematical treatment is already possible. The outfinite approach earns credibility only if it reveals a distinction that matters in practice, supports a useful composition of methods, or prevents an error that a less explicit representation makes easy.

The arithmetic exemplar should compare at least three meanings of addition: exact integer arithmetic, fixed-width modular arithmetic, and floating-point or approximate arithmetic. A theorem that is valid over mathematical integers should be transported into each implementation only under an explicit relation. The fixed-width case requires a range condition if integer semantics is intended; otherwise the correct structure is modular. The floating-point case requires a numerical-error account. The experiment should measure whether the outfinite type and bridge discipline prevents an AI or automated prover from importing algebraic identities into a representation where they fail.

The loop-safety exemplar should force several proof technologies to cooperate. A deductive verifier can generate verification conditions, an abstract interpreter can discover or check a range invariant, symbolic execution can search for concrete violations, and bounded model checking can close a finite horizon. The benchmark should include a version where a bounded search finds no counterexample but a later iteration fails. The correct system must preserve the bounded result without promoting it to global safety. A second version should allow an inductive invariant to close the proof globally. This demonstrates that the framework distinguishes frontier from theorem and can record the additional witness that lifts the result.

The retrying-protocol exemplar tests observational equivalence and process identity. Two implementations may differ in internal retries, timeouts, or message ordering while exposing the same externally relevant behavior under a stated environment model. The task is not to prove syntactic equality but to choose the correct equivalence relation. The benchmark should then perturb the environment so that the equivalence fails, showing that the theorem is indexed by assumptions and observation. This is a direct test of theory-local identity and category-theoretic bridge ideas.

The collective-existence exemplar tests the most unconventional part of the programme. A simulated organization or distributed system can be given legal, structural, and functional views. Member turnover can occur while some views preserve identity and others do not. The system should state, for each view, which invariant supports persistence and where the frontier lies. A deliberately designed scenario can preserve formal registration while destroying coordinated function. The expected result is not a contradiction but two differently typed existence claims. If the framework cannot represent this more clearly than ordinary predicates, the collective ontology has not earned its complexity.

A fifth exemplar should be scientific rather than purely mathematical or computational. A simple dynamical model can be connected to synthetic measurements and a numerical solver. The exact theory, numerical approximation, calibration model, and observations should remain distinct. The benchmark can then introduce a parameter outside the calibrated regime, a solver step size that violates stability, or a unit mismatch. An AI reviewing only the equations may produce a plausible conclusion; the executable theory should identify the invalid bridge or frontier.

A sixth exemplar should focus on statistics. The same numerical quantity can be presented once as a model probability, once as an observed frequency, once as a confidence level, and once as a fuzzy membership score. The system should reject inferences that treat these as interchangeable. This benchmark is valuable because language models often exploit surface numerical similarity. Typed uncertainty should provide a measurable advantage if the foundational thesis is correct.

For every exemplar, the evaluation should compare three representations. The first is conventional prose plus ordinary mathematics. The second uses an established formal system or domain tool without the outfinite layer. The third uses the proposed typed existence, frontier, and evidence semantics. Human experts and AI agents should solve the same questions in each representation. Metrics should include semantic errors, unsupported theorem transport, time to formalize, proof burden, and quality of explanations.

The examples should also test incremental change. A theory is modified after its first verification: a numeric representation changes, a bound is extended, a model assumption is weakened, or new evidence is added. The system should identify which conclusions survive and which must be revalidated. This property is central to executable science because real theories evolve. A framework that verifies static snapshots but cannot manage revision will have limited practical value.

Finally, the exemplars should be public and adversarially extensible. External researchers should be encouraged to add cases designed to break the foundation, including ambiguous existence claims, pathological frontier interactions, conflicting evidence, and transformations that appear structure-preserving but are not. The programme will become more credible when its examples function as challenge problems rather than demonstrations selected only because the proposed system handles them well.

Chapter 24. A staged research trajectory

24.1 Phase one: semantic minimalism

The first phase should resist the temptation to build a large language or library. Its objective is to state a minimal formal semantics for existence modes, theory locality, bridges, outfinite processes, frontiers, and typed evidence. At least two alternative encodings should be explored in existing proof assistants so that the research does not prematurely confuse one implementation with the concept itself.

The deliverables of this phase are mathematical rather than cosmetic. The programme needs definitions precise enough to support metatheorems about bridge composition, preservation under frontier extension, conservativity of imported classical fragments, and separation of theory-local proof from empirical evidence. A small corpus of arithmetic and transition-system examples should be sufficient to expose early contradictions.

AI can assist with formal proof search and library construction, but human review should remain intensive because the semantics itself is being designed. Every proposed axiom should be accompanied by examples showing what it permits and what it forbids.

Research obligations and falsifiability

The phase fails if independent formalizers cannot reproduce the semantics or if the central distinctions repeatedly reduce to labels with no inferential consequences. A successful phase ends with a kernel-level formal model, not merely a persuasive vocabulary.

24.2 Phase two: proof engines and evidence interoperability

Once the semantic core is stable enough, the second phase integrates heterogeneous analysis engines. The objective is not broad tool coverage but a demonstration that deductive proofs, abstract invariants, symbolic counterexamples, bounded model-checking results, taint or dataflow facts, and equivalence certificates can inhabit one typed evidence architecture.

Existing tools should be wrapped rather than rewritten. Where they emit certificates, the prototype should check them independently. Where they do not, the result should explicitly enlarge the trusted base. The system should preserve coverage and frontier information across tool boundaries.

Coding agents can become substantially more autonomous in this phase. They can choose tools, generate translations, propose invariants, and repair failed proofs. The kernel remains independent of agent behavior.

Research obligations and falsifiability

The phase fails if composition requires manual interpretation of every tool report or if semantic information is lost during translation. The principal benchmark is whether one tool can safely consume the result of another without human reconstruction of its hidden assumptions.

24.3 Phase three: mathematical reconstruction

The third phase expands the mathematical library. Arithmetic and algebra come first because their semantics is well understood. Analysis, probability, topology, and dynamical systems follow because they test the relationship between ideal objects and finite information. Category theory and non-classical logics should be developed only where concrete benchmark demands justify them.

This phase should deliberately include theorems that stress the programme's limits. Nonconstructive existence, compactness, infinite-dimensional structures, diagonal arguments, and asymptotic probability are not inconveniences to be avoided. They determine whether outfinitism is a broad foundation, a specialized applied layer, or a family of interoperable subtheories.

A crucial deliverable is bidirectional translation with established formal libraries. Outfinite mathematics should import proven results rather than rederive the world from scratch, and important outfinite theorems should be exportable for independent checking.

Research obligations and falsifiability

The phase fails as a foundational project if the outfinite layer cannot recover a substantial mathematical corpus without either silently reintroducing its rejected assumptions or becoming unusably verbose. The correct scientific response may then be to narrow the project to executable applied mathematics.

24.4 Phase four: executable scientific theories

The fourth phase moves from mathematics to science. Small but complete models should be formalized together with data, numerical methods, units, assumptions, and empirical bridges. The aim is to demonstrate machine-supported review, not only theorem proving.

This phase requires collaboration with domain scientists because modeling adequacy cannot be determined by logicians alone. The selected theories should contain known approximation regimes and documented failure boundaries so that the formal representation can be tested against reality.

AI agents can perform literature extraction, theory assembly, proof generation, and regression maintenance. Independent review agents can compare the formal theory with the source and search for semantic drift. Human experts decide whether the scientific bridge is faithful.

Research obligations and falsifiability

The phase fails if formalization captures equations but not the assumptions and evidence that determine their use, or if machine review cannot detect seeded scientific errors better than existing tooling. This phase is the decisive test of the executable-science hypothesis.

24.5 Phase five: AI-native theory infrastructure

Only after executable theories are demonstrated should the programme investigate a dedicated human- and AI-facing language at scale. By then, actual usage data will reveal recurring concepts, authoring bottlenecks, and patterns suitable for automation.

The surface language may combine ordinary mathematical notation, controlled prose, structured declarations, and graphical views. AI can translate between these forms and the semantic core. The system can support collaborative authoring in which an agent proposes a formalization and a scientist approves the meaning before low-level proof search begins.

A theory repository can then support versioning, dependency management, certified transformations, and reusable domain modules. This is the point at which economic infrastructure becomes plausible rather than speculative.

Research obligations and falsifiability

The phase fails if domain experts cannot author or review theories efficiently, if AI translation introduces unacceptable semantic drift, or if the ecosystem remains dependent on a small number of specialists. Adoption is part of the scientific test because the programme claims to provide a practical bridge between mathematical rigor and scientific work.

24.6 Governance of a foundational research programme

A final methodological requirement concerns governance. Foundational projects are vulnerable to self-confirmation because the vocabulary evolves with the objections. The programme should therefore maintain public definitions, versioned hypotheses, benchmark suites, and recorded negative results. Major changes to the meaning of outfinite, existence mode, frontier, or evidence should be documented as theory revisions rather than silently absorbed.

Independent implementation is especially important. A second team should be able to implement the kernel and reproduce benchmark results from the specification. External researchers should be encouraged to construct counterexamples and alternative encodings. The programme gains credibility when criticism produces sharper definitions rather than defensive rhetoric.

Research obligations and falsifiability

Governance itself can be evaluated by reproducibility. If benchmark results cannot be replicated from public specifications, the programme has not achieved the transparency it asks of science. If definitions change so frequently that earlier results cannot be interpreted, the foundational layer is not yet mature enough for downstream claims.

24.7 From philosophical thesis to a sequence of increasingly expensive commitments

The staged trajectory should be interpreted as a risk-control mechanism. Foundational projects can consume years by committing too early to notation, implementation, or philosophical identity. The outfinite programme should instead require each phase to earn the right to proceed by producing artifacts that can be tested independently of later ambitions.

The first commitment is semantic minimalism. Researchers should define only the distinctions needed to state typed existence, theory locality, frontiers, witnesses, and bridges. Several candidate formalizations should be implemented in parallel when uncertainty is high. The deliverable is not a polished language but a set of small calculi and counterexamples

showing what each can and cannot express. If an existing formal framework captures the requirements with minor extensions, adopting it should count as success rather than defeat.

The second commitment is evidence interoperability. Before reconstructing broad mathematics, the project should show that a common claim can receive evidence from heterogeneous engines and that the resulting certificates can be composed safely. A small arithmetic and program-semantics corpus is enough. The critical result is architectural: the system must preserve the difference between a proof term, an inductive invariant, a bounded-unsatisfiability result, a concrete counterexample, and an approximate numerical enclosure.

The third commitment is mathematical reconstruction. Only after the core survives evidence experiments should the programme attempt sustained work in arithmetic, algebra, analysis, probability, category theory, and non-classical logics. The objective is not to rewrite textbooks. It is to identify which classical results are conservative imports, which obtain more useful frontier-sensitive companions, and which reveal genuine limitations of the outfinite approach. Negative results should shape the boundaries of the new mathematics.

The fourth commitment is executable scientific theory. At this stage the framework must leave the relative safety of mathematical examples and confront data, measurement, numerical models, model uncertainty, and revision. A small number of deeply formalized scientific theories is preferable to broad but shallow coverage. Domain experts should be involved because semantic fidelity is the central risk.

The fifth commitment is AI-native infrastructure. Once the theory representation is stable enough, coding and reasoning agents can automate formalization, proof search, refactoring, literature extraction, and adversarial review. AI should not be used to hide weaknesses in the formalism. On the contrary, agents should be evaluated on whether explicit outfinite structure improves their reliability compared with direct work on prose or conventional formal languages.

The programme should maintain a registry of open foundational questions. Examples include whether existence modes are best expressed by modalities or types, whether frontier composition has a useful algebra, whether category theory provides essential structure or merely elegant description, how empirical evidence should enter the core without weakening deductive soundness, and which proof certificates can be reduced to a small trusted kernel. Each question should have candidate answers and benchmark consequences.

Governance matters because a foundational project can become unfalsifiable through terminology. New concepts should not be added merely because they sound philosophically compatible with outfinitism. Every primitive must justify itself by enabling a theorem, preventing an error, simplifying a bridge, or improving automation. Deprecated concepts should remain documented so that the history of failed ideas is visible rather than silently erased.

A mature research community around the programme would therefore look less like a school defending a doctrine and more like an experimental formal-methods effort. Competing semantics, rival implementations, negative results, and external criticism are assets. The philosophical intuition supplies the direction, but the scientific identity of the project is earned only through precise definitions, reusable theorems, benchmark performance, and the willingness to abandon components that do not survive comparison.

Conclusion. From a philosophy of limits to a candidate science of executable knowledge

Outfinitism begins with a philosophical intuition: reality, computation, observation, and organization are saturated with limits, while mathematical language often treats those limits as external to the ideal objects it studies. The intuition becomes more specific when infinity is examined. Actual infinity is coherent as a theoretical mode of existence within theories that admit completed totalities. Potential infinity captures the openness of a rule, but it can obscure the distinction between logical extensibility and physically realized continuation. The term outfinite is proposed for processes whose specification has no relevant internal stopping point while every realized trace remains finite and every further extension is an event subject to context and frontier.

The research programme becomes more ambitious when this distinction is generalized. Existence itself should be typed when mathematical theories are connected to the world. Theoretical objects, physical objects, computations, processes, phenomena, and collectives do not require the same witnesses or identity relations. A proof inside a theory, an observation, a simulation, and a statistical estimate are not interchangeable forms of truth. Approximation and model validity have boundaries. Formal methods produce different kinds of evidence with different coverage. These distinctions can remain implicit for expert humans, but they become increasingly dangerous when scientific knowledge is generated and manipulated at machine scale.

The proposed response is not to discard classical mathematics. It is to construct a meta-level in which mathematical theories can coexist with explicit commitments. A classical set theory can continue to use actual infinity. Constructive modules can require witnesses. Category theory can organize translations. Algebra can specify reusable structure. Modal, temporal, fuzzy, paraconsistent, and resource-sensitive logics can be used where their semantics fits. Abstract interpretation, deductive verification, symbolic execution, bounded model checking, dataflow analysis, equivalence checking, and other methods can contribute typed evidence without being forced into a false binary classification.

This architecture realizes a technical version of meta-rationality. Different theories can describe the same phenomena under different scales and purposes. Rationality consists not in pretending that one model has eliminated all alternatives, but in controlling how conclusions move between models. Theories are local; bridges are explicit; losses and approximations are recorded; unresolved regions remain unresolved.

Artificial intelligence gives the programme both urgency and feasibility. AI makes scientific and mathematical generation cheap enough that verification, semantic fidelity, and applicability become critical bottlenecks. At the same time, coding agents can automate much of the labor required to formalize literature, generate proof obligations, search for invariants, maintain dependencies, and translate theories into existing verification engines. The correct trust architecture keeps AI outside the final kernel: agents propose, explore, translate, and explain; small deterministic components and explicit evidence decide what is certified.

The long-term vision is executable scientific knowledge. A theory would no longer be only a document for human interpretation. Its operational core would be available as a versioned, machine-checkable artifact connected to implementations and evidence. Computers could

review assumptions, reproduce calculations, detect invalid bridges, compare theory versions, and identify where a conclusion exceeds its validated frontier. Human scientists would remain responsible for concept formation, relevance, and empirical judgment, but they would work with a far more inspectable reasoning substrate.

None of this has yet been demonstrated. That fact is central to the positioning of the programme. Outfinite mathematics is not currently a mature mathematical foundation. Its key concepts remain research hypotheses. Existence types may prove better represented as ordinary dependent types. The outfinite process may collapse into known notions from transition systems and constructive mathematics. Category theory may add elegance without practical value. The empirical layer may prove too context-dependent for a general formal semantics. AI formalization may remain too expensive or semantically unstable.

The programme is credible only if these possibilities are allowed to defeat it. Its next phase must therefore be formal, comparative, and experimental. The semantic core must be defined independently of surface language. Metatheorems must be proven. Classical results must be reconstructed without distortion. Theories must be compiled to established proof systems. Real analysis engines must be integrated. Scientific benchmarks must contain deliberately planted boundary failures. AI-assisted formalization must be compared with strong direct baselines. Independent implementations must agree on certificates.

If those tests fail, the useful components should be retained and the larger philosophical claim narrowed. If they succeed, however, outfinitism may develop beyond a philosophy of limits into a genuine research field: a mathematics of typed existence, open processes, boundaries, theory translation, and finite justification. Its most consequential contribution would then not be the rejection of infinity. It would be a new discipline for stating exactly what kind of thing a theory says exists, what its conclusions depend on, how far those conclusions travel, and where our justified reasoning ends.

A final criterion for success

The ultimate criterion is not whether the term outfinite becomes accepted vocabulary. The programme succeeds if it produces a formal discipline in which limits that are currently left implicit become easier to reason about, and if that discipline improves the reliability of scientific and AI-mediated work. It may succeed while importing large parts of existing mathematics unchanged. It may even succeed if the final implementation is recognized technically as a particular modal type theory, categorical semantics, or evidence calculus rather than as a wholly unprecedented logic. The intellectual contribution would lie in identifying the right problem and organizing existing and new machinery around it.

The most important scientific claim is that applicability is itself mathematizable to a greater degree than current practice assumes. A theorem is not merely a sentence with a truth value; it exists inside a theory, under assumptions, about entities interpreted in particular ways, and it reaches practice through bridges that can fail. A computation is not merely a numerical answer; it is a finite realization under a representation and resource context. An observation is not merely a number; it is produced by an instrument and an interpretation. A collective is not merely a set; it can persist through structured change. Making these relations explicit could provide the missing semantic layer between formal mathematics and executable science.

The strongest practical claim is correspondingly modest. AI systems need not be made infallible. They can be made easier to constrain by moving part of scientific meaning from tacit human context into machine-checkable structure. Generative systems can remain creative and probabilistic while their conclusions are filtered through explicit theory, evidence, and frontier rules. This separation between generative intelligence and mechanical validation may prove more scalable than attempting to obtain reliability from language-model behavior alone.

The programme therefore has two possible legacies. The narrow legacy would be a set of useful techniques for resource-aware proof, theory provenance, and executable scientific models. The broader legacy would be a shift in mathematical foundations toward typed existence, process, and explicit frontiers. The research should be organized so that the narrow result remains valuable even if the broader ambition fails. That asymmetry makes the project worth testing: the strongest philosophical claims are risky, while several intermediate engineering and scientific outcomes can be independently useful.

Appendix A. Evidence profile of major formal and analytical techniques

The following table summarizes how techniques discussed throughout the programme can contribute to an outfinite evidence architecture. The classifications are intentionally conditional. A method's role depends on its semantics, coverage, and implementation, not only on its traditional name.

Technique	Outfinite interpretation and principal limitation
Deductive verification and theorem proving	Produces general logical consequences from explicit specifications. Strongly proof-oriented, but only as good as the specification, formal semantics, and trusted base.
Dependent types	Encode propositions and constraints in types, making some proofs part of type checking. Highly promising for existence modes, but can become difficult to author and automate.
Equivalence and refinement checking	Establish that a transformation, implementation, or representation preserves declared behavior. Central for theory bridges; equivalence must be chosen at the correct observational level.
Abstract interpretation	Computes sound abstractions and invariants that can prove global safety properties. Precision loss and widening can create false alarms; the abstraction relation must itself be justified.
Dataflow analysis	Propagates abstract information across program structure. Can be formally sound for its property, although commonly used for compiler optimization rather than theorem proving.
Taint and information-flow analysis	Tracks abstract flow of information. Its guarantees depend on which explicit, implicit, reflective, environmental, and side-channel flows are modeled.
Symbolic execution	Executes families of inputs through symbolic path conditions and produces strong counterexamples and local proofs. Usually incomplete on large programs because of path explosion unless summaries or finite completeness arguments are available.
Concolic execution	Combines concrete runs with symbolic constraints to drive new paths. Excellent for witness generation and coverage, but normally does not prove absence of errors.
Bounded model checking	Gives exact formal results inside the chosen depth and model. It becomes a global proof only when a completeness threshold or another lifting argument is established.

Technique	Outfinite interpretation and principal limitation
CEGAR	Alternates abstraction, counterexample analysis, and refinement. Can converge to a proof or real counterexample, but termination is not guaranteed in general.
Partial evaluation	Specializes programs with respect to known information. Primarily a transformation; it participates in proof when semantic equivalence with the original program is certified.
Symbolic interpretation and rewriting	Simplifies or transforms expressions under formal rules. Correctness depends on the semantics of those rules and on preserving types, representations, and theory context.
Fuzzing	Searches for concrete failures and can produce valuable witnesses. Successful testing does not normally establish absence of failures beyond the explored inputs and behaviors.

Appendix B. Core conceptual vocabulary

Term	Working meaning in the research programme
Outfinitism	A philosophical position and research programme centered on limits, typed existence, process, and explicit boundaries of justification.
Outfinite process	A finitely specified process with no relevant internal logical terminal condition, whose realized traces are finite and whose continuation depends on actual extension events and contexts.
Existence mode	A formally relevant way in which an entity is admitted or witnessed, such as theoretical, computational, physical, processual, phenomenal, or collective existence.
Frontier	A structured boundary reached by a realization, analysis, observation, model, or theory bridge; it records why justification or continuation does not currently extend further.
Theory-local validity	The principle that a result is valid relative to the theory, semantics, assumptions, and bridges under which it was established.
Bridge	An explicit transformation or relation connecting theories, representations, existence modes, or levels of description, together with preservation and approximation obligations.
Evidence object	A finite, typed artifact that supports a claim, such as a proof term, witness, counterexample, invariant, solver certificate, measurement record, or statistical result.
Residual	The part of a question, state space, uncertainty, or approximation not resolved by the current evidence.
Meta-rationality	Controlled use of multiple models or theories under explicit contexts, with disciplined switching and translation rather than unmarked model conflation.
Executable science	The research goal of representing the operational core of scientific theories as machine-manipulable, reviewable, and partially executable artifacts linked to evidence and implementation.

Appendix C. Consolidated hypotheses and direct falsification routes

Research hypothesis	Direct route to falsification
Typed existence prevents important semantic category errors.	Compare expert and AI reasoning with and without existence typing on adversarial theory-to-reality tasks; no measurable reduction in errors weakens the hypothesis.
Outfinite process theory adds value beyond ordinary bounded transition systems.	Formalize both accounts on the same problems; if no new compositional laws, diagnostics, or usability gains appear, reduce the claim.
Explicit frontiers prevent bounded analyses from being overgeneralized.	Seed defects just beyond search bounds and test whether the framework preserves the open region instead of reporting global safety.
Typed evidence allows heterogeneous formal methods to compose safely.	Integrate proof, abstract analysis, symbolic execution, and model checking; any silent promotion of bounded or approximate results is a failure.
Category-theoretic structure improves theory translation and process composition.	Compare direct and categorical formalizations; if categories add terminology without proof reuse or clarity, keep them optional.
Non-classical logics improve treatment of processes, graded phenomena, and contradictory evidence.	Use benchmarks where probability, fuzzy degree, contradiction, and temporal status diverge; lack of error reduction weakens the claim.
Outfinite mathematics improves executable scientific formalization.	Compare against direct proof-assistant and workflow-based formalization on complete scientific models using expert-scored semantic fidelity and review tasks.
AI can lower the cost of building the formal ecosystem without becoming part of the trusted base.	Measure human correction, formalization cost, verification success, and semantic drift in coding-agent workflows.
The foundation improves AI reasoning about applicability.	Compare agents on scope, model-reality, finite-asymptotic, and theory-bridge tasks; no improvement means the AI argument is unsupported.
Executable theory artifacts can create economic value through reuse and assurance.	Measure formalization, maintenance, review, and reuse costs in realistic pilots; if lifecycle cost exceeds alternatives without assurance gain, the economic thesis fails.

Bibliography

- Aristotle. *Physics*, Books III and IV. In *The Complete Works of Aristotle*, edited by Jonathan Barnes. Princeton University Press, 1984.
- Belnap, Nuel D., Jr. "A Useful Four-Valued Logic." In *Modern Uses of Multiple-Valued Logic*, edited by J. Michael Dunn and George Epstein. D. Reidel, 1977.
- Biere, Armin, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. "Symbolic Model Checking without BDDs." In *Tools and Algorithms for the Construction and Analysis of Systems*, LNCS 1579. Springer, 1999, pp. 193-207.
- Birkhoff, Garrett. "On the Structure of Abstract Algebras." *Proceedings of the Cambridge Philosophical Society* 31, no. 4, 1935, pp. 433-454.
- Bishop, Errett, and Douglas Bridges. *Constructive Analysis*. Springer, 1985.
- Blok, Willem J., and Don Pigozzi. *Algebraizable Logics*. *Memoirs of the American Mathematical Society* 396. American Mathematical Society, 1989.
- Brouwer, L. E. J. "The Unreliability of the Logical Principles." 1908; later translations in collections on intuitionism and philosophy of mathematics.
- Cantor, Georg. *Contributions to the Founding of the Theory of Transfinite Numbers*. Translated by Philip E. B. Jourdain. Open Court, 1915.
- Clarke, Edmund M., and E. Allen Emerson. "Design and Synthesis of Synchronization Skeletons Using Branching-Time Temporal Logic." In *Logic of Programs*, 1981, pp. 52-71.
- Clarke, Edmund M., Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. "Counterexample-Guided Abstraction Refinement." In *Computer Aided Verification*, LNCS 1855. Springer, 2000, pp. 154-169.
- Cousot, Patrick, and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *Proceedings of the 4th ACM Symposium on Principles of Programming Languages*, 1977, pp. 238-252.
- Denning, Dorothy E. "A Lattice Model of Secure Information Flow." *Communications of the ACM* 19, no. 5, 1976, pp. 236-243.
- Eilenberg, Samuel, and Saunders Mac Lane. "General Theory of Natural Equivalences." *Transactions of the American Mathematical Society* 58, no. 2, 1945, pp. 231-294.
- Euclid. *The Thirteen Books of the Elements*. Translated and edited by Thomas L. Heath. Book IX, Proposition 20. Cambridge University Press, second edition, 1926.
- Floyd, Robert W. "Assigning Meanings to Programs." *Proceedings of Symposia in Applied Mathematics* 19, 1967, pp. 19-32.
- Galilei, Galileo. *Discourses and Mathematical Demonstrations Relating to Two New Sciences*. 1638.
- Girard, Jean-Yves. "Linear Logic." *Theoretical Computer Science* 50, no. 1, 1987, pp. 1-101.
- Godefroid, Patrice, Nils Klarlund, and Koushik Sen. "DART: Directed Automated Random Testing." *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2005, pp. 213-223.
- Gödel, Kurt. "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I." *Monatshefte für Mathematik und Physik* 38, 1931, pp. 173-198.
- Harel, David, Dexter Kozen, and Jerzy Tiuryn. *Dynamic Logic*. MIT Press, 2000.
- Hilbert, David. "On the Infinite." Lecture of 1925, published 1926; English translations in standard collections on logic and foundations of mathematics.
- Hoare, C. A. R. "An Axiomatic Basis for Computer Programming." *Communications of the ACM* 12, no. 10, 1969, pp. 576-580 and 583.
- Kildall, Gary A. "A Unified Approach to Global Program Optimization." *Proceedings of the 1st Annual ACM Symposium on Principles of Programming Languages*, 1973, pp. 194-206.

- King, James C. "Symbolic Execution and Program Testing." *Communications of the ACM* 19, no. 7, 1976, pp. 385-394.
- Kripke, Saul A. "Semantical Analysis of Modal Logic I: Normal Modal Propositional Calculi." *Zeitschrift für mathematische Logik und Grundlagen der Mathematik* 9, no. 5-6, 1963, pp. 67-96.
- Lawvere, F. William. "An Elementary Theory of the Category of Sets." *Proceedings of the National Academy of Sciences* 52, no. 6, 1964, pp. 1506-1511.
- Lawvere, F. William. "Adjointness in Foundations." *Dialectica* 23, nos. 3-4, 1969, pp. 281-296.
- Martin-Löf, Per. *Intuitionistic Type Theory*. Notes by Giovanni Sambin. Bibliopolis, 1984.
- Kung, Po-Nien, Linfeng Song, Dawsen Hwang, Jinsung Yoon, Chun-Liang Li, Simone Severini, Mirek Olšák, Edward Lockhart, Quoc V. Le, Burak Gokturk, Thang Luong, Tomas Pfister, and Nanyun Peng. "LEAP: Supercharging LLMs for Formal Mathematics with Agentic Frameworks." *arXiv:2606.03303*, 2026.
- Meadows, Jordan, Lan Zhang, and Andre Freitas. "FormalScience: Scalable Human-in-the-Loop Autoformalisation of Science with Agentic Code Generation in Lean." *arXiv:2604.23002*, 2026.
- Necula, George C. "Proof-Carrying Code." *Proceedings of the 24th ACM Symposium on Principles of Programming Languages*, 1997, pp. 106-119.
- Nelson, Edward. *Predicative Arithmetic*. Princeton University Press, 1986.
- Nie, Tianjiao, Ao Zhang, Yusen Tang, Damiano Testa, Shing-Tung Yau, Peng Li, and Yuan Zhou. "FormaTheoria: Constructing Large-Scale Lean Theories from Mathematical Literature - Toward the Formalization of the Classification of Finite Simple Groups." *arXiv:2608.10894*, 2026.
- Pnueli, Amir. "The Temporal Logic of Programs." *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, 1977, pp. 46-57.
- Pnueli, Amir, Michael Siegel, and Eli Singerman. "Translation Validation." In *Tools and Algorithms for the Construction and Analysis of Systems*, LNCS 1384. Springer, 1998, pp. 151-166.
- Pratt, Vaughan R. "Semantical Considerations on Floyd-Hoare Logic." *Proceedings of the 17th Annual Symposium on Foundations of Computer Science*, 1976, pp. 109-121.
- Queille, Jean-Pierre, and Joseph Sifakis. "Specification and Verification of Concurrent Systems in CESAR." *International Symposium on Programming*, 1982, pp. 337-351.
- Rutten, J. J. M. M. "Universal Coalgebra: A Theory of Systems." *Theoretical Computer Science* 249, no. 1, 2000, pp. 3-80.
- Sen, Koushik, Darko Marinov, and Gul Agha. "CUTE: A Concolic Unit Testing Engine for C." *Proceedings of the 10th European Software Engineering Conference and the 13th ACM SIGSOFT Symposium on Foundations of Software Engineering*, 2005, pp. 263-272.
- Tarski, Alfred. "A Lattice-Theoretical Fixpoint Theorem and Its Applications." *Pacific Journal of Mathematics* 5, no. 2, 1955, pp. 285-309.
- Turing, Alan M. "On Computable Numbers, with an Application to the Entscheidungsproblem." *Proceedings of the London Mathematical Society*, second series, 42, 1936, pp. 230-265.
- Weyl, Hermann. *Das Kontinuum: Kritische Untersuchungen über die Grundlagen der Analysis*. Veit, 1918.
- Zadeh, Lotfi A. "Fuzzy Sets." *Information and Control* 8, no. 3, 1965, pp. 338-353.
- Zermelo, Ernst. "Untersuchungen über die Grundlagen der Mengenlehre I." *Mathematische Annalen* 65, 1908, pp. 261-281.

End of research programme